DATA PREPARATION FOR MACHINE LEARNING AND AI · TEXT DATA MODULE

Text pre-processing in the LLM era

Text data, class 2 of 2. Subword tokenizers and who they tax, web-scale corpus curation, instruction data, chunking for RAG, and clinical text with learned de-identification.

Asst. Prof. Thanapong Intharah, PhD

Visual Intelligence Laboratory · Khon Kaen University

A curated compilation of openly published university teaching materials. Every section names its source; original slides are shown unmodified.

Today, in three hours

0:00–0:20	**D0 · Why data preparation decides what an LLM learns.** Scaling laws, tokens as the lever you hold, ten thousand times more text, quality over quantity.	SLP3 intro + CS224N L7 + CS329X + EPFL L3 + authored
0:20–0:45	**D1 · Subword tokenization.** How language models see text, BPE trained by hand, SentencePiece, glitch tokens.	CS224N L14 + SLP3 + authored
0:45–1:00	**D2 · Tokenization beyond English.** Cross-lingual transfer, the curse of multilinguality, the Thai premium measured live.	CS224N L14 + authored
1:00–1:10	*Break*	
1:10–1:25	**E1 · Where pretraining data comes from.** Common Crawl, C4, The Pile, BookCorpus, copyright and memorization.	SLP3 LLM + CS224N L7 + CS329X
1:25–1:50	**E2 · Cleaning at web scale.** Deduplication, quality filters, privacy, data age, packing, mixing; MinHash and Gopher rules live.	EPFL EE-628 L3 (CC BY-NC-SA) + authored
1:50–2:05	**E3 · Contamination, ownership, biased filters.** Benchmark leakage, robots.txt, what "quality" quietly excludes.	CS224N L11 + CS329X + authored
2:05–2:25	**F1 · Instruction data.** Self-Instruct, Alpaca, FLAN, selection (LIMA to LESS), synthetic data and model collapse.	CS224N L8 + EPFL L4 + authored
2:25–2:40	**F2 · RAG data preparation.** Chunking in code, late chunking, contextual retrieval, the contested cost of semantic chunking.	SLP3 IR + CS224N L10 + authored
2:40–3:00	**F3 · Clinical text, learned era.** Context, model-based de-identification, Thai notes and PDPA.	MIT OCW 6.S897 L8 + authored

PART D0 · 0:00–0:20

Why data preparation decides what a language model learns

Scaling laws made training tokens one of three levers, and the only one a data team controls. Before any tokenizer or filter: how much text, from where, counted how, and why quality started beating quantity.

SOURCE DECKS

Large Language Models, introduction (SLP3 slides, Jurafsky and Martin, Aug 2026 and Aug 2025 versions) · **Lecture 7: Pretraining** (CS224N W2026) · **Lecture 5: Data, Data, and Data** (CS 329X, Held) · **Lecture 3: Data** (EPFL EE-628, CC BY-NC-SA); framed by authored slides.

© the respective authors · shown unmodified · selected pages

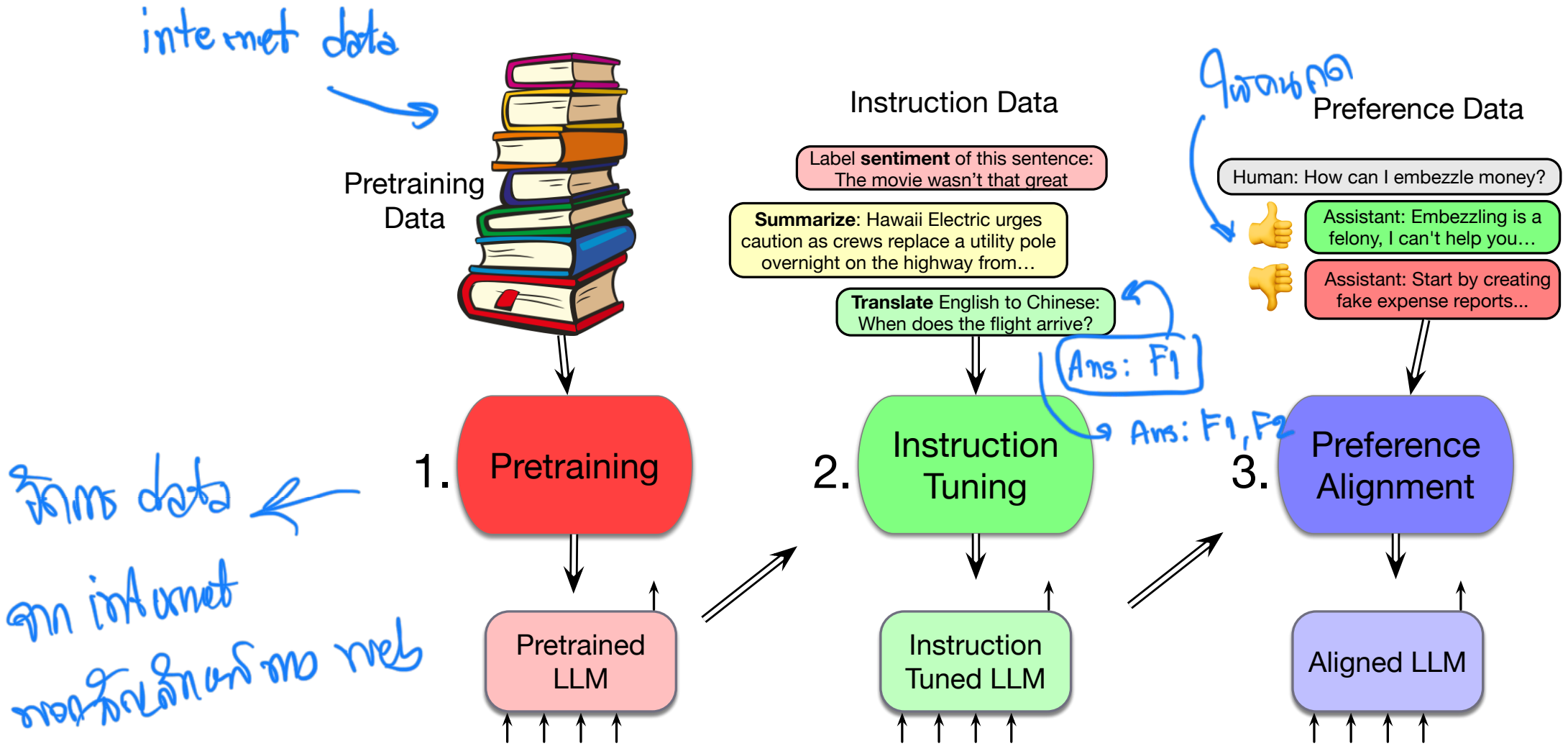
Three levers, and the one you actually hold

LEVER (SCALING LAWS)	WHO CONTROLS IT	PREPROCESSING QUESTION
parameters N	the architecture team, the budget	none
compute C	the GPU budget	none, except not wasting it on junk tokens
training tokens D	**the data team: you**	how many, from where, in which language, deduplicated how, tokenized how, mixed how

Chinchilla (Hoffmann et al. 2022): for a fixed compute budget, optimal parameters and optimal tokens grow together, roughly 20 tokens per parameter.

- In class 1 the model consumed **word counts**; an LLM consumes **token counts at internet scale**, and its knowledge is a compression of whatever text survived preprocessing.
- Three training stages (SLP3 slide that follows) mean **three datasets**, each with its own preparation problem: a web corpus, an instruction set, a preference set.
- The rest of today follows the token: how it is made (D1, D2), where it comes from (E1), what is removed and why (E2, E3), how the small datasets are built (F1), and how documents are cut for retrieval (F2).

Three stages of training in LLMs



Scaling laws

LLM performance goes up as these 3 factors increase:

- model size (the number of parameters)
- training dataset size (in tokens)
- amount of compute used for training

Scaling laws

For a fixed amount of compute C

Optimal # of parameters N

Optimal amount of data D

Should go up equally as a power law of compute

$$N_{opt} \propto C^{\frac{1}{2}} \qquad D_{opt} \propto C^{\frac{1}{2}}$$

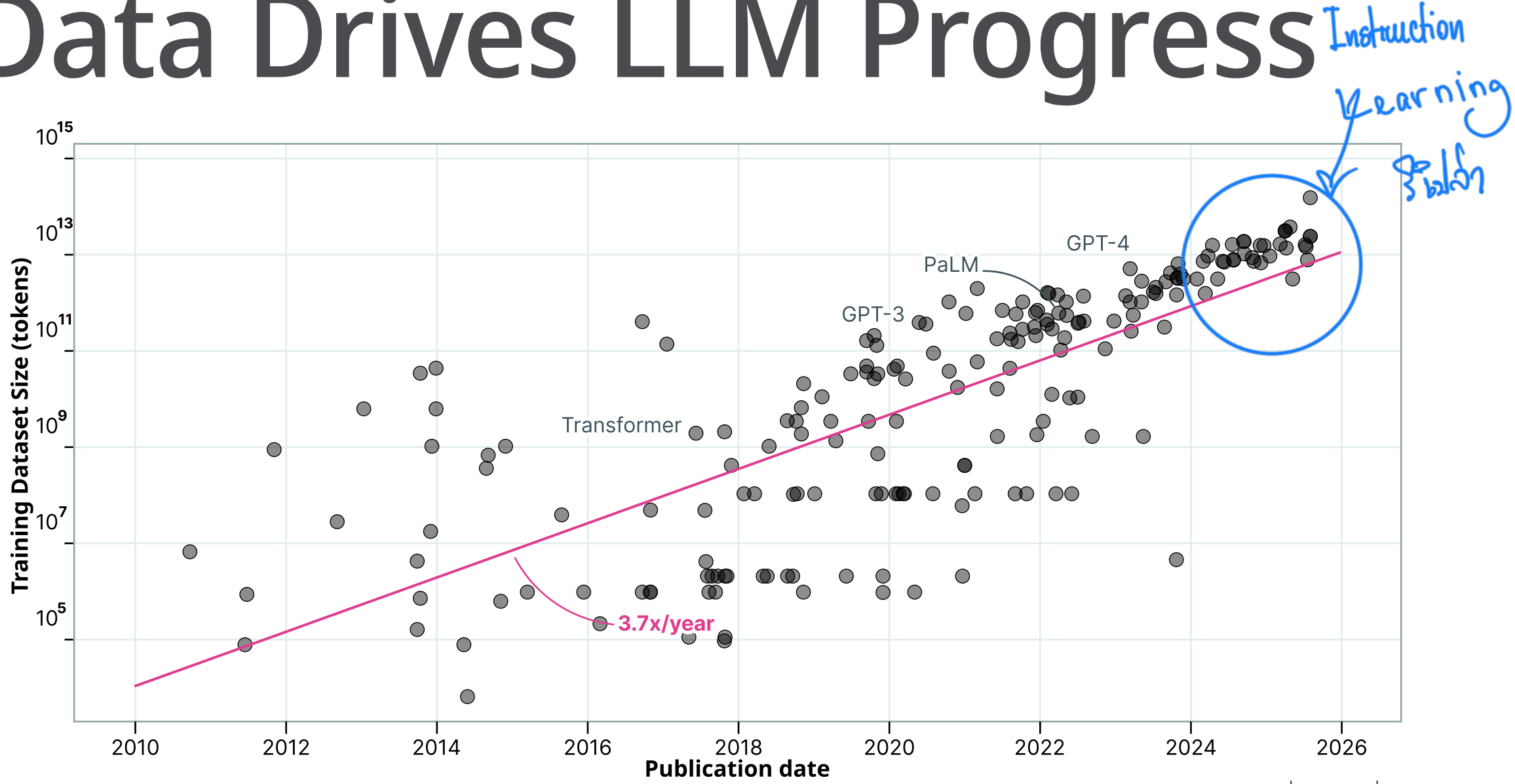
Scaling laws

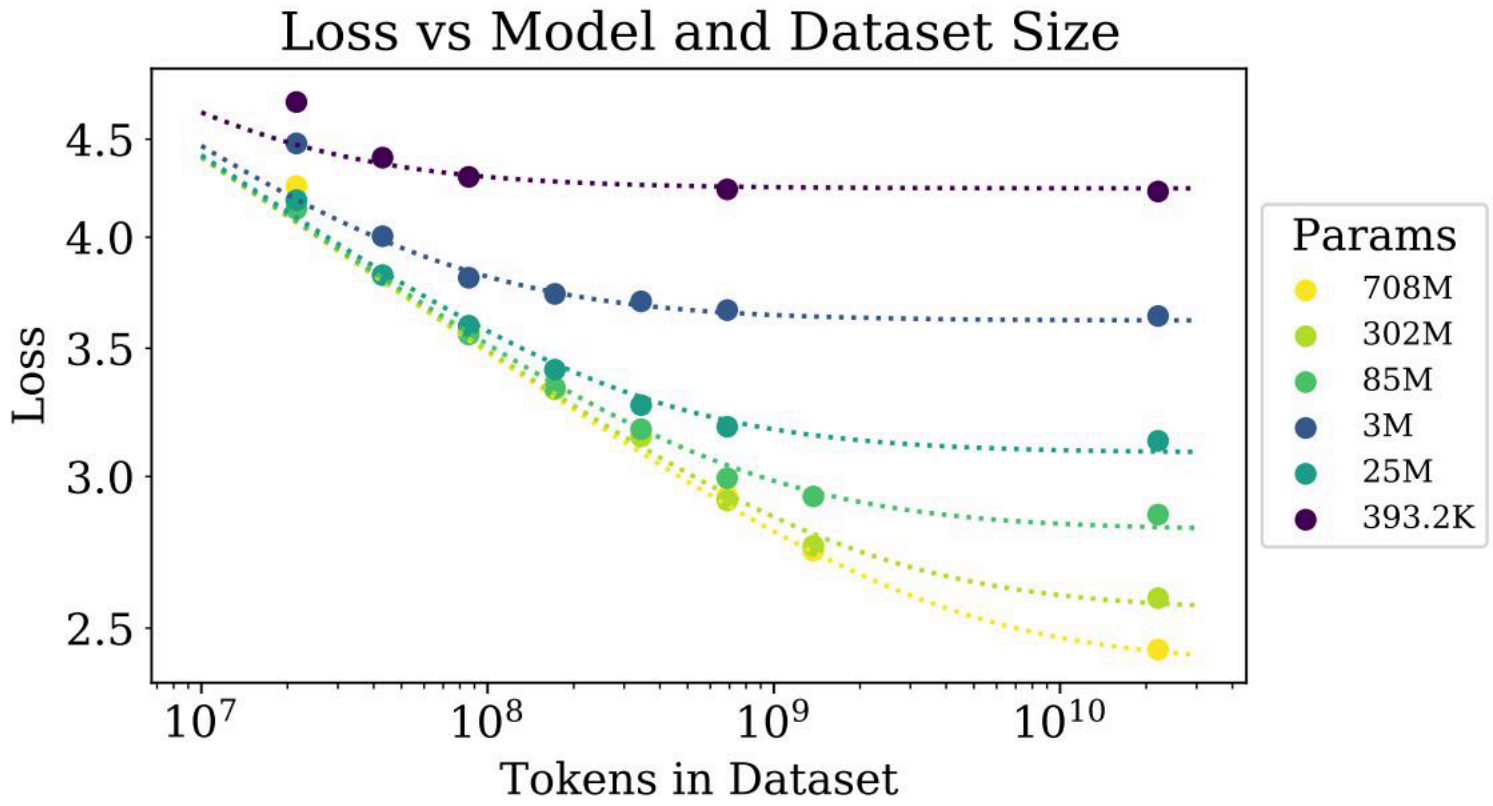
Explain why models get bigger

- For good (better performance)
- And ill (more energy, water, $$$) and expensive to run inference.

Also explain that we can improve the same size model by increasing compute and and training data

Data Drives LLM Progress

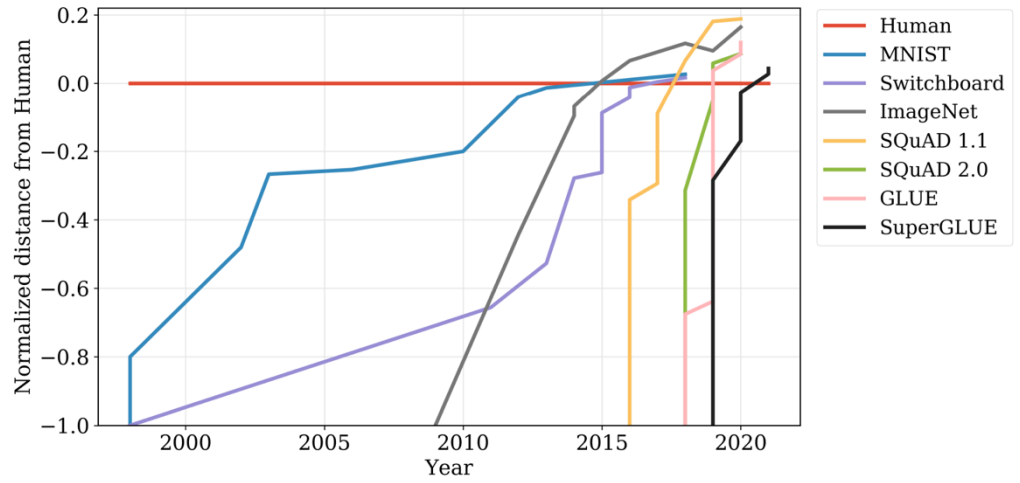


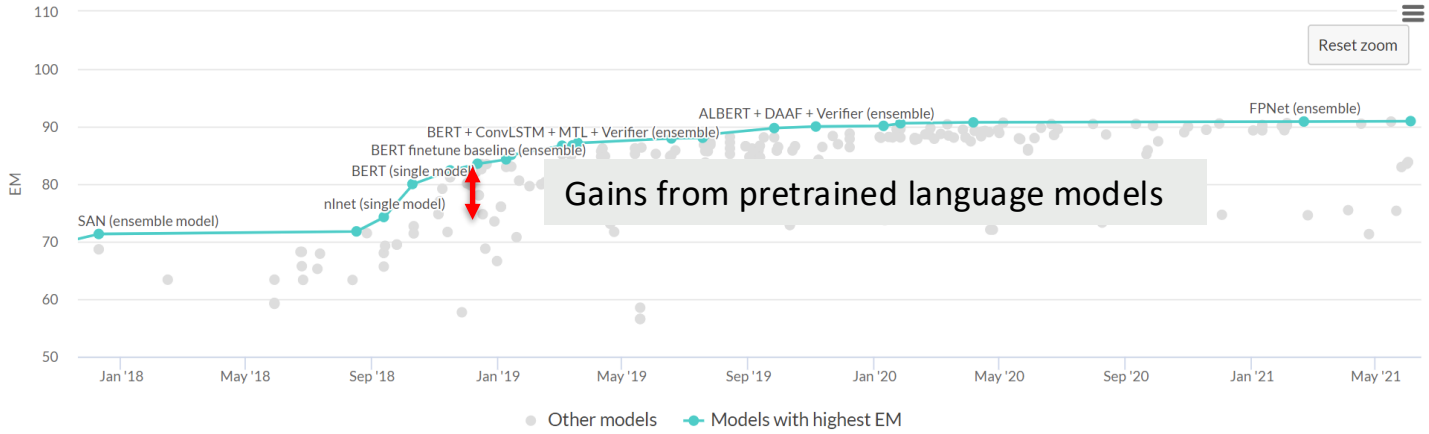


Kaplan et al. 2020

Increasing data is a reliable lever

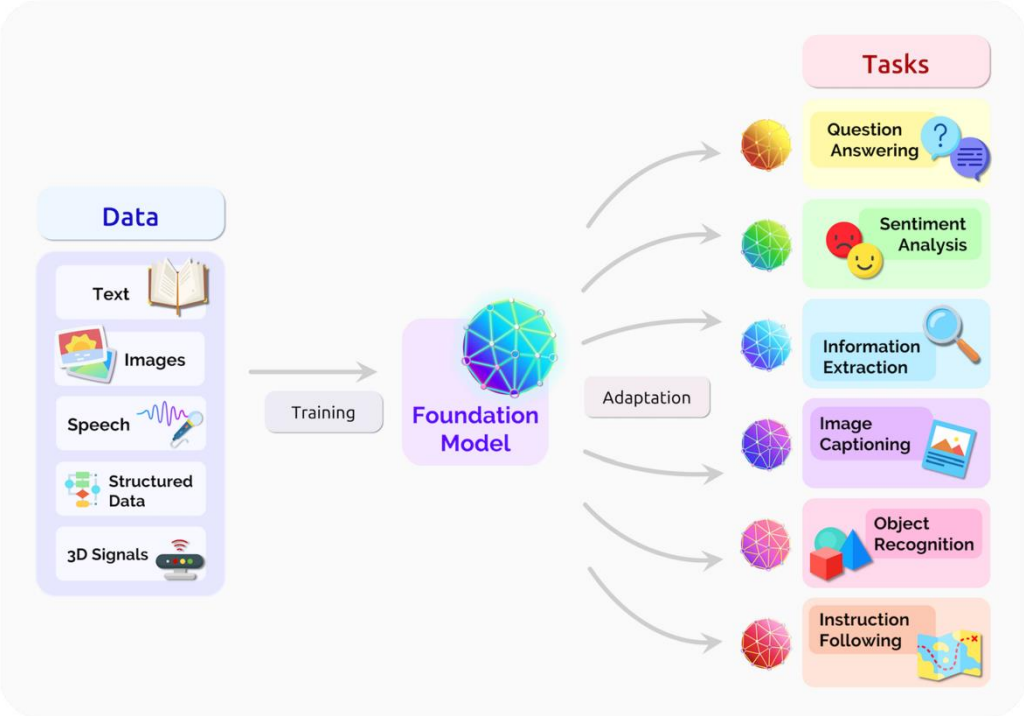
The pretraining revolution





Pretraining has had a major, tangible impact on how well NLP systems work

Pretraining – scaling unsupervised learning on the internet



Key ideas in pretraining

- Make sure your model can process large-scale, diverse datasets
- Don’t use labeled data (otherwise you can’t scale!)
- Compute-aware scaling

Fundamental intuition of large language models

Text contains enormous amounts of knowledge

Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much

Intuition from humans

Vocabulary size of young adult speakers of American English: 30,000 to 100,000 words

Children learn **7 to 10 words a day** to get to that level by 20 years old

How do they do it?

Reading! (Laundauer and Dumais 1997)

What is this text that models pretrain on?

For proprietary models, we don't know!

Open LLMs are mainly trained on the web

CommonCrawl, snapshots of the entire web produced by the non-profit Common Crawl with billions of pages

Fineweb corpus (2025)

18.5 trillion tokens of English

From 96 CommonCrawl dumps from 2013-2024.

Dolma corpus: 3 trillion tokens of English

Source	Doc Type	UTF-8 bytes *(GB)*	Documents *(millions)*	Unicode words *(billions)*	Llama tokens *(billions)*
Common Crawl	web pages	9,812	3,734	1,928	2,479
GitHub	code	1,043	210	260	411
Reddit	social media	339	377	72	89
Semantic Scholar	papers	268	38.8	50	70
Project Gutenberg	books	20.4	0.056	4.0	6.0
Wikipedia, Wikibooks	encyclopedic	16.2	6.2	3.7	4.3
Total		**11,519**	**4,367**	**2,318**	**3,059**

Motivation: Data as a resource

The world's most valuable resource is no longer oil, but data

The data economy demands a new approach to antitrust rules

The "it" in AI models is the dataset.

Posted on June 10, 2023 by jbetker

I've been at OpenAI for almost a year now. In that time, I've trained a **lot** of generative models. More than anyone really has any right to train. As I've spent these hours observing the effects of tweaking various model configurations and hyperparameters, one thing that has struck me is the similarities in between all the training runs.

It's becoming awfully clear to me that these models are truly approximating their datasets to an incredible degree. What that means is not only that they learn what it means to be a dog or a cat, but the interstitial frequencies between distributions that don't matter, like what photos humans are likely to take or words humans commonly write down.

What this manifests as is – trained on the same dataset for long enough, pretty much every model with enough weights and training time converges to the same point. Sufficiently large diffusion conv-unets produce the same images as ViT generators. AR sampling produces the same images as diffusion.

This is a surprising observation! It implies that model behavior is not determined by architecture, hyperparameters, or optimizer choices. It's determined by your dataset, nothing else. Everything else is a means to an end in efficiently delivery compute to approximating that dataset.

Then, when you refer to "Lambda", "ChatGPT", "Bard", or "Claude" then, it's not the model weights that you are referring to. It's the dataset.

Figure: (Left) Economist 2017 article on the importance of data. (Right) The relative importance of data within the model.

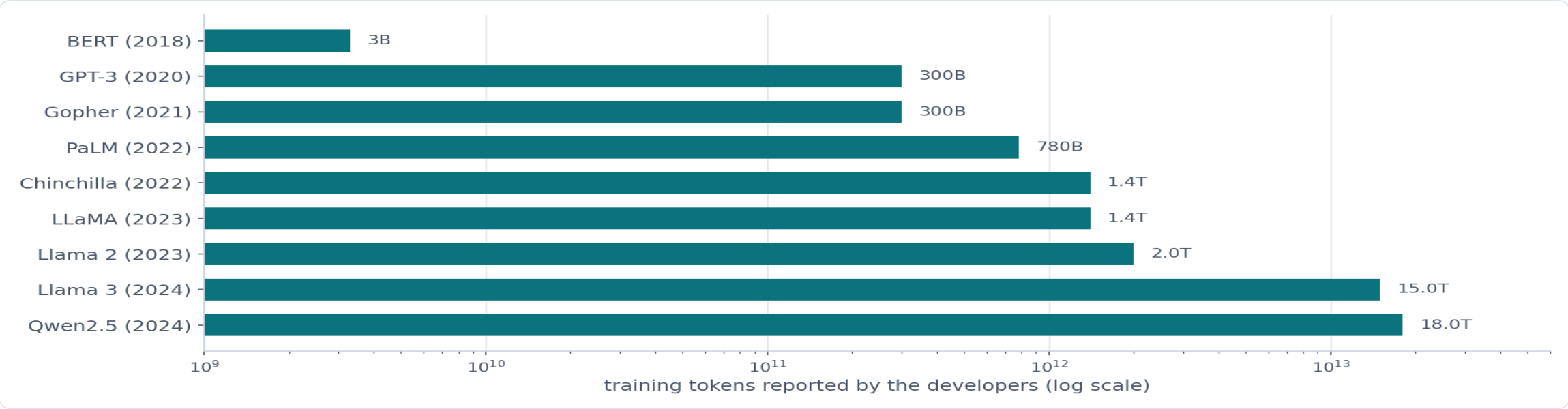
- Architectures determine
  - inference resources
  - inference capabilities
- Algorithms determine
  - training resources
  - generalization
- Data determines
  - generalization
  - training resources

Motivation for data selection [64, 6]

- Why does data selection matter?
  - Costs: Training on large data is computationally expensive and produces significant carbon emissions
  - Quality over quantity: Smaller, high-quality datasets can yield better performance
  - Bias mitigation: Real-world datasets are biased; selection can help address imbalances
  - Robustness: Selection can reduce impact of corrupted examples and noisy data

- We will cover the following approaches in the sequel:
  - Data masking: Focus computation on informative tokens/samples.
  - Document packing: Optimize computational efficiency during training / reduce waste.
  - Data ordering: Structure training sequences for improved learning.

Ten thousand times more text in six years



- **Tokens are the unit of everything**: compute (about 6 N D floating-point operations), cost (API billing), context (what fits in a prompt), and data-constrained scaling (repeating data past a few epochs stops helping).
- So the first question about any corpus is **how many tokens, under which tokenizer**; words are not tokens, and class 1's word counts do not transfer. Figures as reported by the developers (BERT's in words); verify before quoting.

```python
import tiktoken
enc = tiktoken.get_encoding("o200k_base")
n = sum(len(enc.encode(d)) for d in corpus)
# report per source, per language,
# per document-length bucket
```

Each failure of an LLM has a preprocessing address

SYMPTOM IN THE MODEL	CAUSE IN THE DATA	PREPROCESSING FIX	TODAY
regurgitates training text, leaks addresses and emails	duplicated documents, scraped PII	deduplication at three levels, PII filtering	E2
wastes capacity on menus, spam, SEO filler	boilerplate and low-quality pages	extraction, heuristic and classifier quality filters	E2
suspiciously high benchmark scores	test sets inside the crawl	decontamination before training	E3
Thai costs 2 to 4 times more and works worse	English-heavy tokenizer and mix	multilingual vocabulary, language-aware mixing	D2, E2
weak on code, maths, or medicine	domain under-represented in the mix	data mixing, mid-training curricula	E2, F1
cannot follow instructions, or follows them blandly	instruction data absent, noisy, or all alike	instruction data generation, filtering, selection	F1
cites the wrong passage in RAG	chunks cut mid-clause, no provenance	chunking strategy, metadata, evaluation on real queries	F2

The same lesson as the image classes: the model did not fail, the pipeline did, and the fix is upstream of training.

Throwing data away is now how you make models better

92%

of FineWeb removed by an educational-quality classifier to make FineWeb-Edu; MMLU 33 to 37, ARC 46 to 57 in the authors' ablations (Penedo et al., NeurIPS 2024)

3.04%

of C4 near-duplicate, one sentence 60,000 times; deduplicated models emit memorized text 10 times less often (Lee et al., ACL 2022)

1k

curated examples were enough to align a 65B model in LIMA (Zhou et al., NeurIPS 2023); AlpaGasus kept 9k of Alpaca's 52k and trained 5.7 times faster

15x

tokenization-length disparity across languages for the same content (Petrov et al., NeurIPS 2023): the preprocessing tax on non-English users

- Data-centric AI (course week 1) is the frame: with architectures converging, **curation choices drive performance at least as much as model choices**, and unlike compute they are reproducible and publishable (Dolma, FineWeb, and DCLM publish their full recipes).
- Every number above is the proposers' own measurement; the course's standing caveat applies, and the take-home asks you to reproduce the cheap ones on your corpus.
- The counting questions for an LLM corpus: **tokens per source and language, duplicate rate, contamination overlap, tokens per word per language, document length versus context window**. Each has a slide today.

PART D1 · 0:20–0:45

Subword tokenization: how language models see text

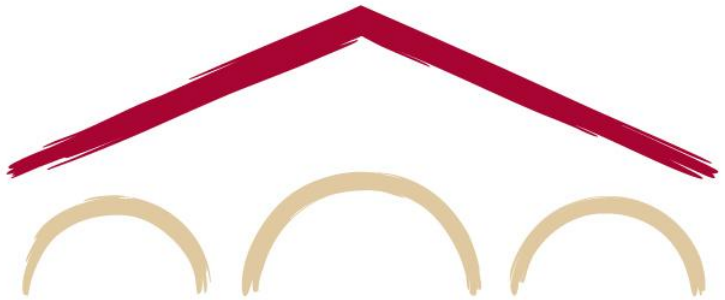
Byte pair encoding trained step by step, why it became the standard, and the strange failures it causes: spelling, glitch tokens, whitespace conventions.

SOURCE DECKS

Lecture 14: Tokenization and Multilinguality (guest lecture, Julie Kallini), CS224N Natural Language Processing with Deep Learning, Winter 2026, Stanford University · then **Words and Tokens**, BPE section, SLP3 slides (Jurafsky and Martin). https://web.stanford.edu/class/cs224n/ · https://web.stanford.edu/~jurafsky/slp3/slides/

© the authors · shown unmodified · selected pages

Natural Language Processing with Deep Learning CS224N/Ling284



Guest Lecture: **Julie Kallini**

Lecture 14: Tokenization and Multilinguality

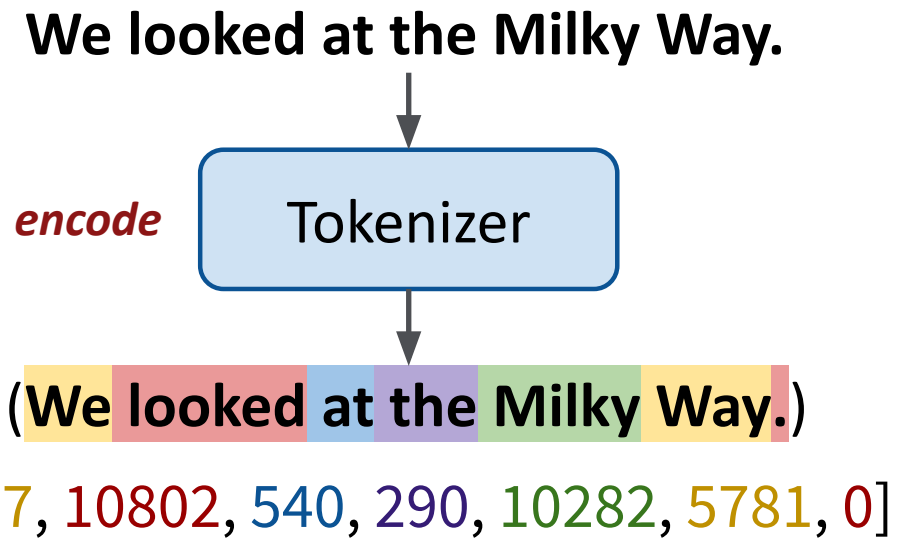
Agenda for Today

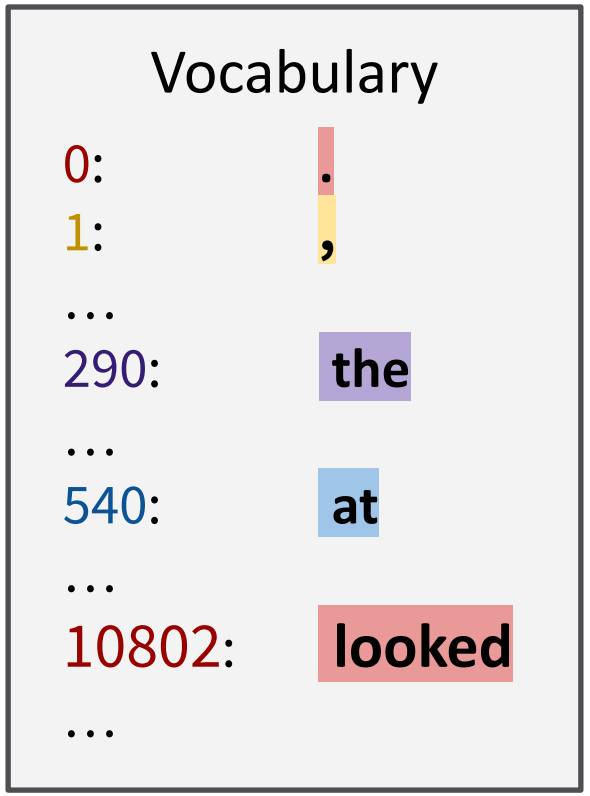
1. **Tokenization approaches:** Word, character/byte, and subword tokenization
2. **Byte Pair Encoding (BPE):** Training algorithm and walkthrough
3. **Case studies:** Spelling, glitch tokens, and where tokenization breaks down
4. **Multilinguality:** Languages of the world, cross-lingual transfer, and fairness
5. **Multilingual tokenization:** Why it's important and what the challenges are

How do language models see text?

For a language model:

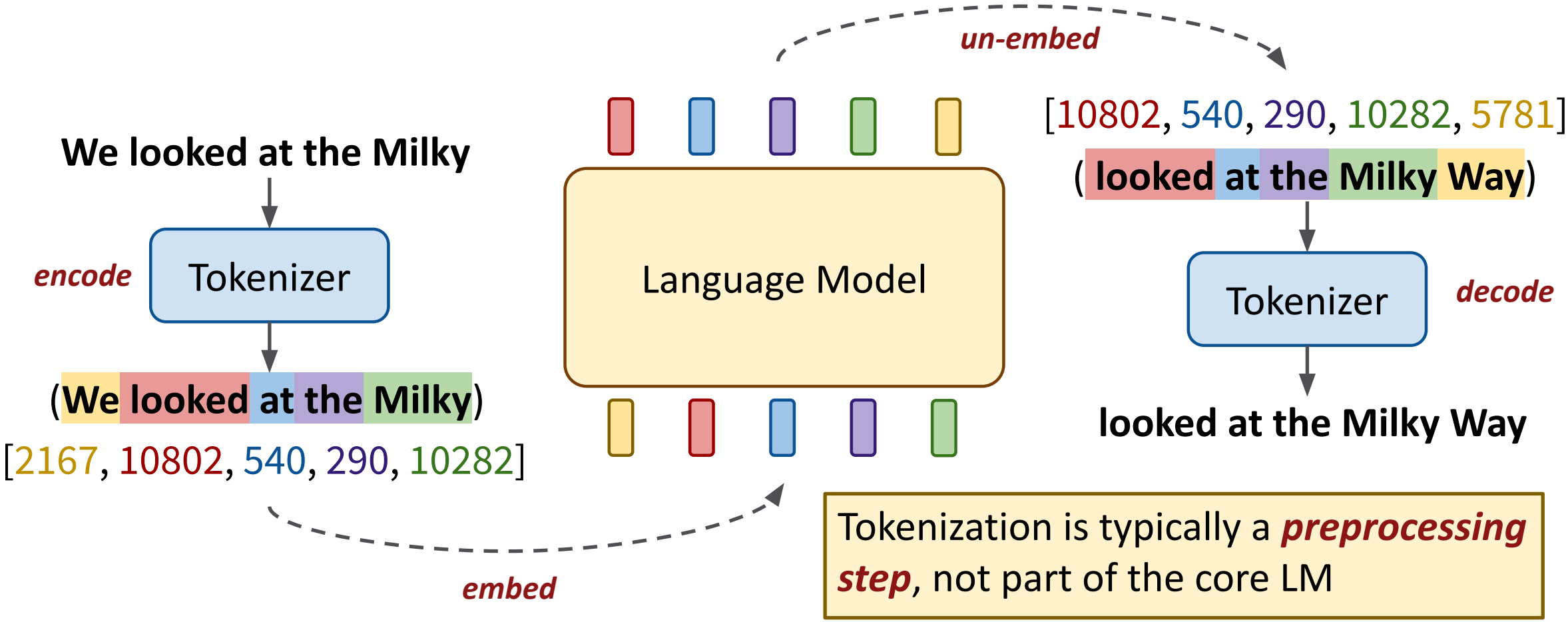
- The fundamental units of language are ***tokens***
- ***Tokenization*** is the process of breaking text into tokens
- A ***tokenizer*** translates between text and a sequence of tokens that a language model (LM) learns over
- The ***vocabulary*** V is the set of known tokens





How do language models see text?

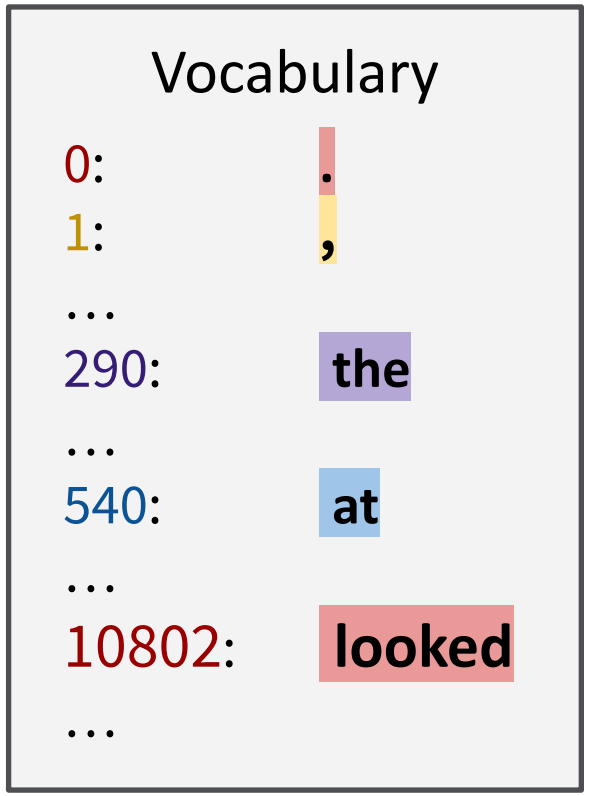
Tokenization influences how LMs learn from and process text, before they have even been trained



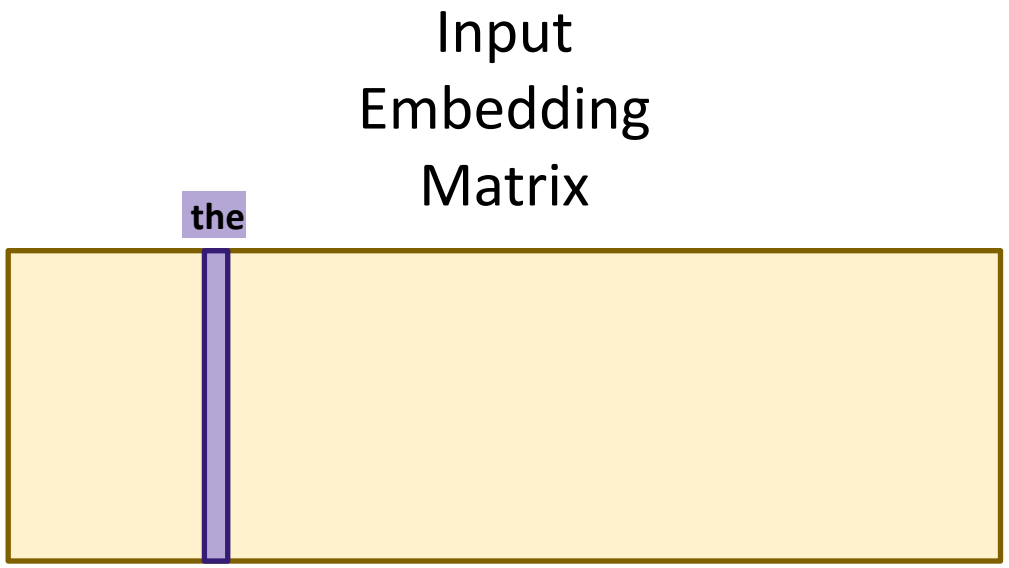
Tokenization is typically a ***preprocessing step***, not part of the core LM

How do language models see text?

Tokenization influences the efficiency of LMs, both in terms of **sequence lengths** and **parameter counts**



$$|V| = 32,000$$



$$\mathbb{R}^{d_{\text{model}} \times |V|}$$

Word tokenization

As a first tokenization scheme,
let's consider simple ***word tokenization***

We'd sat back on the grass, and time flew by as we looked at the Milky Way.

❌ **Cons:**

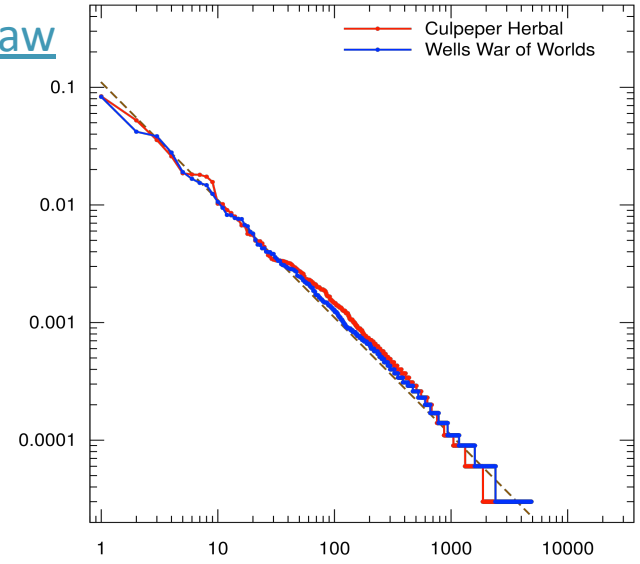
- How do we deal with unknown words? The ***out-of-vocabulary (OOV)*** problem
  - **Solution 1:** use regular expressions/other rule-based methods to preprocess text
    - Typos: **importaNt** ⟶ **important**
    - Regularize spellings: **ESPRESSSOOO** ⟶ **ESPRESSO**
  - **Solution 2:** use a single token for unknown words (**UNK**)
    - Many words would often get mapped to **UNK** 😬

Zipf's law is importaNt in NLP. ⟶ **UNK law isUNK in NLP.**
see Sutskever et al. 2014 & 2015

Corpus	Types = $\|V\|$
Shakespeare	31 thousand
Brown corpus	38 thousand
Switchboard telephone conversations	20 thousand
COCA	2 million
Google n-grams	13 million

Jurafsky & Martin Ch.2

Zipf's Law



Character/byte tokenization (aka “tokenizer-free”)

Another simple tokenization scheme:
character/byte tokenization

We’d sat back on the grass, and time flew by as we looked at the Milky Way.

We’ll assume:

- **Tokens** = individual bytes/characters in the text
  - Note: the distinction between bytes and characters is relevant
    - 😀 = 1 character, 4 bytes; 地 = 1 character, 3 bytes
    - Byte and character-level models tend to be grouped together
- **Vocabulary** V = {a, b, c, …, z, A, B, C, …, Z} (+ spaces, punctuation, numbers, etc.)

Aside: should we call these *tokenizer-free*? See Arnett’s blog

Byte/character-level models: CharBERT, CANINE, Charformer, ByT5

✅ **Pros:**
- Small vocabulary size
- Complete coverage of input (no OOV!)
- Direct observation of spelling

❌ **Cons:**
- Really long sequences…
- Potentially more difficult to build higher-level representations (debatable!)

Between words and characters: Subword Tokenization

How can we combine the efficiency of word tokenization with the high coverage of character tokenization?

Solution: ***subword tokenization***, where tokens are words or parts of words

Tokenization with **ChatGPT's tokenizer**:

We'd sat back on the grass, and time flew by as we looked at the Milky Way.

Intuition behind subword tokenization:

- Use data + algorithms to define what the vocabulary should be
- Algorithms use corpus frequency statistics of adjacent segments of text to build the vocabulary
- While we sometimes get intuitive token segmentations (e.g., **We'd**), most of the time, these are not linguistically aligned (e.g., **Milky**, **looked**)

Between words and characters: Subword Tokenization

Common subword tokenization algorithms:

- **Byte Pair Encoding (BPE)** (Sennrich et al., 2016)
  - Start with a set of individual bytes, then iteratively merge neighboring tokens with the highest probability/frequency, up to the desired vocab size $|V|$
- **WordPiece** (Wu et al., 2016)
  - Similar to BPE, but iteratively merge neighboring tokens which maximize the likelihood of the data after the merge
- **Unigram Language Modeling (ULM)** (Kudo, 2018)
  - Start with a large vocabulary (all-subword strings) and discard tokens until we reach the desired vocab size $|V|$

We will be focusing on the **BPE algorithm** in this lecture

Subword tokenization: Byte Pair Encoding (BPE)

Why focus on BPE? It is the tokenization algorithm used in all popular/frontier language models

What made BPE so universal?

- It was popularized by GPT-2, and OpenAI set the standard for LM training
- BPE is simple, easy to implement, efficient, and deterministic

Model Family	Type	Vocab Size
Gemini 2.x/Gemma 3	BPE	262k
Gemini 1.x/Gemma 2	BPE	256k
Llama 4	BPE	201k
GPT-4o/GPT-5/o1/o3	BPE	200k
Kimi K2	BPE	163k
Qwen2.5/Qwen3/GLM-4	BPE	151k
Grok 2	BPE	131k
Longcat Flash Chat	BPE	131k
Llama 3/Hunyuan T1/TurboS/Large	BPE	128k
DeepSeek v3/R1	BPE	128k
DeepSeek v2.5	BPE	100k

Subword tokenization: Byte Pair Encoding (BPE)

Required:

Training corpus D

Desired vocab size N

Training Algorithm:

1. Pre-tokenize by splitting on whitespace
2. Initialize V as bytes in D
3. Convert D into sequence of tokens (i.e., bytes)
4. While $|V| < N$:
    a. Get counts of all bigrams in D
    b. Merge most frequent pair (v_i, v_j)
        into new token $v_n = v_i v_j$
        where $n = |V| + 1$
    c. Replace all instances of $v_i v_j$ in D with v_n

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

```
Peter Piper picked
a peck of pickled
peppers
```

This is our (very interesting) training corpus D

Subword tokenization: Byte Pair Encoding (BPE)

Corpus

P e t er, _ P i p
er, _pick e d, _
a, _pe ck, _ o f,
_pick l e d, _pe p
p er s

Vocab

{_, a, c, d, e, f,
i, k, l, o, p, P,
r, s, t, _p, ck,
er, _pe, _pi, _pick}

Token Pair	Frequency
_pi + ck	2
e + d	2
p + er	2
_ + a	1
_ + o	1
_ + P	1
_pe + ck	1

Merge List

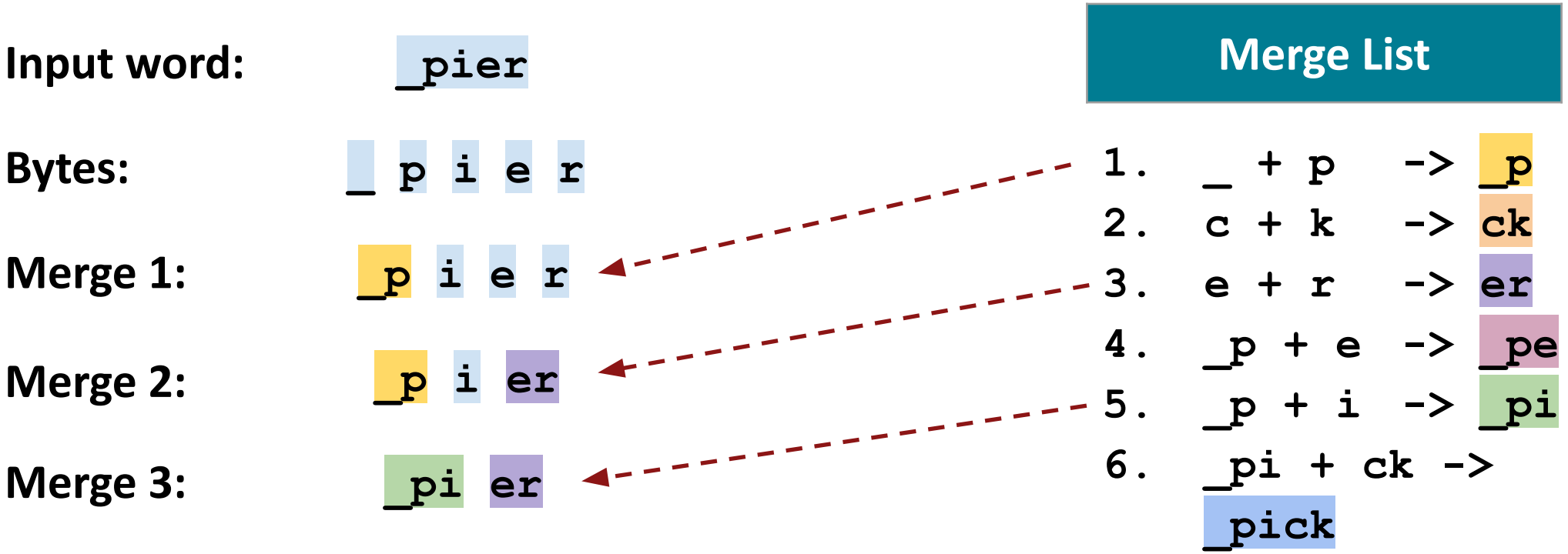
1. _ + p -> _p
2. c + k -> ck
3. e + r -> er
4. _p + e -> _pe
5. _p + i -> _pi
6. _pi + ck -> _pick

...

continue until the desired vocab size N is reached

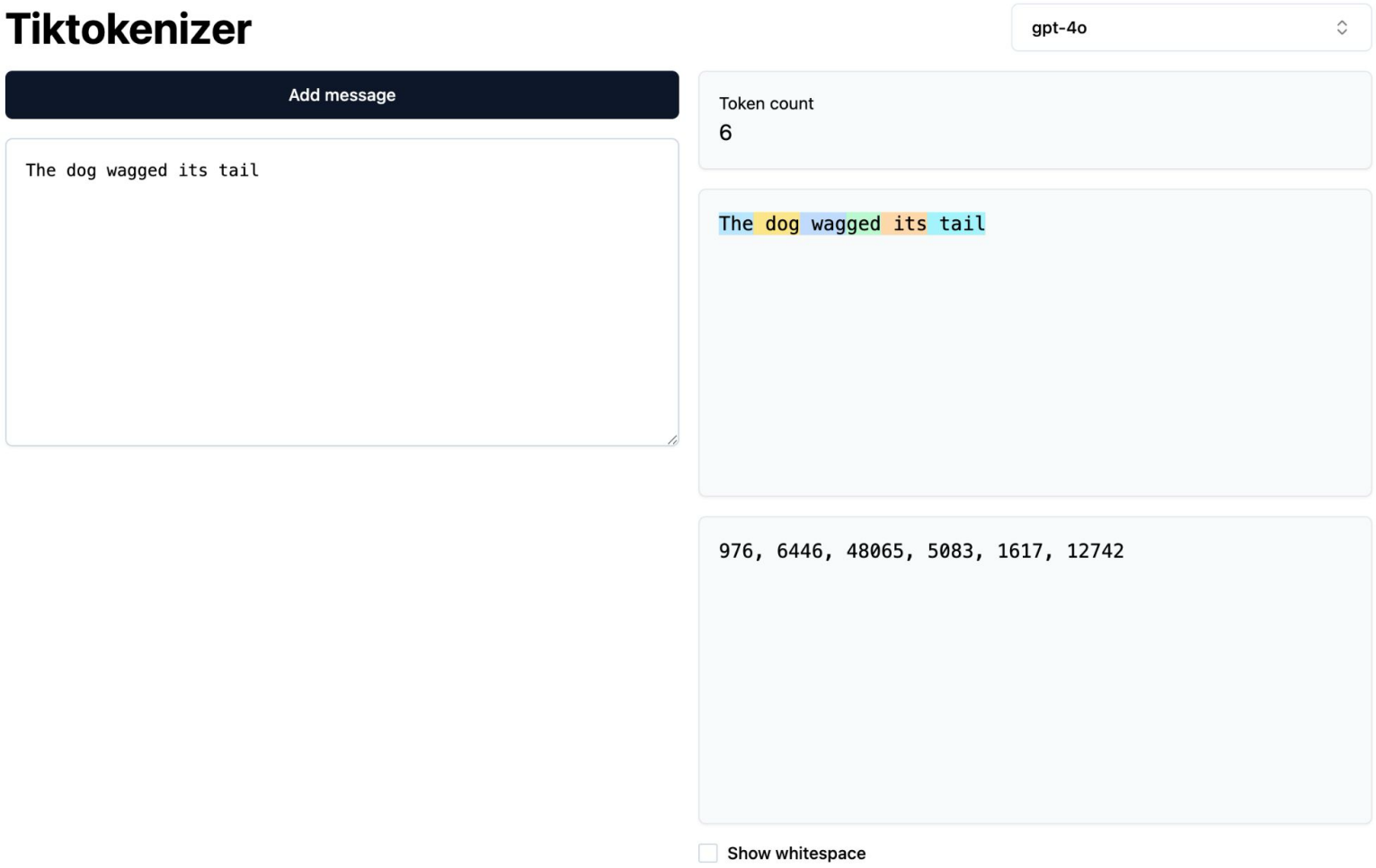
Subword tokenization: Byte Pair Encoding (BPE)

We just went over the **BPE training algorithm**. To tokenize **new text at test time**, we split it into bytes and apply merge rules in order



Tiktokenizer demo

Take a look at how some common tokenizers segment text:
https://tiktoke



Beyond Subwords: SentencePiece & Superword Tokenization

Pre-tokenizing based on whitespace: does it always make sense?

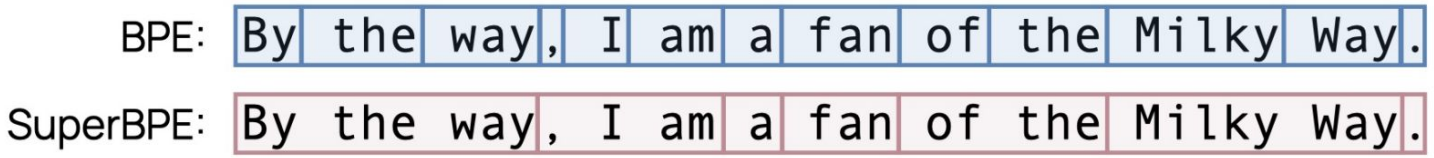
- Some languages don't use whitespace to separate words (Chinese, Japanese, Thai)
- Multi-word expressions (e.g., "on the other hand", "depend on")

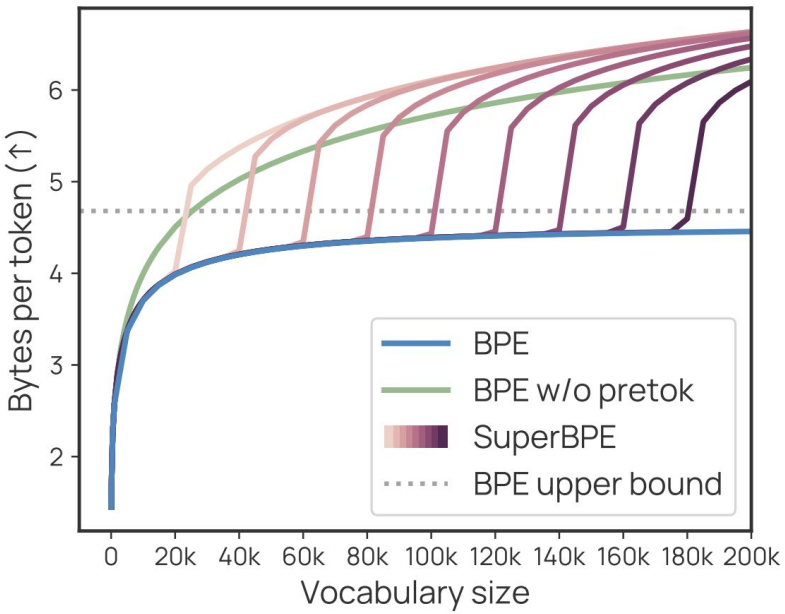
SentencePiece (Kudo & Richardson, 2018): a tokenizer **library** (not algorithm!) that implements BPE and Unigram LM

- Treats whitespace as a basic symbol in the vocabulary
- **Used by:** T5, Llama 2, XLM-R, Gemma, many multilingual models

SuperBPE (Liu et al., 2025): a pre-tokenization curriculum that first learns subwords, then removes whitespace constraints to learn superword tokens

- Smaller vocab size and less inference compute



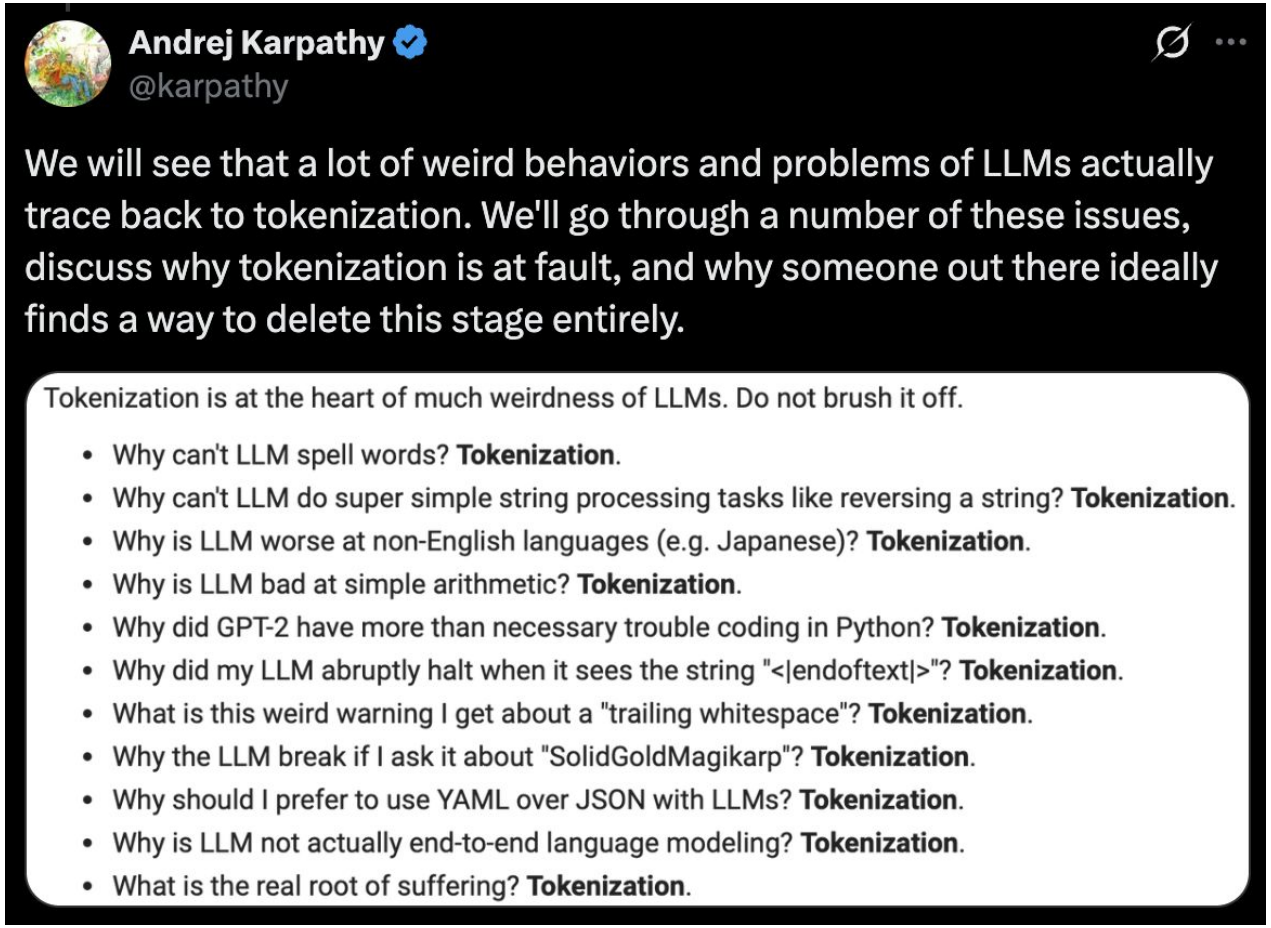


Has Subword Tokenization Solved All Our Problems?

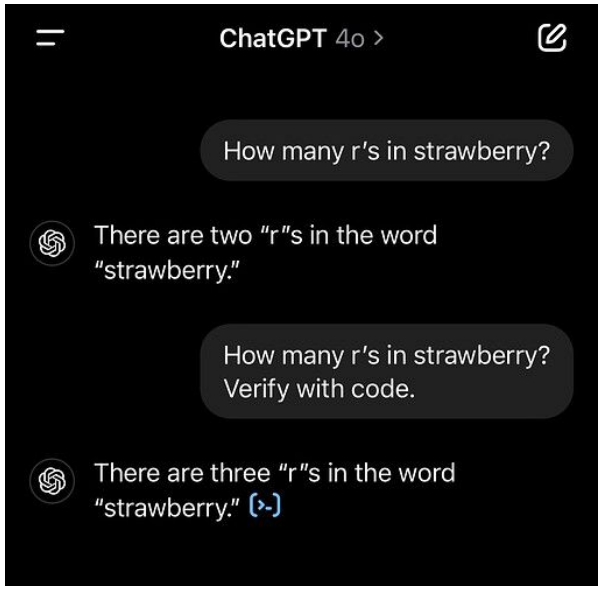
Subword tokenization with BPE is the de-facto standard today:

- BPE is a simple and effective compression algorithm
- Everything can be represented with the vocabulary (no OOV!)
- Some shared representations (e.g., picked, picking)

Not everything is solved! There's still a lot of work to be done



Case Study: Spelling 🍓



Models see ***subword chunks***, not individual characters

- GPT-2: **strawberry** → ["str", "aw", "berry"]
- GPT-4o: **strawberry** → single token (entire word is one opaque unit)
- Neither tokenization lets the model directly count the r's!

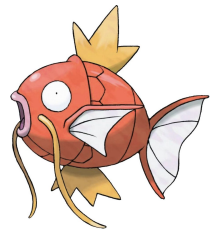
More broadly, subword models struggle with:

- **Spelling out words:** must decompose tokens into characters the model was never trained on
- **Reversing strings:** requires character-level access the tokenizer doesn't provide (Huang et al., 2023)
- **Robustness to typos:** a single character change can shift all BPE merge boundaries, producing a completely different token sequence

See **CUTE** (Edman et al., 2024) and Huang et al. 2023 for character-related LM benchmarks!

The model's atoms of language are subwords, not characters—any character-level task **forces the model to reverse-engineer information** not explicitly contained in the token

Case Study: Glitch Tokens



Tokenizer and model training data can be misaligned

- BPE tokenizers are trained on one corpus; the LM is trained on a (potentially different) corpus
- Some tokens enter the vocabulary but are rare/absent in LM training data
  - End up randomly initialized or poorly trained embeddings → undefined behavior

The famous example: SolidGoldMagikarp

- A Reddit username over-represented in the tokenizer training data
- When GPT-3 was asked to repeat/explain this token, it hallucinated about the word “distribute”, evaded the question or failed to repeat the token

🎣 Fishing for Magikarp (Land & Bartolo, 2024)

- Developed automatic detection methods; found glitch tokens prevalent across many models (GPT-2, LLaMA 2/3, OLMo, Qwen, etc.)

Not just a curiosity: wasted vocabulary slots + adversarial attack surface

More generally: don’t train your model on different data from your tokenizer (see another blog post from Arnett)

The NLP standard for tokenization

Instead of

- white-space / orthographic words
  - Lots of languages don't have them
  - The number of words grows without bound
- Unicode characters
  - Too small as tokens for many purposes
- morphemes
  - Very hard to define

We use the data to tell us how to tokenize.

Why tokenize?

Using a deterministic series of tokens means systems can be compared equally

- Systems agree on the length of a string

Algorithms like perplexity assume all texts have a fixed tokenization

Eliminates the problem of unknown words

If some word occurs in test set but not training set, we still know how to segment it into known tokens.

BPE and Unicode

We run BPE on large Unicode corpora, with vocabulary sizes of 50,000 to 200,000

On individual bytes of UTF-8-encoded text

- Not on Unicode characters
- BPE rediscovers 2-byte and common 3-byte UTF-8 sequences
- Only 256 possible values of a byte, so no unknown tokens
- (BPE might learn a few illegal UTF-8 sequences across character boundaries, but these can be filtered)

Tokenizing across languages

- Even though BPE tokenizers are multilingual
- LLM training data is still vastly dominated by English
  - Most BPE tokens used for English, leaving less for other languages
  - Words in other languages are often split up

Tokenization is better in English

Tat Dat Duong's Tiktokenizer visualizer on GPT4o

A recipe sentence in two languages

English: 18 tokens; no words are split into multiple tokens):

```
In·a·deep·bowl,·mix·the·orange·juice·with·the·sugar,·g
inger,·and·nutmeg.
```

Spanish: 33 tokens; 6/16 words are split

```
En·un·recipiente·hondo,·mezclar·el·jugo·de·naranja·con
·el·azúcar,·jengibre,·y·nuez·moscada.
```

Ninety tokens learned from a toy Thai and English corpus

```
from tokenizers import Tokenizer, models
from tokenizers import trainers, pre_tokenizers
tok = Tokenizer(models.BPE(unk_token="[UNK]"))
tok.pre_tokenizer = pre_tokenizers.Whitespace()
trainer = trainers.BpeTrainer(vocab_size=90,
                              special_tokens=["[UNK]"])
tok.train_from_iterator(corpus, trainer)   # 60 short
lines

tok.encode("lowest newer").tokens
# ['lowest', 'newer']            whole words merged
tok.encode("โรงพยาบาล ผู้ปกครอง").tokens
# ['โรงพย', 'าบาล', 'ผู้ปกค', 'รอง']   pieces cross
syllables
tok.encode("โรงรถ").tokens
# ['โรง', 'ร', '[UNK]']   ถ never seen: unknown token
```

- Merges follow **frequency**, not linguistics: โรงพย + าบาล splits the word hospital in a place no Thai speaker would.
- A character absent from training becomes **[UNK]**; that is why production tokenizers start from **bytes** (256 base symbols, nothing is ever unknown), at the cost the next section measures.
- Vocabulary size is the budget: 90 here, 100k to 260k in frontier models (CS224N table). Bigger vocabularies mean fewer tokens per sentence and a larger embedding matrix.
- The trained tokenizer **is part of the dataset**: change it and every token count, context length, and cost estimate changes.

PART D2 · 0:45–1:00

Tokenization beyond English: who pays the premium

Multilingual models transfer across languages, up to a point; tokenizers trained on English-heavy text charge other scripts more tokens for the same sentence. Thai is a textbook case, measured live on this deck.

SOURCE DECK

Lecture 14: Tokenization and Multilinguality, multilingual section (guest lecture, Julie Kallini), CS224N Winter 2026, Stanford University; followed by authored measurement slides.

https://web.stanford.edu/class/cs224n/

© the authors · shown unmodified · selected pages

Tokenization Beyond English

Why is tokenization so important for multilingual language modeling?

So far, we've only been considering English tokenization… **What happens when we apply ChatGPT's tokenizer to languages other than English?**

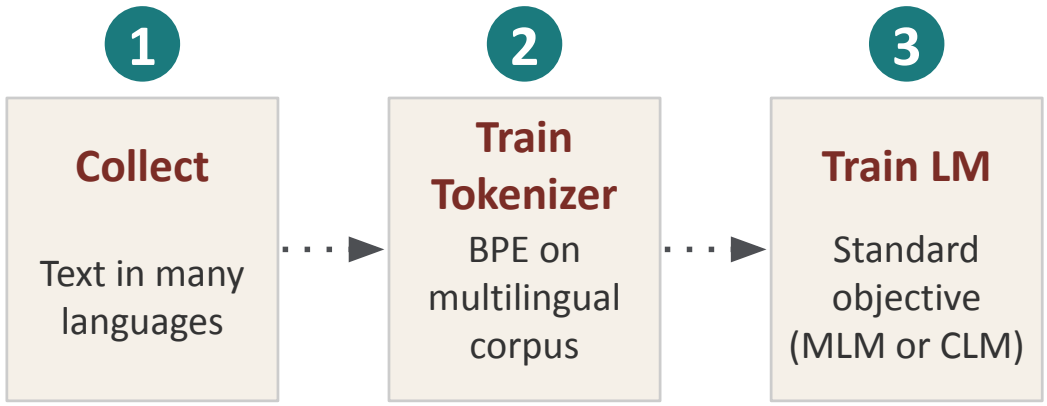
Language	Tokenization	# Tokens	# Characters
English	We'd sat back on the grass, and time flew by as we looked at the Milky Way.	21	75
French	Nous étions assis dans l'herbe, et le temps a filé tandis que nous contemplions la Voie lactée.	27	95
Somali	Waxaan dib u fadhiisanay cawskii, wakhtiguna wuu duulay markii aanu eegin Jidka caanaha.	30	88
Thai	พวกเรานั่งพักผ่อนบนพื้นหญ้า และเวลา��่านไปอย่างรวดเร็วขณะที่เรามองดูทางช้างเ��อก	36	79

Multilingual Language Modeling

What exactly is a multilingual language model?

A multilingual language model is ***architecturally identical to any LM***

- Same Transformer, same training objective
- The only differences: use training data in many languages for tokenizer and LM



Model	Year	Languages	Type
mBERT	2018	104	Encoder
XLM-R	2020	100	Encoder
mT5	2021	101	Encoder-Decoder
BLOOM	2022	46	Decoder
GPT-4, Llama, etc.	2023+	*Implicit*	Decoder

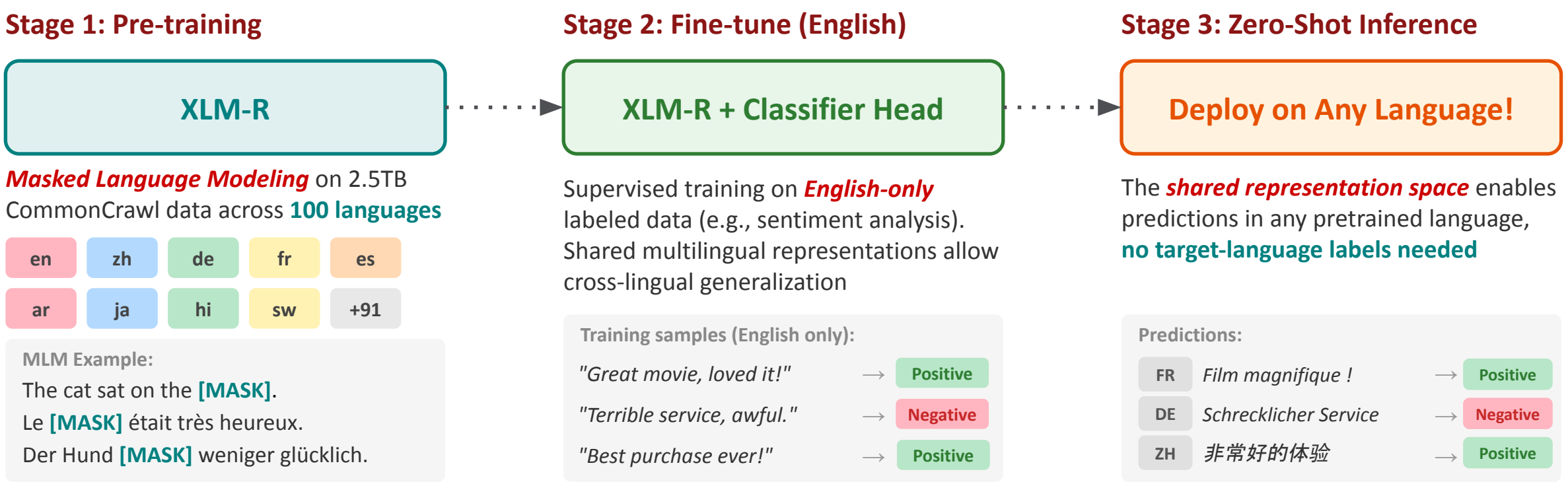
Aside: this wasn't always the status quo!

- **XLM** (2019) used ***Translation Language Modeling (TLM)***: parallel sentences from two languages are concatenated and masked language modeling is applied across both

The Power of Cross Lingual Transfer

Multilingual language models trained on mixtures of language corpora show remarkable abilities for ***cross-lingual transfer***, even without translation language modeling

- ***Cross-lingual transfer:*** a model fine-tuned on labeled data in one language (e.g., English) can perform the task in another language it was never fine-tuned on



The Curse of Multilinguality

Cross-lingual transfer seems to be a solution to high-/low-resource language disparities!

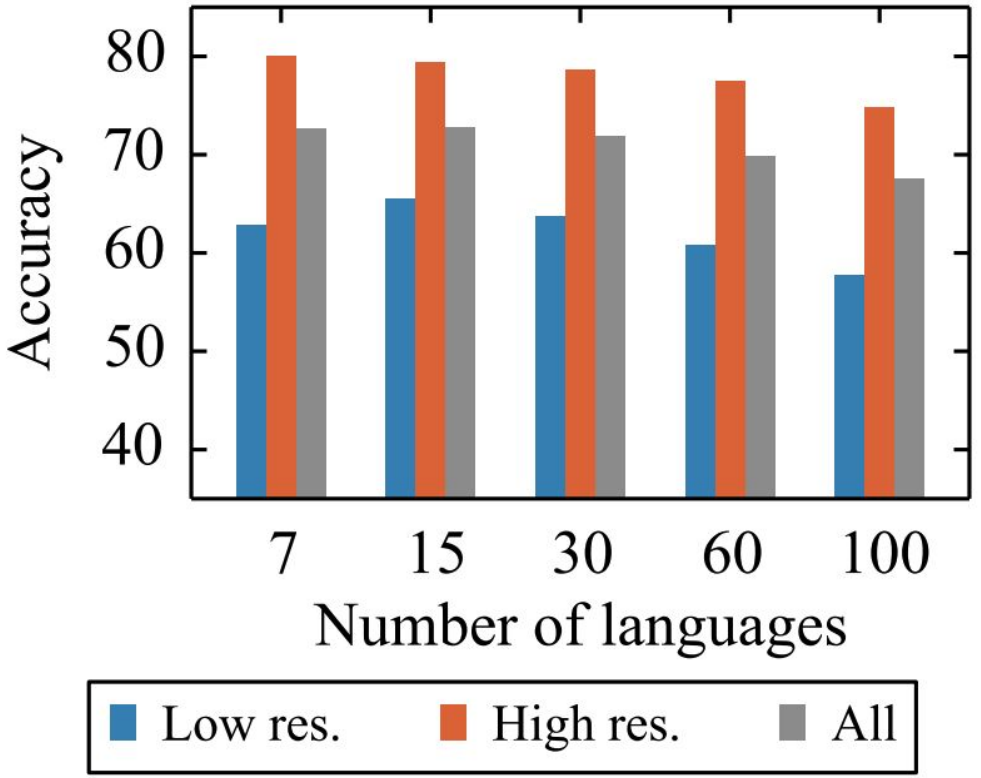
- Just fine-tune on English and apply my multilingual LM to new languages! **(not exactly…)**

The curse of multilinguality: adding more languages helps cross-lingual transfer... up to a certain point (also introduced in the XLM-R paper)

- On cross-lingual natural language inference (XNLI), XLM-R's accuracy drops from 71.8% (7 langs) to 67.7% (100 langs) with fixed model capacity

Why does this happen?

- **Fixed model capacity:** a model has finite parameters
- Adding more languages means **each language gets a smaller "share"**
- **High-resource languages** (English, French) see performance drop as languages are added
- **Low-resource languages** initially benefit from transfer, but eventually suffer too



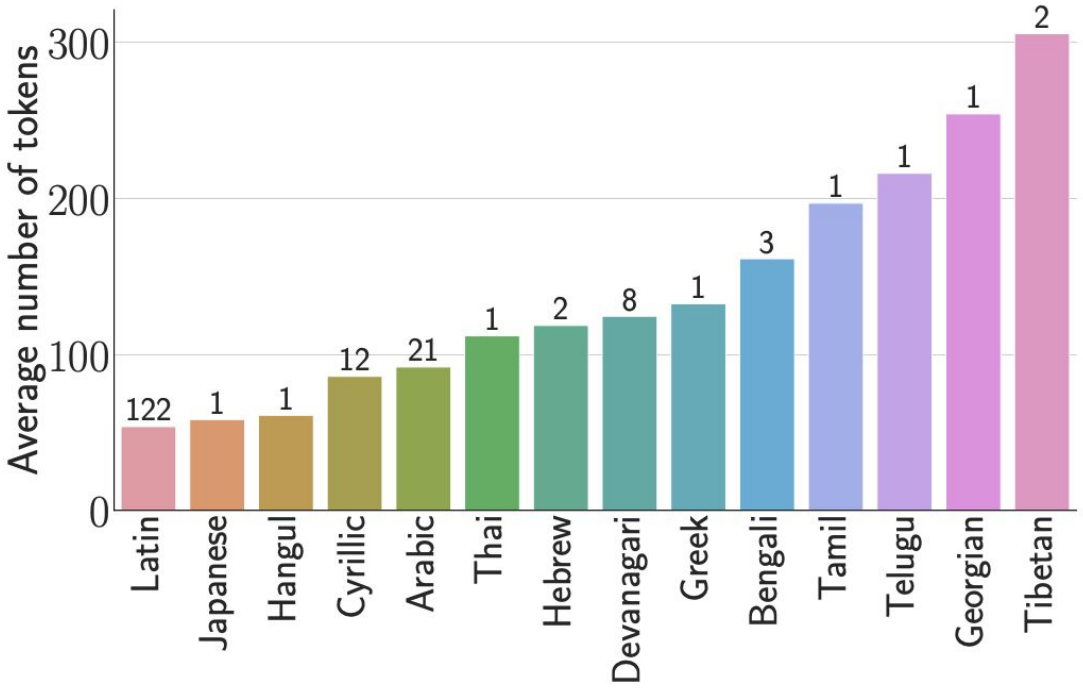
Tokenization Introduces Unfairness Between Languages

The problem starts even **before the model is trained, at the tokenizer**

- Tokenization can matter just as much as pre-training dataset size (Rust et al., 2021)
- Tokenization length differences of up to 15x between languages for equivalent text (Petrov et al., 2023)

Non-English speakers are overcharged by commercial LM APIs (Ahia et al., 2023)

- **Systematic analysis** of OpenAI API across 22 typologically diverse languages
- **Users of many languages:** pay more per unit of meaning, get poorer results, and tend to come from regions where APIs are already less affordable
- ***Subword fertility***: the average number of subword tokens produced per word
  - **Subword fertility and downstream performance are strongly negatively correlated**



Revisiting Character/Byte Tokenization

Multilinguality may be another reason to revisit character/byte tokenization! Let's first take a closer look at the multilingual results for CANINE, Google's character-level counterpart to mBERT:

Model	Input	MLM	Params	TyDiQA SelectP	TyDiQA MinSpan
mBert (public)	Subwords	Subwords	179M	63.1	50.5
mBert (ours)	Subwords	Subwords	179M	63.2	51.3
	Chars	Single Chars	127M	59.5 (–3.7)	43.7 (–7.5)
	Chars	Subwords	127M	63.8 (+0.6)	50.2 (–1.0)
Canine-S	Chars	Subwords	127M	66.0 (+2.8)	52.5 (+1.2)
Canine-C	Chars	Autoreg. Chars	127M	65.7 (+2.5)	53.0 (+1.7)
Canine-C + n-grams	Chars	Autoreg. Chars	167M	**68.1** (+4.9)	**57.0** (+5.7)

Language	mBert	Canine-C	Canine-C + *n*-grams
	MasakhaNER		
Amharic	0.0	44.6 (+44.6)	50.0 (+50.0)
Hausa	89.3	76.1 (–13.2)	88.0 (–1.3)
Igbo	84.6	75.6 (–9.0)	85.0 (+0.4)
Kinyarwanda	73.9	58.3 (–15.6)	72.8 (–1.1)
Luganda	80.2	69.4 (–10.8)	79.6 (–0.6)
Luo	75.8	63.4 (–12.4)	74.2 (–1.6)
Nigerian Pidgin	89.8	66.6 (–23.2)	88.7 (–1.1)
Swahili	87.1	72.7 (–14.4)	83.7 (–3.4)
Wolof	64.9	60.7 (–4.2)	66.5 (+1.6)
Yorùbá	78.7	67.9 (–10.8)	79.1 (+0.4)
Macro Avg	72.4	65.5 (–6.9)	76.8 (+4.3)

Since CANINE covers the full unicode alphabet, **it could process a language it had never seen!**

Revisiting Character/Byte Tokenization

Character/byte tokenization also addresses other issues of subword tokenization:

- **No glitch tokens:** every character/byte has been covered in the training data
- **Robustness** to character-level manipulations or noise
- Direct observation of **character composition of words**

Latent Tokenization: byte/character-level models are learning the relevant units of meaning as a part of the architecture

- Can they get us closer to capturing all these notions at once?

Characters:	**We looked at the Milky Way.**
Morphemes:	**We looked at the Milky Way.**
Words:	**We looked at the Milky Way.**
Phrases:	**We looked at the Milky Way.**

Acknowledgments 🤗

Thank you to the following people for sharing sample slides:

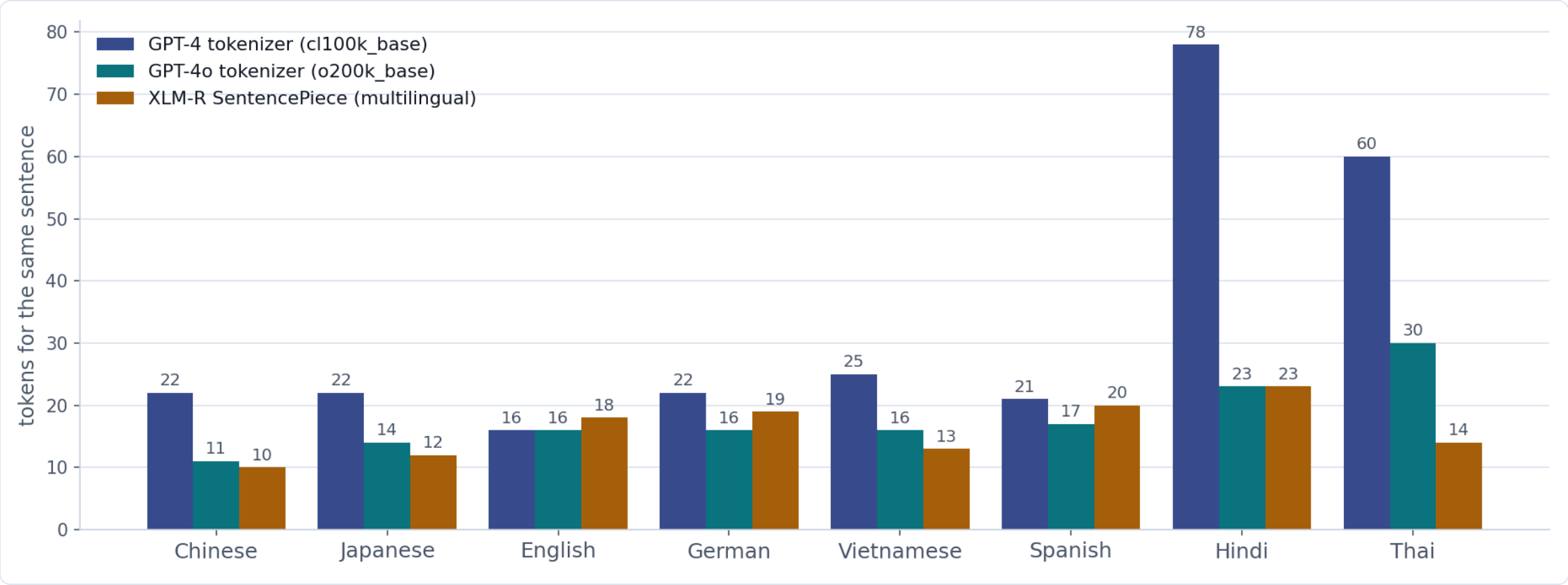
- Dan Jurafsky
- Alisa Liu
- Yejin Choi / Taylor Sorensen
- Christopher Potts

Thank you to the following people for sharing sample papers and reference pointers:

- Tolúlọpẹ́ Ògúnrẹ̀mí
- Catherine Arnett

Thank you to the Jurafsky Lab members for early feedback on this lecture

One clinical sentence, eight languages, three tokenizers



Counted with tiktoken (cl100k_base, o200k_base) and the XLM-R SentencePiece model. "The patient was admitted to the hospital with chest pain and shortness of breath": English 16 tokens; Thai 60 with GPT-4's tokenizer, 30 with GPT-4o's, 14 with a tokenizer trained on 100 languages.

"Hospital" is one token; โรงพยาบาล is five, or ten, or one

TOKENIZER	PIECES	COUNT
o200k_base (GPT-4o)	โรง | พ | ย | าบ | าล	5
cl100k_base (GPT-4)	ten byte-level fragments	10
XLM-R SentencePiece	โรงพยาบาล	1
any of the three	hospital	1

Orphaned vowel signs (าบ, าล) are not syllables; the model must learn to reassemble them.

- **Three bytes per Thai character** plus an English-heavy merge table: the premium is structural, not a bug.
- The premium costs **context window** (a Thai document fills it 2 to 4 times faster), **API money** (billed per token), and **quality** (fewer merges, weaker representations).
- A tokenizer trained on a multilingual mix (XLM-R, or a Thai-aware vocabulary such as those built for Thai LLM efforts) removes most of it.
- For your own projects: **measure fertility** (tokens per word) on a sample of your corpus before choosing a model; it is a two-line experiment.

What the literature measured, and what you can do

- **Petrov et al., NeurIPS 2023**: tokenization-length disparity of up to 15x across languages for equivalent text; longer sequences mean less context, more compute, and worse downstream fairness.
- **Ahia et al., EMNLP 2023**: "Do all languages cost the same?" API cost for the same content varies by an order of magnitude; users in poorer regions pay more per unit of meaning.
- **Tokenizer-free models**: ByT5 (TACL 2022), CANINE (TACL 2022) work on bytes or characters; newer proposals (MYTE, MAGNET, 2024) reduce the premium with morphology-aware byte encodings.
- **Thesis-sized question** (course section 2.2): multilingually fair tokenizers for Thai and Southeast Asian scripts.

```
# fertility check on your own corpus, two lines
import tiktoken
enc = tiktoken.get_encoding("o200k_base")
fert = [len(enc.encode(s)) / len(word_tokenize(s))
        for s in sample_sentences]      # tokens per word

# context budget: tokens, not characters
n_tokens = len(enc.encode(document))
assert n_tokens < MAX_CONTEXT - RESERVED_FOR_ANSWER
```

Budget your prompts in tokens per language; a 4k-character Thai note is not a 4k-character English note.

PART E1 · 1:10–1:25

Where pretraining data comes from, and why that is a preprocessing problem

Common Crawl to C4 to The Pile: what is actually in the corpus, which books were scraped, what the model memorizes, and the three families of filters everyone uses.

SOURCE DECKS

Large Language Models (SLP3 slides, Jurafsky and Martin, Aug 2025) · **Lecture 7: Pretraining** (CS224N Winter 2026, Stanford) · **Lecture 5: Data, Data, and Data** (CS 329X Human-Centered LLMs, William Held, Stanford).
web.stanford.edu/~jurafsky/slp3/slides · web.stanford.edu/class/cs224n · web.stanford.edu/class/cs329x © the respective authors · shown unmodified · selected pages

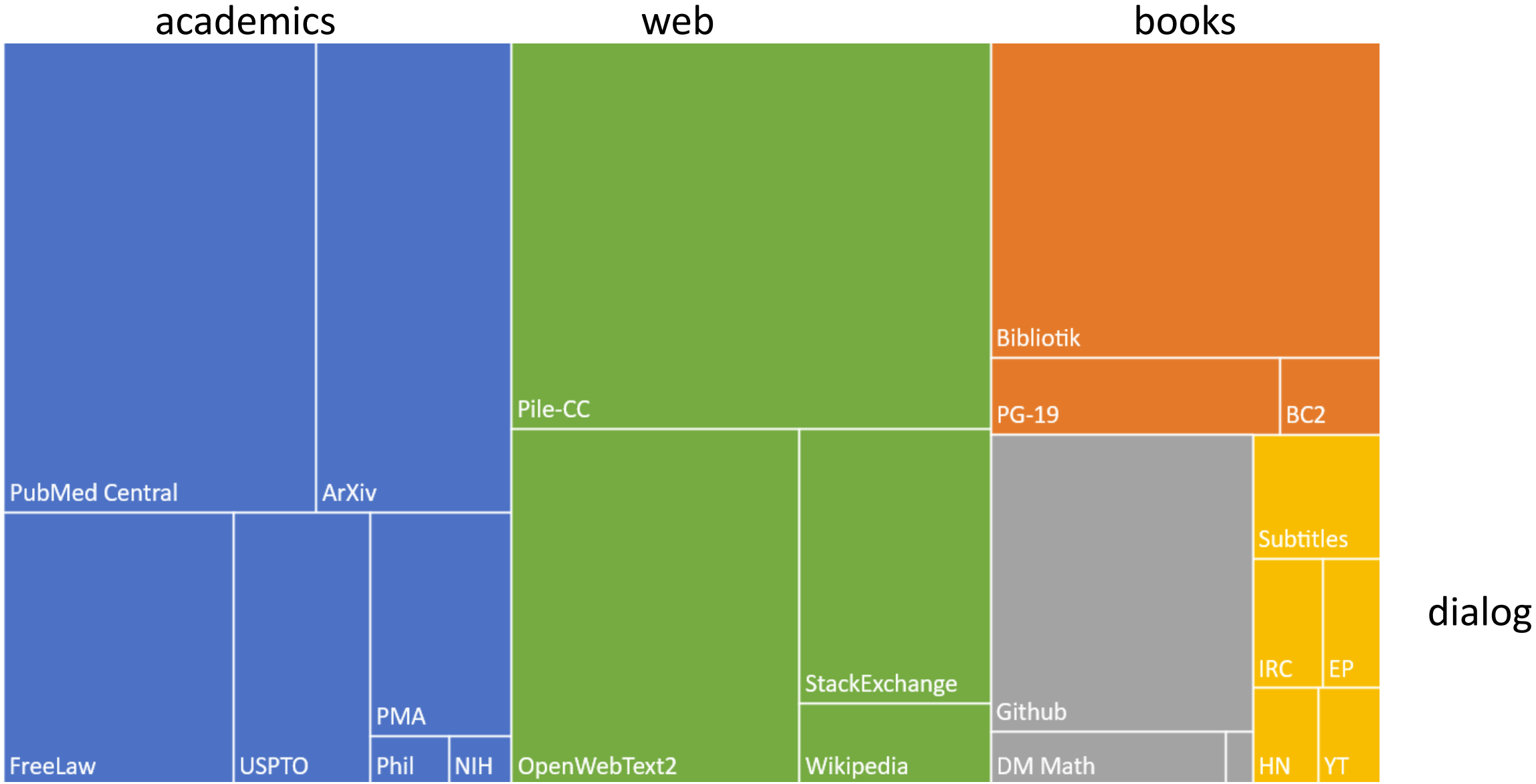
LLMs are mainly trained on the web

Common crawl, snapshots of the entire web produced by the non- profit Common Crawl with billions of pages

Colossal Clean Crawled Corpus (C4; Raffel et al. 2020), 156 billion tokens of English, filtered

What's in it? Mostly patent text documents, Wikipedia, and news sites

The Pile: a pretraining corpus



Filtering for quality and safety

Quality is subjective

- Many LLMs attempt to match Wikipedia, books, particular websites
- Need to remove boilerplate, adult content
- Deduplication at many levels (URLs, documents, even lines)

Safety also subjective

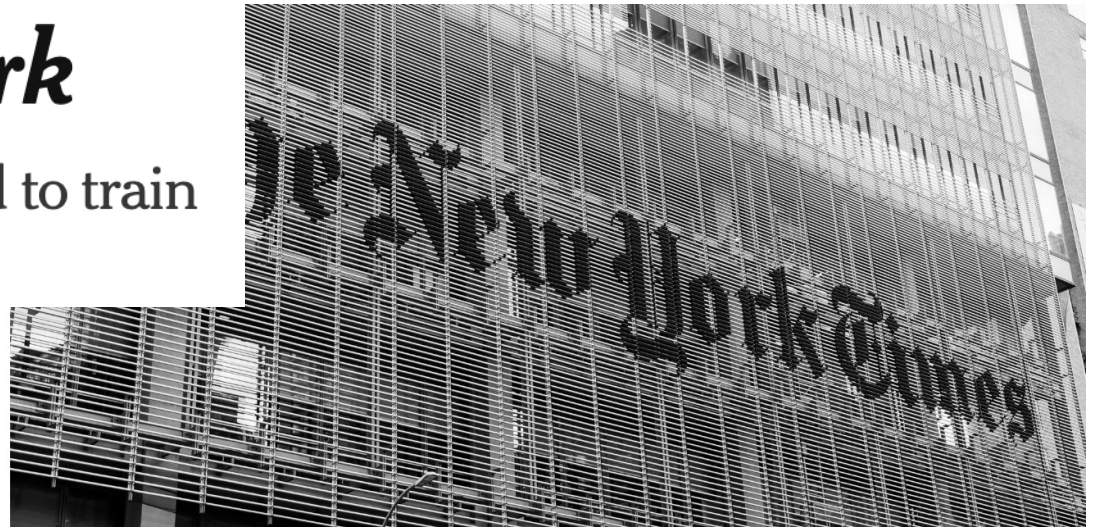
- Toxicity detection is important, although that has mixed results
- Can mistakenly flag data written in dialects like African American English

There are problems with scraping from the web

Authors Sue OpenAI Claiming Mass Copyright Infringement of Hundreds of Thousands of Novels

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.



There are problems with scraping from the web

Copyright: much of the text in these datasets is copyrighted

- Not clear if fair use doctrine in US allows for this use
- This remains an open legal question across the world

Data consent

- Website owners can indicate they don't want their site crawled

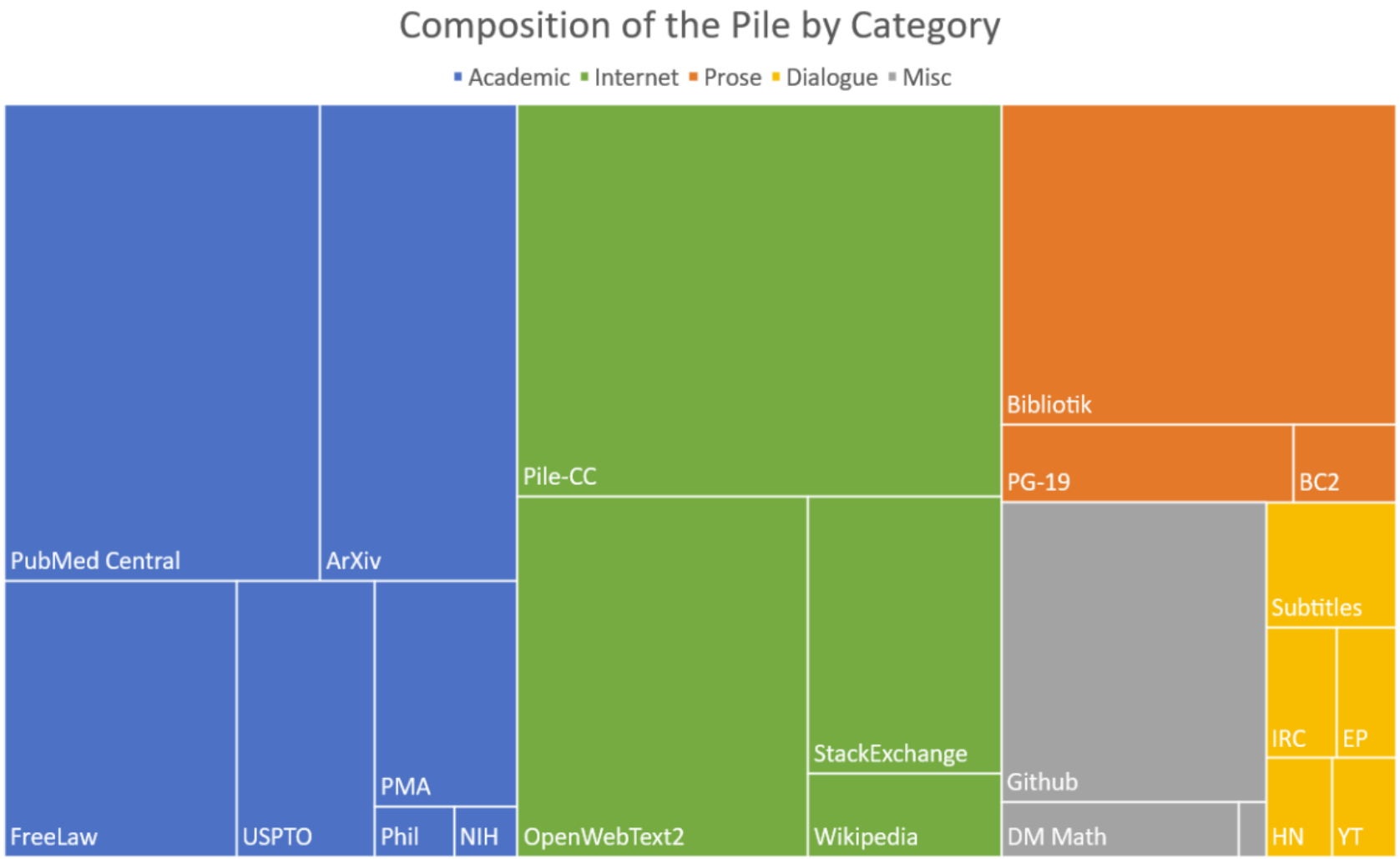
Privacy:

- Websites can contain private IP addresses and phone numbers

Skew:

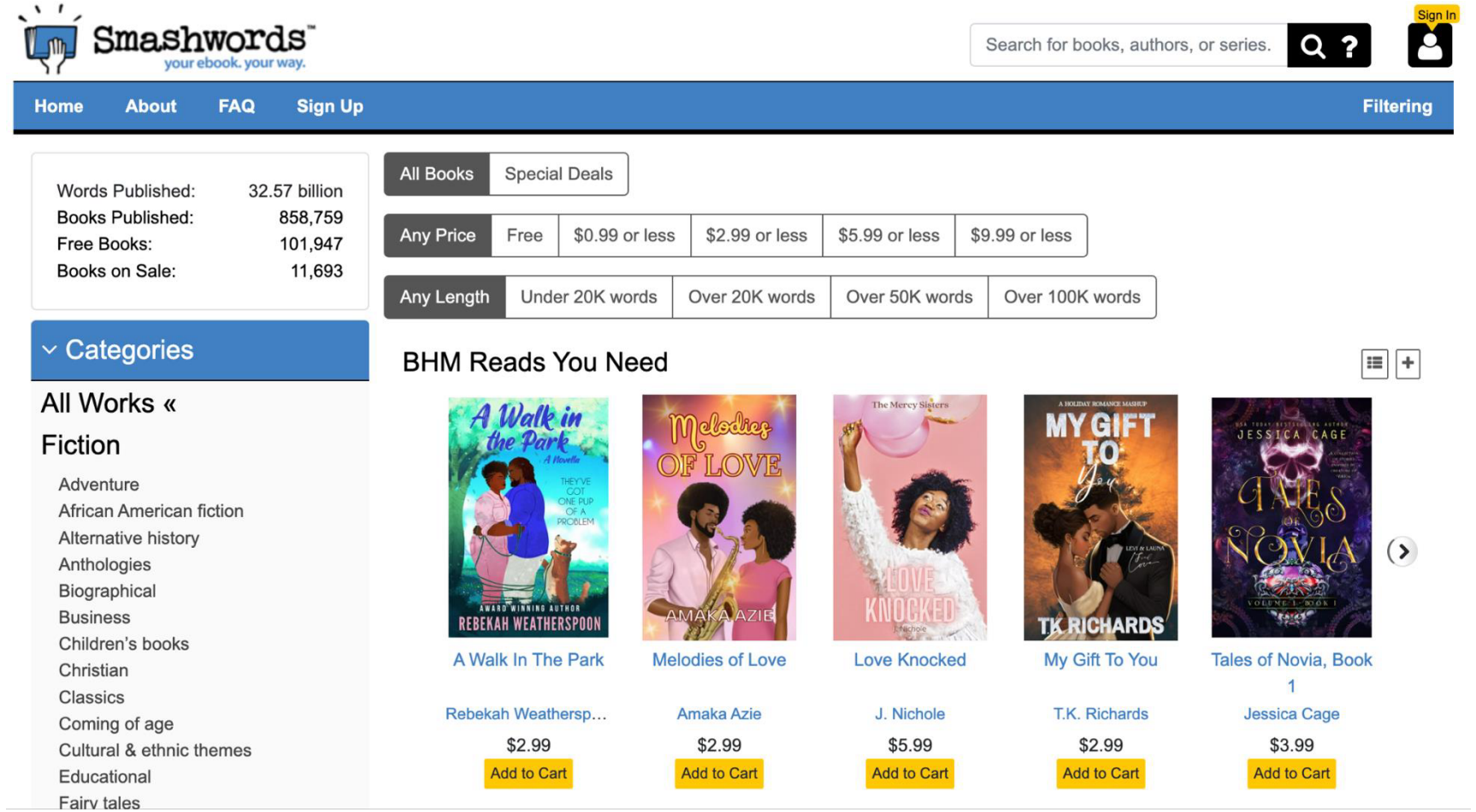
- Training data is disproportionately generated by authors from the US which probably skews resulting topics and opinions

Where does this data come from?



Model	Training Data
BERT	BookCorpus, English Wikipedia
GPT-1	BookCorpus
GPT-3	CommonCrawl, WebText, English Wikipedia, and 2 book databases (“Books 1” and “Books 2”)
GPT-3.5+	Undisclosed

Bookcorpus.. what's that?



- Scraped ebooks from the internet – highly controversial

Fair use and other concerns

Google swallows 11,000 novels to improve AI's conversation

As writers learn that tech giant has processed their work without permission, the Authors Guild condemns 'blatantly commercial use of expressive authorship'

'It doesn't harm the authors' ... Google's headquarters in Mountain View, California. Photograph: Marcio Jose Sanchez/AP

Arts and Humanities, Law, Regulation, and Policy, Machine Learning

Reexamining "Fair Use" in the Age of AI

Generative AI claims to produce new language and images, but when those ideas are based on copyrighted material, who gets the credit? A new paper from Stanford University looks for answers.

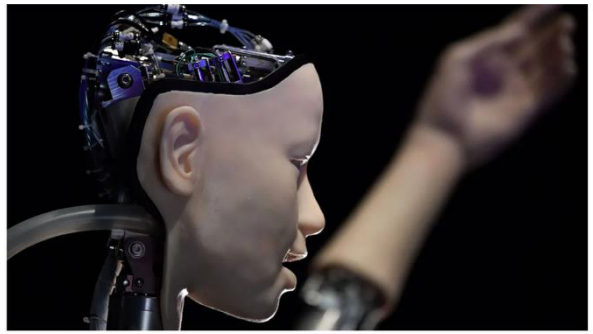
Jun 5, 2023 | Andrew Myers

Sometimes it also memorizes copyrighted material

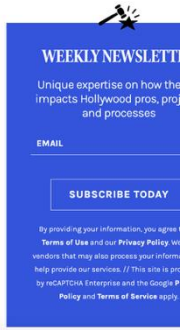
AI Art Generators Spark Multiple Copyright Lawsuits

Getty and a trio of artists sued AI art generators in separate suits accusing the companies of copyright infringement for pilfering their works.

BY WINSTON CHO JANUARY 17, 2023 4:10PM



BEN STANSALL/AFP VIA GETTY IMAGES



Anthropic fires back at music publishers' AI copyright lawsuit

By Blake Brittain

January 17, 2024 3:30 PM PST · Updated 19 days ago

ARTICLE

Insights from the Pending Copilot Class Action Lawsuit

October 4, 2023

Bloomberg Law

By Daniel R. Mello, Jr.; Jenevieve J. Maerker; Matthew C. Berntsen; Ming-Tao Yang

GitHub Inc. offers a cloud-based platform that is popular among many software programmers for hosting and sharing source code, and collaborating on source code drafting. GitHub's artificial

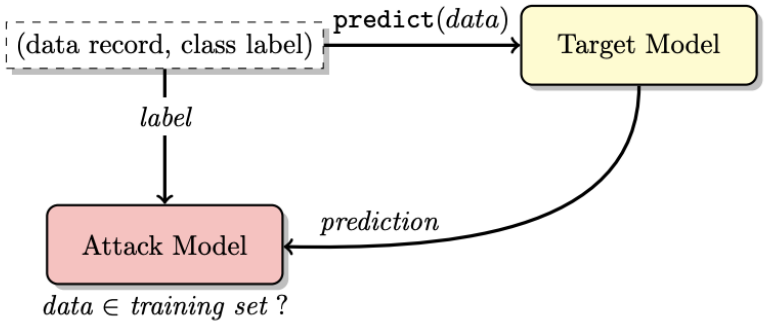
The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.

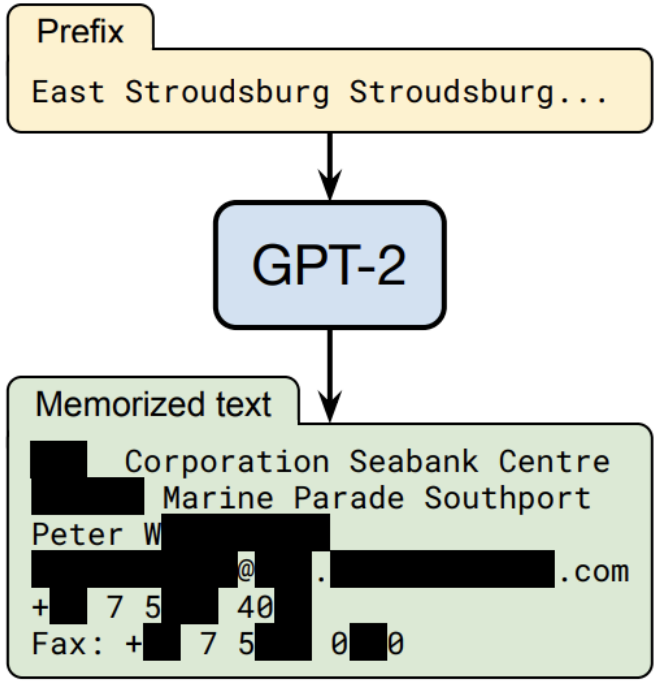
Share free access 1.3K

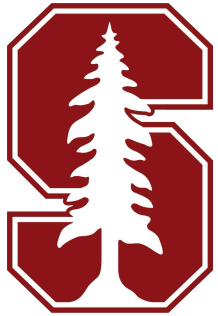
Sometimes it learns some things we don't want..

- *Membership inference* lets you recover parts of the training data



- Sometimes this training data is semi-private material from the web (addresses, emails)



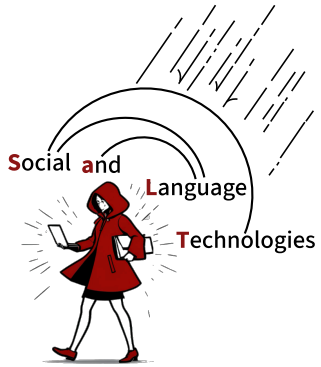


CS 329X: Human-Centered LLMs

Data, Data, and Data

Collection Filtering Curation

William Held



CommonCrawl

An Open Non-Profit Crawl of the Internet

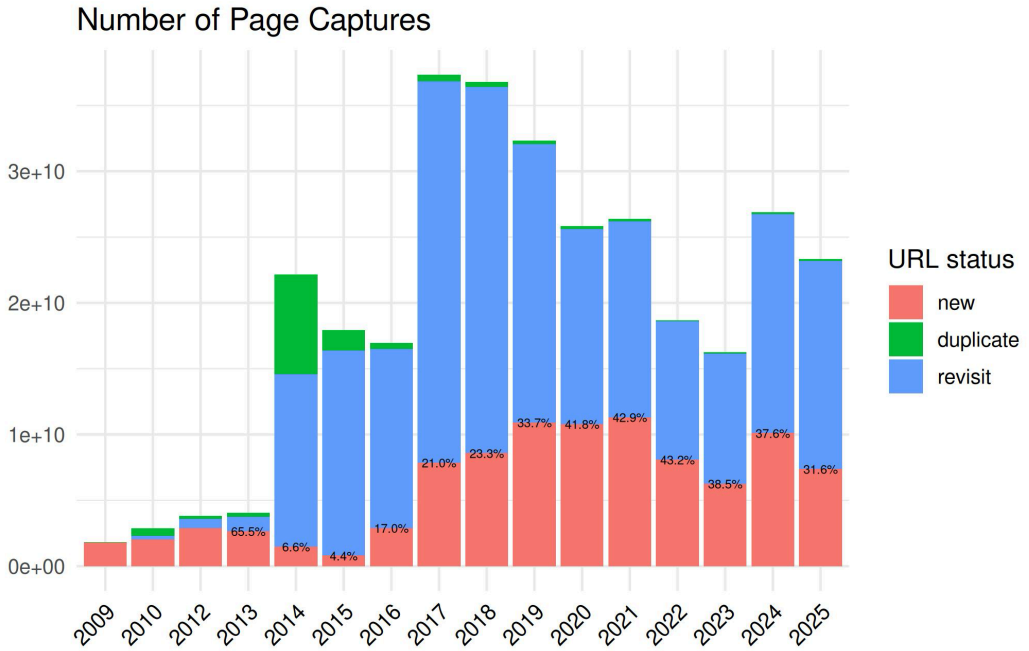
Size

$1x10^{14}$ Tokens

Collection

Indiscriminate web crawling from a seed set of websites to try and cover as much of the web as possible!

CommonCrawl.org



Common Crawl as the “raw” foundation for LLM data

There's clearly a lot more data in CC!
Most of it is garbage (or worse) though...

Random Sample of Common Crawl Data

How do we get usable training data at scale from the web?

Three Common Types of LLM Data Filtering

Perplexity Based Filtering

Given curated “good” data, use a lightweight language model to remove OOD data.

Rule Based Filtering

Researchers define programmatic rules of what “good” data looks like!

Classifier Based Filtering

Given an expensive heuristic for “good” data, use a lightweight classifier as a proxy to remove!

Open-Data Era: Perplexity Filtering

CCNet

CommonCrawl Filtered by
Language + PPL Filter

Size

$5x10^{11}$ Tokens

Filtering

Given a CommonCrawl snapshot, removes documents that are overly “surprising” to a KenLM model trained on Wikipedia

Wenzek et al. 2019

Then, remove documents that have “un-natural” language as measured by a Wikipedia N-Gram LM

Open-Data Era: Rule Based Filtering

C4

CommonCrawl Filtered by Human-Written Heuristics

Size

$3x10^{11}$ Tokens

Filtering

Given a CommonCrawl snapshot, trains removes documents & passages that fail any of the following rules.

Raffel et al. 2019

- We only retained lines that ended in a terminal punctuation mark (i.e. a period, exclamation mark, question mark, or end quotation mark).
- We discarded any page with fewer than 3 sentences and only retained lines that contained at least 5 words.
- We removed any page that contained any word on the "List of Dirty, Naughty, Obscene or Otherwise Bad Words".[6]
- Many of the scraped pages contained warnings stating that Javascript should be enabled so we removed any line with the word Javascript.
- Some pages had placeholder "lorem ipsum" text; we removed any page where the phrase "lorem ipsum" appeared.
- Some pages inadvertently contained code. Since the curly bracket "{" appears in many programming languages (such as Javascript, widely used on the web) but not in natural text, we removed any pages that contained a curly bracket.
- Since some of the scraped pages were sourced from Wikipedia and had citation markers (e.g. [1], [citation needed], etc.), we removed any such markers.
- Many pages had boilerplate policy notices, so we removed any lines containing the strings "terms of use", "privacy policy", "cookie policy", "uses cookies", "use of cookies", or "use cookies".
- To deduplicate the data set, we discarded all but one of any three-sentence span occurring more than once in the data set.

Open(ish)-Data Era: Classifier Based Filtering

Mostly Low Quality

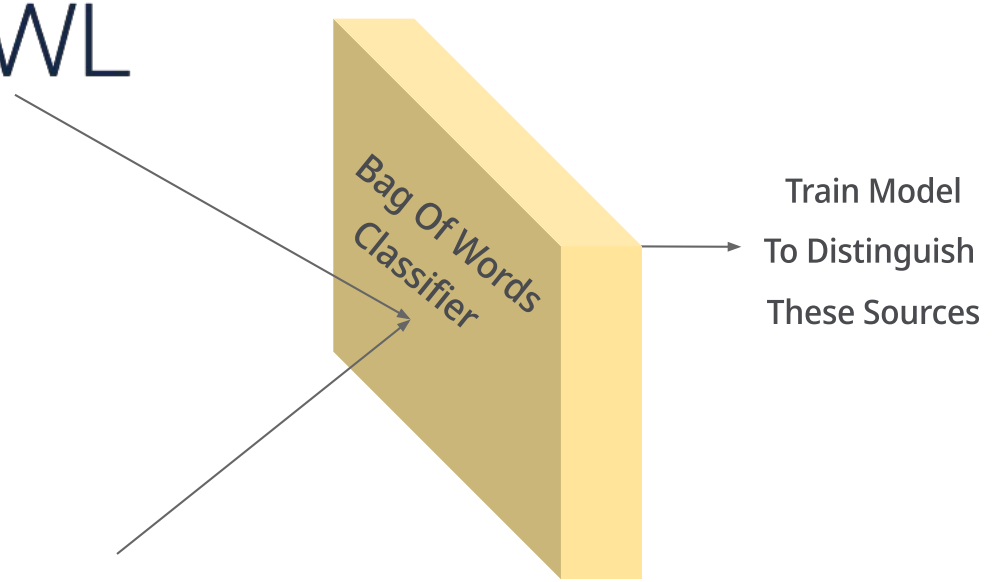
COMMON CRAWL

GPT-3 Corpus

CommonCrawl Filtered by a Classifier trained on the GPT-2 Corpus

Size

$4x10^{11}$ Tokens



Filtering

Given a CommonCrawl snapshot, keep only the "false positives" that are classified as similar to GPT-2 training data.

Mostly High Quality

Radford et al. 2019

Dataset	Percentage
Dolmino DCLM HQ	67.8%
Dolma peS2o	10.8%
FineMath 3+	6.3%
Dolma Arxiv	5.2%
Dolma StackExchange	3.2%
StarCoder	2.2%
Dolma Algebraic Stack	2.1%
Dolma Open Web Math	0.9%
Dolma Megawika	0.8%
Dolma Wikipedia	0.7%

Marin 2025

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

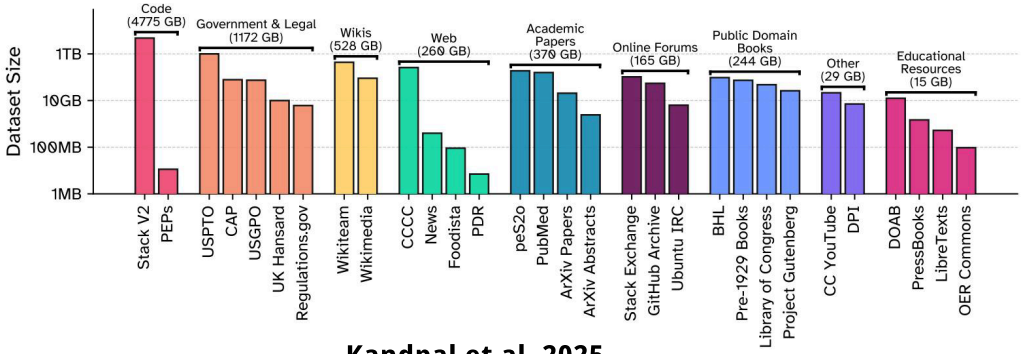
Touvron et al. 2023

Source	Type	Tokens	Words	Bytes	Docs
Pretraining ✦ OLMo 2 1124 Mix					
DCLM-Baseline	Web pages	3.71T	3.32T	21.32T	2.95B
StarCoder filtered version from OLMoE Mix	Code	83.0B	70.0B	459B	78.7M
peS2o from Dolma 1.7	Academic papers	58.6B	51.1B	413B	38.8M
arXiv	STEM papers	20.8B	19.3B	77.2B	3.95M
OpenWebMath	Math web pages	12.2B	11.1B	47.2B	2.89M
Algebraic Stack	Math proofs code	11.8B	10.8B	44.0B	2.83M
Wikipedia & Wikibooks from Dolma 1.7	Encyclopedic	3.7B	3.16B	16.2B	6.17M
Total		**3.90T**	**3.48T**	**22.38T**	**3.08B**

Soldaini et al. 2024

At a big company, many data engineers are working on collecting & filtering new datasets for particular use-cases!

In the open, the same is true for different labs and research groups!



Kandpal et al. 2025

Component	Raw Size	Weight	Epochs	Effective Size	Mean Document Size
Pile-CC	227.12 GiB	18.11%	1.0	227.12 GiB	4.33 KiB
PubMed Central	90.27 GiB	14.40%	2.0	180.55 GiB	30.55 KiB
Books3†	100.96 GiB	12.07%	1.5	151.44 GiB	538.36 KiB
OpenWebText2	62.77 GiB	10.01%	2.0	125.54 GiB	3.85 KiB
ArXiv	56.21 GiB	8.96%	2.0	112.42 GiB	46.61 KiB
Github	95.16 GiB	7.59%	1.0	95.16 GiB	5.25 KiB
FreeLaw	51.15 GiB	6.12%	1.5	76.73 GiB	15.06 KiB
Stack Exchange	32.20 GiB	5.13%	2.0	64.39 GiB	2.16 KiB
USPTO Backgrounds	22.90 GiB	3.65%	2.0	45.81 GiB	4.08 KiB
PubMed Abstracts	19.26 GiB	3.07%	2.0	38.53 GiB	1.30 KiB
Gutenberg (PG-19)†	10.88 GiB	2.17%	2.5	27.19 GiB	398.73 KiB
OpenSubtitles†	12.98 GiB	1.55%	1.5	19.47 GiB	30.48 KiB
Wikipedia (en)†	6.38 GiB	1.53%	3.0	19.13 GiB	1.11 KiB
DM Mathematics†	7.75 GiB	1.24%	2.0	15.49 GiB	8.00 KiB
Ubuntu IRC	5.52 GiB	0.88%	2.0	11.03 GiB	545.48 KiB
BookCorpus2	6.30 GiB	0.75%	1.5	9.45 GiB	369.87 KiB
EuroParl†	4.59 GiB	0.73%	2.0	9.17 GiB	68.87 KiB
HackerNews	3.90 GiB	0.62%	2.0	7.80 GiB	4.92 KiB
YoutubeSubtitles	3.73 GiB	0.60%	2.0	7.47 GiB	22.55 KiB
PhilPapers	2.38 GiB	0.38%	2.0	4.76 GiB	73.37 KiB
NIH ExPorter	1.89 GiB	0.30%	2.0	3.79 GiB	2.11 KiB
Enron Emails†	0.88 GiB	0.14%	2.0	1.76 GiB	1.78 KiB
The Pile	**825.18 GiB**			**1254.20 GiB**	**5.91 KiB**

Gao et al. 2020

Types of LLM Data Curation for Pretraining

Heuristic Curation

We can sample data according to our beliefs about it's relative quality or based purely on high-level metrics such as size.

- Proportional Sampling
- Uniform Sampling
- UniMax (2023)

Most open models are here!

Training Dynamics

We can use the degree to which our model is able to learn different training domains to inform curation!

Resource Optimization

Given some definition of our downstream goals, we can curate our data to optimize this definition of final model quality.

PART E2 · 1:25–1:50

Cleaning at web scale: deduplicate, filter, mix

Exact, approximate, and model-based deduplication; heuristic and classifier quality filters; privacy, bias, data age; packing and domain mixing. The pipeline behind every open model, in one lecture.

SOURCE DECK

Lecture 3: Data, EE-628 Training Large Language Models, Spring 2025 · Volkan Cevher and Caglar Gulcehre, LIONS, EPFL.
https://www.epfl.ch/labs/lions/teaching/ee-628-training-large-language-models/ License: Creative Commons BY-NC-SA (stated on the deck's slide 2, retained here)

Training Large Language Models (LLMs)

Prof. Volkan Cevher
volkan.cevher@epfl.ch

Lecture 3: Data

Laboratory for Information and Inference Systems (LIONS)
École Polytechnique Fédérale de Lausanne (EPFL)

EE-628 (Spring 2025)

License Information for Training LLMs Slides

- This work is released under a Creative Commons License with the following terms:
- **Attribution**
  - The licensor permits others to copy, distribute, display, and perform the work. In return, licensees must give the original authors credit.
- **Non-Commercial**
  - The licensor permits others to copy, distribute, display, and perform the work. In return, licensees may not use the work for commercial purposes – unless they get the licensor's permission.
- **Share Alike**
  - The licensor permits others to distribute derivative works only under a license identical to the one that governs the licensor's work.
- Full Text of the License

Data processing pipeline for LLM pretraining

- Modern LLMs (e.g., ChatGPT, Gemini, Claude, DeepSeek,...) employ the following general data pipeline:

1. Data cleaning
   - Deduplication
   - Privacy and copyright
   - Quality filtering
   - Data age
2. Data selection
   - Data masking
   - Document packing
   - Data ordering
   - Synthetic data
3. Data mixing
   - Find optimal domain mixtures
   - Target evaluations
   - Online mixing approaches
   - Offline mixing approaches

Data cleaning: Deduplication

- Datasets curated from internet contain substantial amounts of duplicate and near-duplicate content.
- This redundancy can negatively impact the performance of language models [51].
  - Redundancy skews the model's learning, resulting in overemphasis on repeated patterns.
- Effective deduplication helps to create a more balanced and varied training set.
  - It helps with model's generalization and alignment.
  - It reduces storage, which in turn reduces costs associated with infrastructure.
- We discuss three deduplication techniques in the sequel:
  1. Exact matching
  2. Approximate matching
  3. Model-based matching

Deduplication techniques: Exact matching

- We can use hashing for each document.
  - Two documents are duplicates, if their hashes coincide.
  - Secure hash algorithm (SHA)-1, or its first 64 bits, used in CCNet [89].
  - A simple hash function on large scale datasets has extremely low collision probability [24].
- We can use exact substring matching to identify duplicate substring.
  - [51] uses suffix arrays for efficient computation.

Deduplication techniques: Approximate matching

Dataset	Example	Near-Duplicate Example
Wiki-40B	\n_START_ARTICLE_\nHum Award for Most Impactful Character \n_START_SECTION_\nWinners and nominees\n_START_PARAGRAPH_\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]	\n_START_ARTICLE_\nHum Award for Best Actor in a Negative Role \n_START_SECTION_\nWinners and nominees\n_START_PARAGRAPH_\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]
LM1B	I left for California in 1979 and tracked Cleveland 's changes on trips back to visit my sisters .	I left for California in 1979 , and tracked Cleveland 's changes on trips back to visit my sisters .
C4	Affordable and convenient holiday flights take off from your departure country, "Canada". From May 2019 to October 2019, Condor flights to your dream destination will be roughly 6 a week! Book your Halifax (YHZ) - Basel (BSL) flight now, and look forward to your "Switzerland" destination!	Affordable and convenient holiday flights take off from your departure country, "USA". From April 2019 to October 2019, Condor flights to your dream destination will be roughly 7 a week! Book your Maui Kahului (OGG) - Dubrovnik (DBV) flight now, and look forward to your "Croatia" destination!

Figure: Examples of near-duplicates identified by NEARDUP from multiple datasets (from [51]).

- NEARDUP: Efficiently identifies near-duplicates by estimating Jaccard similarity between documents [51].
- Edit Similarity: Similarity based on number of edits (insertions, deletions, substitutions) [51].
- Other similarity metrics used in practice include Levenshtein distance in PaLM [14].

Deduplication techniques: Model-based

- Use learned representations from pretrained models to identify semantically similar documents.
- SEMDEDUP [1]:
  - Uses a pretrained language model to generate embeddings.
  - Calculates a representation for each data point (e.g., last layer embedding of final token).
  - Clusters data points to find groups of similar documents.
  - Computes pairwise cosine similarity within each cluster.
- Document De-Duplication and Diversification (D4) [83]:
  - Combines hashing and clustering.
  - Removes "most prototypical" examples within each cluster, enhancing diversity [76].

Data cleaning: Privacy and copyright

- Privacy (anonymization)
  - Challenge: Web-scraped data often contains Personally Identifiable Information (PII), such as names, phone numbers, addresses, and emails.
  - Risk: An adversary can attack the LLM and extract training-data PPI [12, 53].
  - Mitigation:
    - Rule-based: Identify and replace PII with placeholders (e.g., using regular expressions).
    - Differential Privacy: Add noise to the data to protect individual privacy while preserving utility [22, 23].
- Copyright
  - Challenge: Much of the web content, e.g., books and articles, is copyrighted material.
  - Risk: Training LLMs on copyrighted data without permission or license can lead to legal issues.
  - Mitigation:
    - Obtain explicit permission from copyright holders.
    - Utilize publicly available datasets.

Example (Rule-based anonymization)

Original Text: "John Doe's phone number is 555-123-4567 and his email is john.doe@example.com."

Anonymized Text: "[NAME]'s phone number is [PHONE] and his email is [EMAIL]."

Data cleaning: Bias

- Real-world datasets are biased.
- Data imbalances hurt out-of-distribution performance [28].
- Bias is also within the same research vein as fairness and AI safety [15].

Example (Biased data)

A language model trained predominantly on news articles may exhibit bias towards formal language and struggle with informal slang or dialectical variations.

Remark:

- Mitigation techniques include
  - data augmentation with examples representing underrepresented groups [102]
  - contrastive objectives [34]
  - dropout regularization [87]
  - counterfactual data augmentation [87, 49, 61]

Data cleaning: Filtering low-quality data

- Pretraining datasets for LLMs often contain substantial amounts of low-quality, noisy, or undesirable content.
- Modern LLMs use quality and/or toxicity filtering to achieve optimal model performance and desired behavior.

Example

Consider the Common Crawl dataset, a massive web scrape. It includes

- advertisements and promotional material;
- repetitive, non-human-readable text;
- toxic content.

Filtering approaches

Two primary approaches are used, often in combination:

1. Heuristic-based filtering: Rule-based methods that remove documents based on easily computable features.
2. Classifier-based filtering: Models trained to identify and remove specific types of undesirable content.

Quality filtering: Heuristic-based

- Heuristic methods deploy rules based on simple text features [67, 95, 48, 100].
  - Sentence length: Remove documents with sentences that are too short or too long.
  - Specific-word presence: Filter documents based on the presence of specific words with low-quality content.
  - Repetitiveness: Eliminate documents with excessive repetition of characters or phrases.
  - Language: Ensure the document is primarily in the desired target language (e.g., English-only).

Example

Colossal, cleaned version of Common Crawl's web crawl corpus' (C4) heuristic filtering include [68]:

- Removing lines not ending in a terminal punctuation mark.
- Filtering documents with Javascript alerts.
- Removing documents with fewer than 5 sentences.
- Removing documents containing curly brackets (to filter out code).
- Filtering out filler text, such as "lorem ipsum," or boilerplate policy notes like "terms of use."

Quality filtering: Classifier-based

- Classifier-based methods use machine learning models to identify and filter undesirable content.
  - We train classifiers to distinguish between "high-quality" and "low-quality" documents.
  - High-quality training data is often derived from curated sources like Wikipedia or Books.

Example (Classifier-based quality filtering)

- PaLM and GLAM use a linear classifier based on features, e.g., similar to Wikipedia and Books [14, 21].
- LLaMA uses pages referenced within Wikipedia as high-quality examples [84].

Remarks:

- Classifier-based filtering might be too strict; we can use stochasticity in the selection [10]
  - to allow some lower-scoring documents to be included.

Quality filtering: Impact

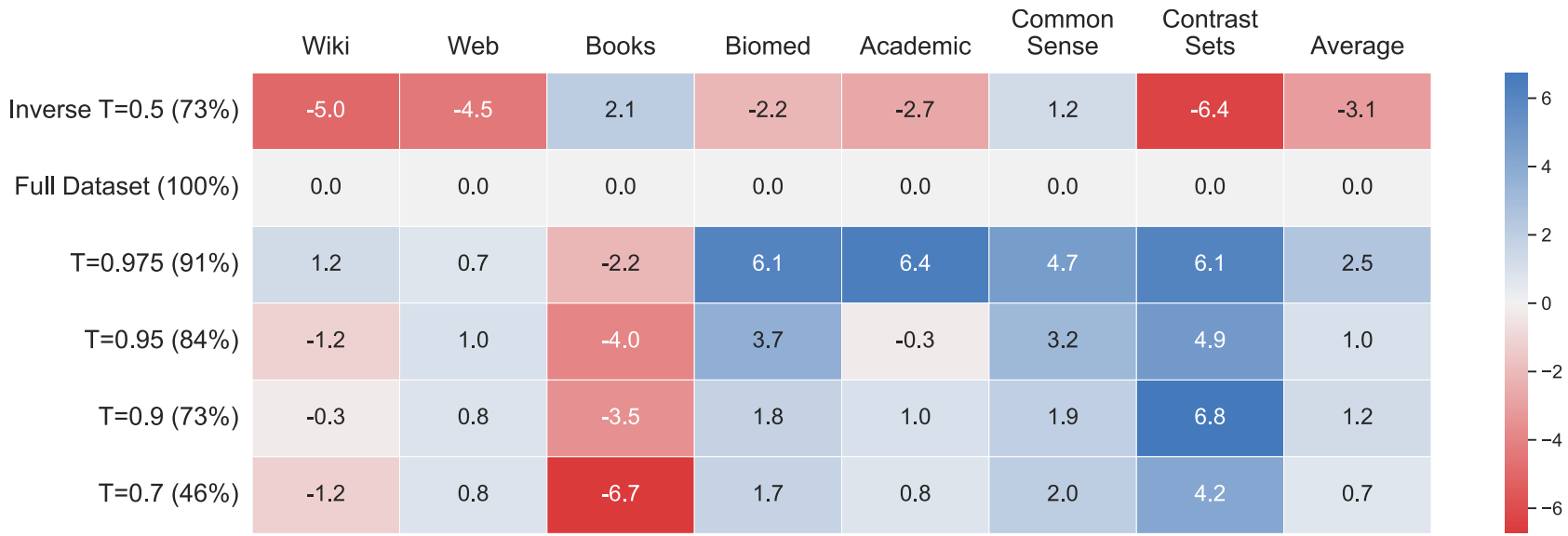


Figure: Performance when applying quality filtering to C4. The quality filter threshold is on the x-axis, with percentage of remaining training data in parenthesis. "Full Dataset" is unfiltered, "Inverse" removes highest quality data. From [59].

Remarks:

- Quality filtering improves performance on almost all downstream tasks.
- Removing $\approx 10\%$ of training data can significantly improve performance.

Data cleaning: Data age

- Language evolves, impacting the LLM performance on temporally misaligned data [59].
  - Temporal misalignment: Discrepancy between the time periods in training and evaluation [50, 60, 4].

Example

An LLM trained on 2019 data may perform poorly on a 2023 evaluation set containing information about events happened after 2019. Similarly, a model trained on 2023 data may struggle with nuances of language and information relevant to a 2019 evaluation set.

Remarks:

- Performance degradation is pronounced when the evaluation year is *after* the pretraining year [60].
- Larger models exhibit greater sensitivity to temporal misalignment than smaller models.
- Temporal misalignment complicates evaluation.
  - Older evaluations may underestimate newer models.
  - Newer evaluations may underestimate older models w.r.t. their original context.

Document packing: Overview

- Training sequences often contain significant padding, wasting computation.
- Document packing combines multiple short sequences into single longer sequences to:
  - Reduce padding waste.
  - Increase computational efficiency.
  - Improve training throughput.
- Different approaches address this problem:
  - Standard packing [86]: Simple concatenation of sequences.
  - Cross-contamination prevention [46]: Mask attention to avoid leakage from independent sequences.
  - Optimized implementations [55]: Models like DeepSeek-V3 incorporate packing while preserving integrity.
- Document packing can yield 2-3× training speedup without sacrificing model quality.

Data mixing problem

- Pretraining an LLM requires large-scale data from diverse sources, such as the Pile and SlimPajama [29, 73].

Data Source	Sampling (%)	Train Tokens	Validation Tokens	Train (% of Total)
CommonCrawl	67.0	200.82B	214.72M	66.99
C4	15.0	44.98B	48.06M	15.00
GitHub	4.5	13.51B	14.42M	4.51
Books	4.5	13.49B	14.39M	4.50
Wikipedia	4.5	13.50B	14.41M	4.50
ArXiv	2.5	7.49B	8.01M	2.50
StackExchange	2.0	5.99B	6.41M	2.00
Total	100.0	299.78B	320.42M	100.00

Table: SlimPajama dataset [73]

- Denote k domains for pretraining as $D_{\text{train}} = \{D_1, D_2, \ldots, D_k\}$.
- The sampling distribution is $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_k) \in \triangle^k$, the probability simplex over k domains.
- How to find the optimal data mixture weight $\boldsymbol{\alpha}$?

Suboptimal heuristics data mixing methods

- Heuristics data mixing methods:
  - Uniform sampling across domains: $\alpha_i = \frac{1}{k}$
  - Sample-level uniform sampling: $\alpha_i = \frac{|D_i|}{|D_{\text{train}}|}$
  - Manual selection: Upweight high-quality sources like Wikipedia and academic texts [10, 29]
- Most current well-known LLMs still use manual selection strategies.
  - Staged training with various mixtures like Gemini [78, 79], Qwen [93, 94], Gemma [80, 81].
  - DeepSeek-V3 increases focus on math and programming samples compared to DeepSeek-V2 [18].
  - Qwen 2.5 uses Qwen2-Instruct models to classify and balance content across different domains [94].
- These heuristic methods may be suboptimal since they ignore:
  - Relevance of topics
  - Variance data quality
  - Toxicity, duplication etc.

Proxy-based algorithms: DoReMi [90]

- DoReMi is a three-step algorithms.
  1. Train a small reference model $\mathbf{x}_{\text{ref}}$ with uniform domain weights.
  2. Train proxy model with group distributionally robust optimization [69] to obtain domain weights.
     - Trains a robust model by optimizing the worst-case loss over domains:

$$\min_{\mathbf{x}} \max_{\boldsymbol{\alpha} \in \triangle^k} L(\mathbf{x}, \boldsymbol{\alpha}) := \sum_{i=1}^{k} \alpha_i \cdot [f_i(\mathbf{x}) - f_i(\mathbf{x}_{\text{ref}})]$$

     - The domain weights are guided by loss difference. At step t,

$$\alpha_i^t \propto \alpha_i^{t-1} \exp\Big(f_i(\mathbf{x}^t) - f_i(\mathbf{x}_{\text{ref}})\Big).$$

  3. Train large model with new domain weights.

Proxy-based algorithms: DoReMi [90]

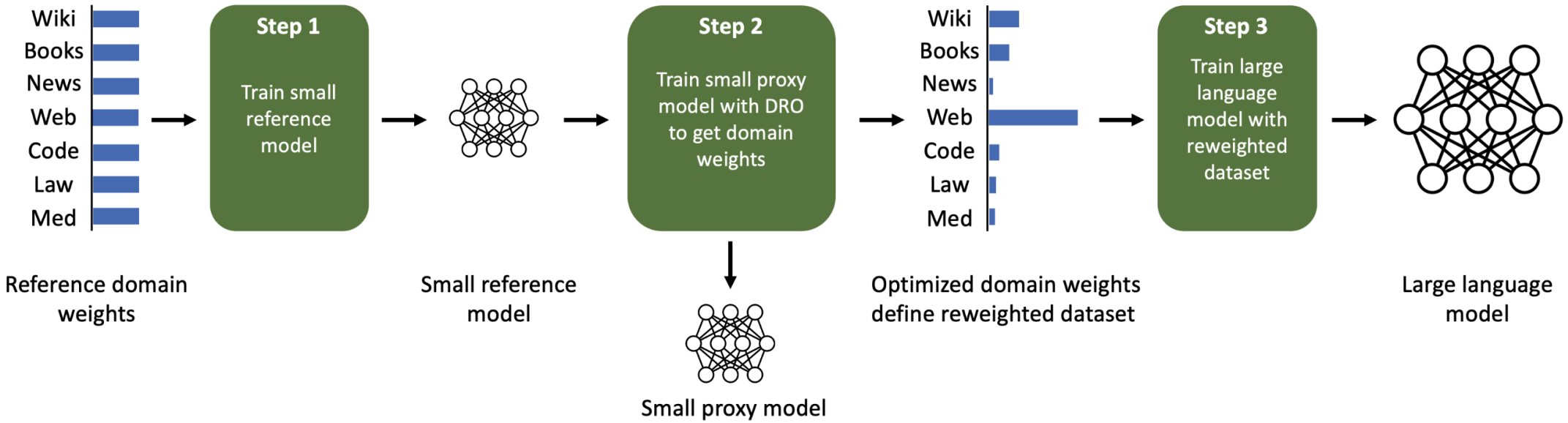


Figure: DoReMi's pipeline [90]

Remarks:

- DoReMi is not efficient enough since it requires two auxiliary models.
- α highly depends on the performance of reference model.

Optimal domain weights are predictable

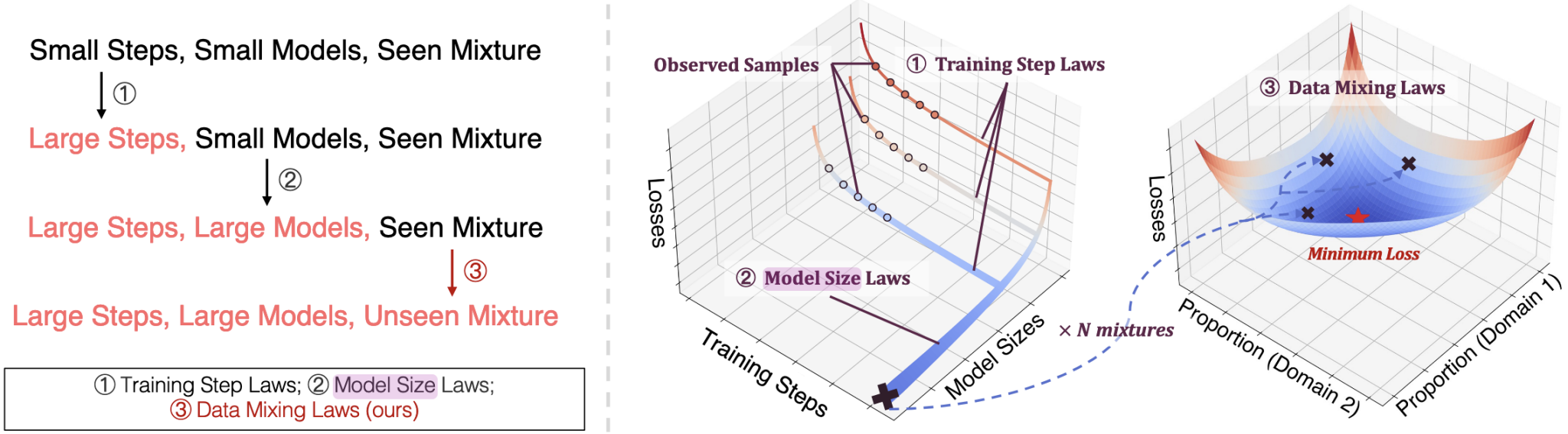


Figure: Data mixing law [97]

- [97, 45] find that the domain losses regarding the domain proportion precisely fit into an exponential law.
- [57] grid searches domain proportions and predicts optimal weights via regression.
- [45] investigates the scaling law of data size in the domain mixing problem.
- [97] predicts domain weights through using scaling laws of training steps, model sizes, and data mixing laws.

From real to synthetic data: Data augmentation vs. Knowledge distillation

- Data augmentation [75]
  - Generates additional data samples from existing datasets.
  - Includes transformations, paraphrasing, or synthetic text generation.
  - Often aims to improve generalization in data-limited scenarios.
- Mathematically:

$$\mathcal{D}_{\text{aug}} = f(\mathcal{D}_{\text{orig}})$$

where f is a transformation function that increases dataset diversity.

- Knowledge distillation [37]
  - Transfers knowledge from a large teacher model (T) to a smaller student model (S).
  - Involves a loss function comparing student predictions with soft labels from the teacher.
  - Can use KL-divergence, mean squared error, or other objectives.
- Mathematically:

$$\mathcal{L}_{\text{KD}} = \text{KL}(\sigma(z_{\text{Teacher}}) \parallel \sigma(z_{\text{Student}}))$$

where σ is the softmax function, $z_{\text{Teacher}}, z_{\text{Student}}$ are teacher and student logits.

Remark: ○ One approach is to generate synthetic data from the teacher model for data augmentation.

Synthetic continued pretraining results [96]

- Balancing synthetic and real data
  - Synthetic data helps but has diminishing returns:

$$\mathcal{L}(N_{\text{synth}}) = A + B \log(N_{\text{synth}})$$

  - Synthetic CPT uses a balanced approach:

$$\min_{\theta} \sum_{(\mathbf{a},b)\in\mathcal{D}_{\text{synth}}} \mathcal{L}(h_{\mathbf{x}}(\mathbf{a}), b) + \lambda \sum_{(\mathbf{a},b)\in\mathcal{D}_{\text{real}}} \mathcal{L}(h_{\mathbf{x}}(\mathbf{a}), b)$$

    where λ is the balance factor.

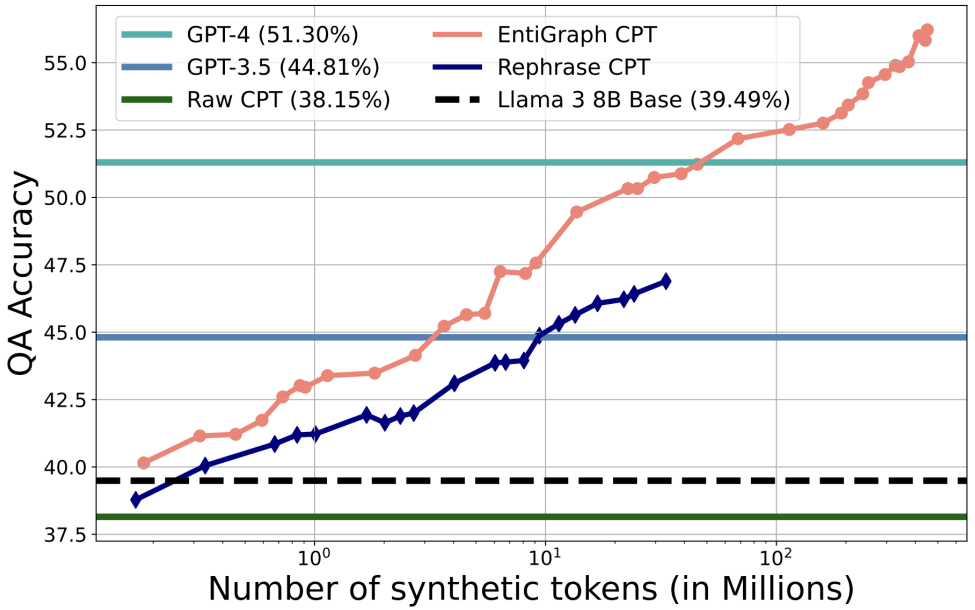


Figure: Impact of synthetic continued pretraining on QA accuracy. The x-axis shows the synthetic token count, and the y-axis shows model accuracy [96].

Wrap up

- The data-mixture selection is an open topic (lots of conflicting conclusions!)
- Project discussions next Thursday!
- We will also have a brief introduction to the CSCS cluster

MinHash and LSH on four documents

```
from datasketch import MinHash, MinHashLSH
def shingles(t, k=5):            # 5-word windows
    w = t.lower().split()
    return {" ".join(w[i:i+k]) for i in range(len(w)-k+1)}

lsh = MinHashLSH(threshold=0.7, num_perm=128)
for doc_id, text in docs.items():
    m = MinHash(num_perm=128)
    for sh in shingles(text): m.update(sh.encode())
    lsh.insert(doc_id, m)

lsh.query(minhash_of["A"])      # -> ['A', 'D']
```

PAIR	WHAT DIFFERS	ESTIMATED JACCARD
A vs D	one punctuation mark	0.85 → caught
A vs B	two synonyms swapped	0.27 → missed
A vs C	unrelated text	0.00

- MinHash approximates Jaccard similarity of shingle sets with a fixed-size signature; LSH finds candidate pairs **without comparing everything to everything**.
- The knobs are the shingle size and the threshold: 5-word shingles missed a paraphrase that a human calls a duplicate. Corpus pipelines tune these per source (CCNet, Dolma, FineWeb all publish theirs).
- Semantic near-duplicates need embeddings (SemDeDup); exact duplicates need only a hash. Real pipelines run all three levels: URL, document, and line.

Gopher-style rules in twelve lines

```
def quality_flags(t):
    w = t.split(); flags = []
    if not 50 <= len(w) <= 100_000:
flags.append("length")
    mean_len = sum(map(len, w)) / len(w)
    if not 3 <= mean_len <= 10:       flags.append("word
length")
    if sum(c.isalpha() for c in t)/len(t) < 0.8:
        flags.append("symbols")
    if not any(x in w for x in STOP8):  flags.append("no
stop words")
    return flags

quality_flags("BUY NOW!!! 50% OFF ### click here >>> ...")
# ['length', 'symbols', '... lines', 'no stop words']
quality_flags(three_clinical_sentences)   # []
```

- The rules from the Gopher paper (Rae et al. 2021) and C4: length bands, symbol ratio, ellipsis lines, stop-word presence. Cheap, transparent, and **English-shaped**.
- "No stop words" fails every Thai document by construction; "mean word length" is meaningless without segmentation. **Port the rules per language** or you filter a language out.
- Classifier filters (GPT-3's Wikipedia-vs-web, FineWeb-Edu's educational score) are the next rung: more precise, and biased in ways part E3 shows.

What deduplication and filtering actually bought

3.04%

of C4 examples are near-duplicates; one 61-word English sentence appears over 60,000 times (Lee et al., ACL 2022)

10x

less memorized text emitted after deduplication; train-test overlap affected over 4% of a standard validation set (Lee et al. 2022)

92%

of FineWeb removed by an educational-quality classifier at threshold 3 or above to make FineWeb-Edu, 1.3T tokens (Penedo et al., NeurIPS 2024)

33 → 37

MMLU after the FineWeb-Edu filter, ARC 46 to 57, in the dataset authors' own ablations across 70+ models

- Read the gains with the course's standing caveat: these are the **dataset authors' own ablations**, strong but not independent (section 2.4). DCLM reaches similar conclusions with a different classifier (fastText on OpenHermes and ELI5).
- The recurring finding across FineWeb-Edu, DCLM, and the mixing literature is **quality over quantity**: throwing away most of the web made the model better, not worse.
- Deduplication is also a **privacy** intervention: Kandpal et al. (ICML 2022) link duplicate counts to extraction risk. One fix, three benefits.

PART E3 · 1:50–2:05

Contamination, ownership, and what "quality" quietly excludes

Your test set may be in the training data; the training data may be someone's copyrighted book; and the quality classifier prefers wealthy suburbs. Three audits every curation pipeline owes its users.

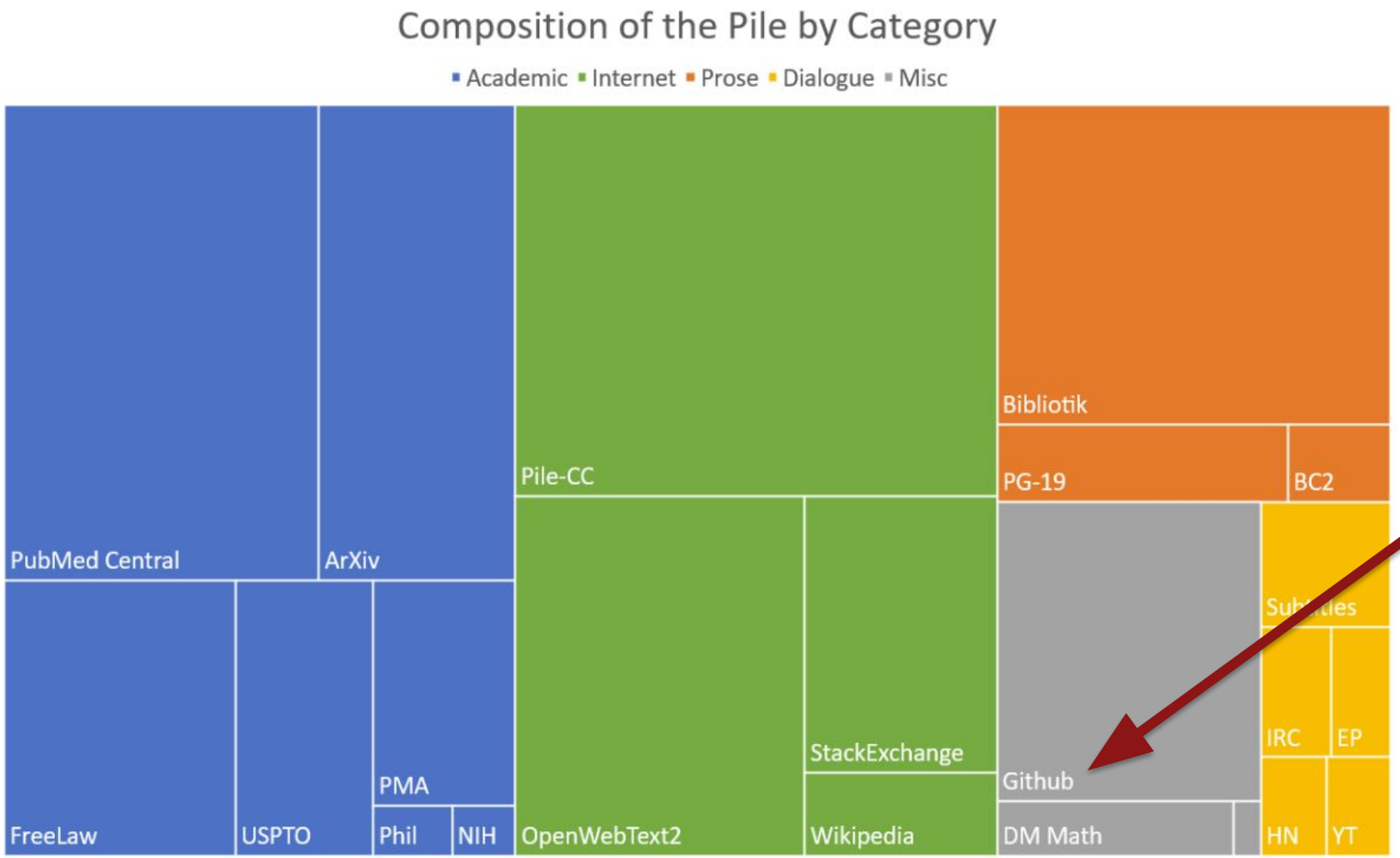
SOURCE DECKS

Lecture 11: Evaluation (CS224N Winter 2026, Stanford) · **Lecture 5: Data, Data, and Data**, ownership and bias sections (CS 329X, William Held, Stanford).

web.stanford.edu/class/cs224n · web.stanford.edu/class/cs329x

© the respective authors · shown unmodified · selected pages

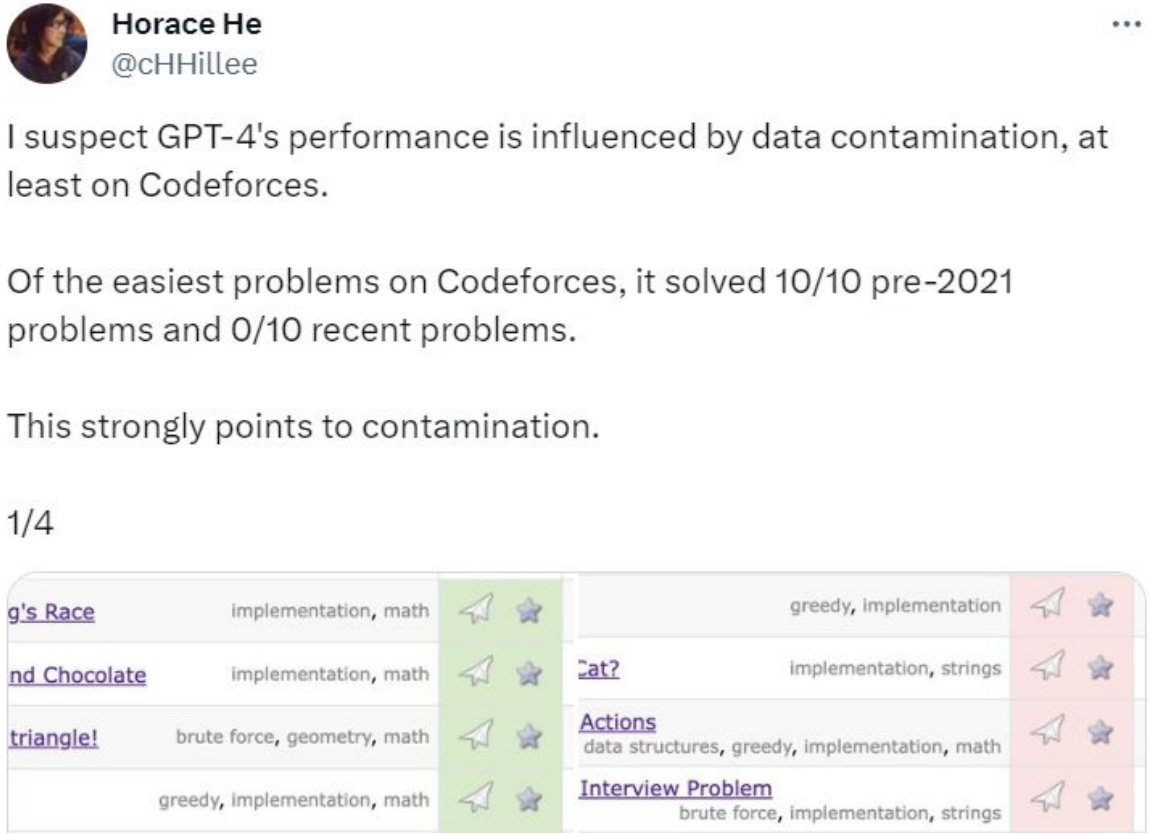
What is in the training data of a LLM

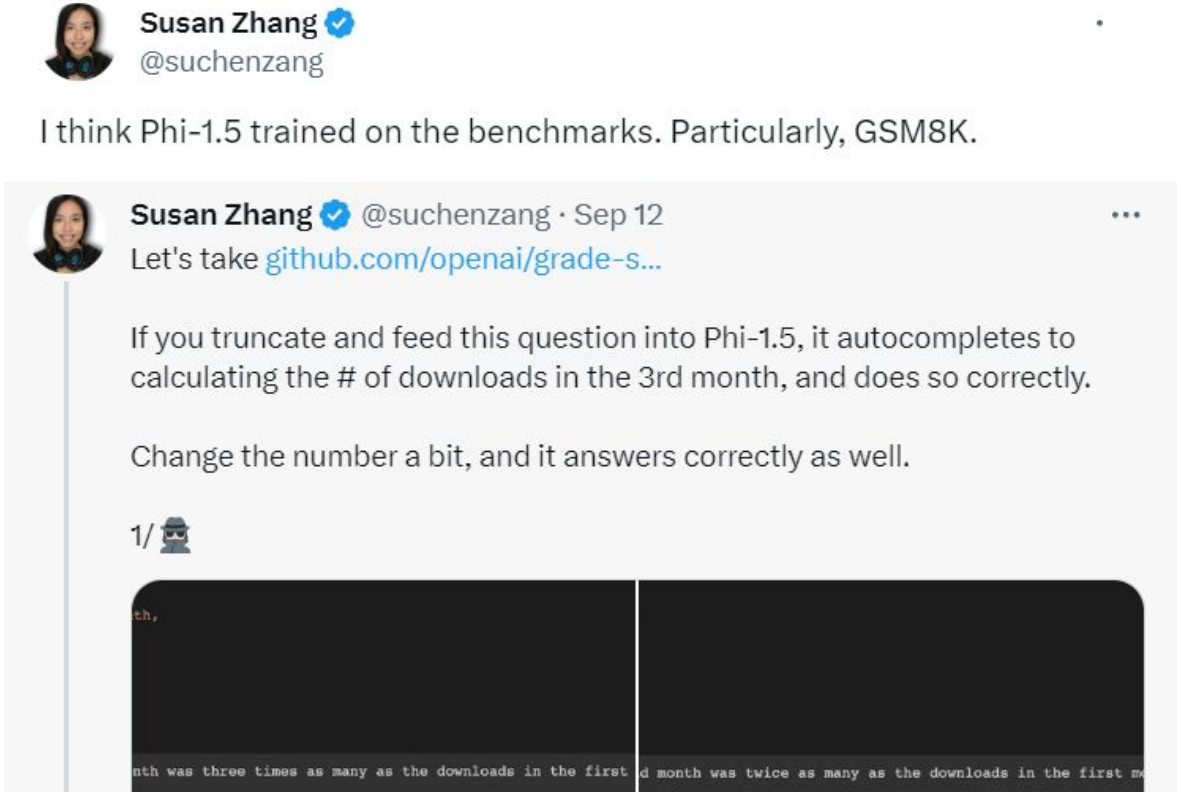


.. But maybe your test set is in here?

CODEFORCES
Sponsored by TON

Benchmarks can be hard to trust for pretrained models





Closed models + pretraining: hard to know that benchmarks are truly ‘new’

Data De-contamination!

- Why does this happen?
  - Why internet data can be contaminated?
  - Why synthetic data can be contaminated?
- Data de-contamination practice
  - N-gram overlap: check for exact or near-exact n-gram matches (commonly 8-13 grams) between training data and benchmark examples
  - Sometimes mixed with embedding-based or paraphase-based near-duplicate detection

Data Ownership - Copyright is King!

Copyright protection applies to 'original works of authorship fixed in any tangible medium of expression, now known or later developed, from which they can be perceived, reproduced, or otherwise communicated, either directly or with the aid of a machine or device'

- Copyright applies to specific works, not to the ideas underlying those works!
- Copyright applies retroactively once registered.
- Since copyright applies only to the specific instance of work, the threshold is relatively low.

Most of the internet is either explicitly (due to website terms of service) or implicitly copyrighted!
This means if you want to train a LLM you only have two routes

- Pay the rights holders money for a license!
- Argue that you don’t need to pay them due to fair use

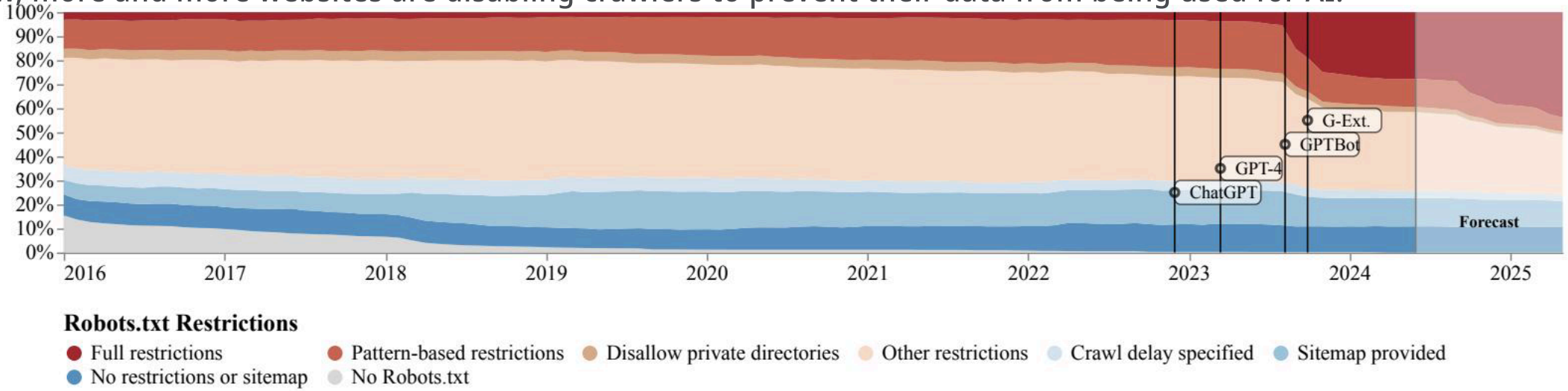
I am not a lawyer, but some real smart lawyers are looking at this:
https://simons.berkeley.edu/news/large-language-models-meet-copyright-law

Copyright is not the only data ownership mechanism

Robots.txt is a file websites can host at their root that defines their rules for how web-scrapers should access them.

Before LLMs, the vast majority of websites had relatively permissive scraping rules so that they could be indexed by search engines!

Now, more and more websites are disabling crawlers to prevent their data from being used for AI.

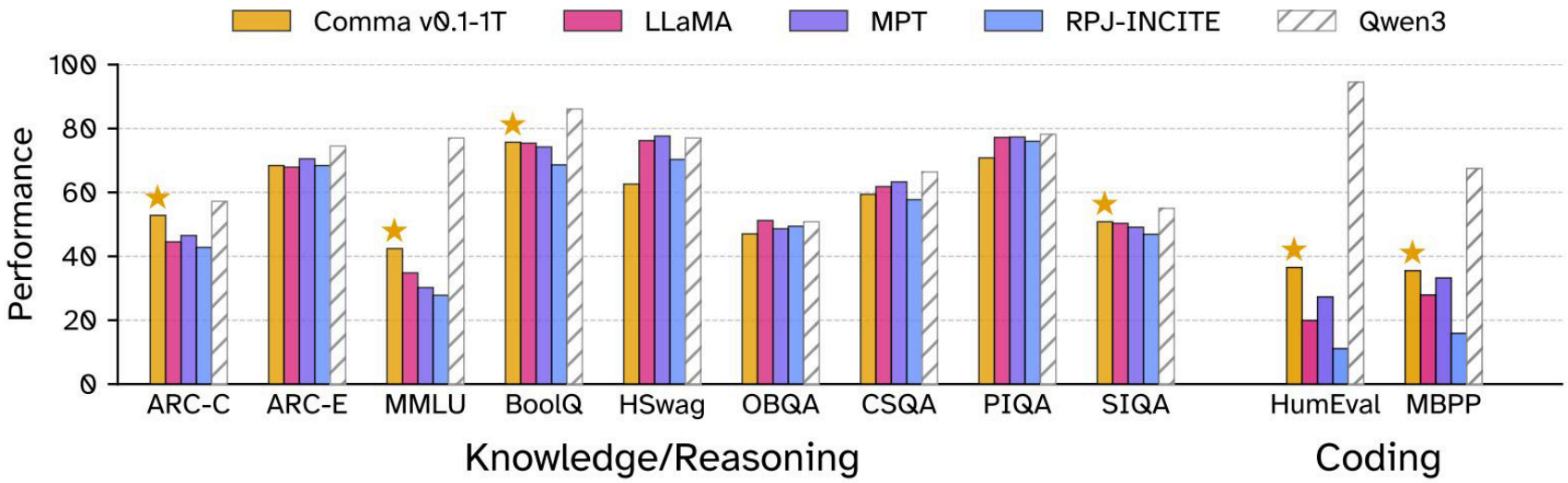


Longpre et al. 2024

How far can we get with truly open data?

The CommonPile focuses on collecting as much data as possible that is released under this permissive license!

The resulting model trained on this data is ~3 years behind the “open-weights” state-of-the-art.



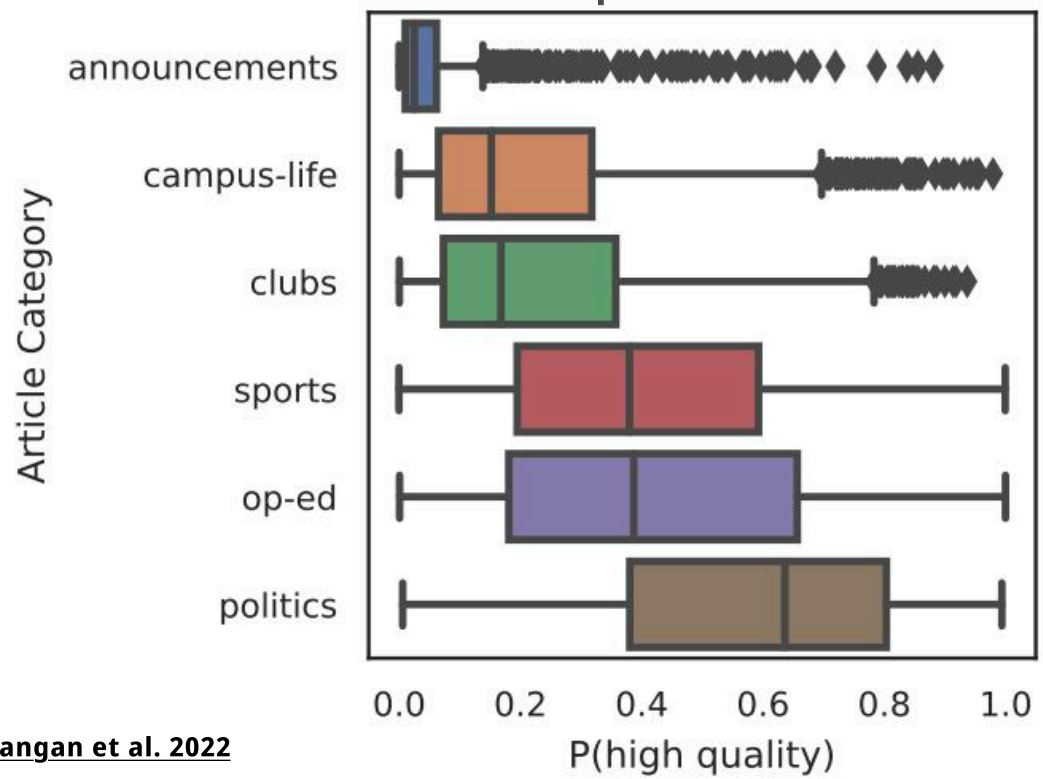
Kandpal et al. 2025

Topical Bias in Quality Classification

Top Source Domains

Table 1: The most p
in OpenWebText. Ma
whelming majority of

Topical Biases



Gururangan et al. 2022

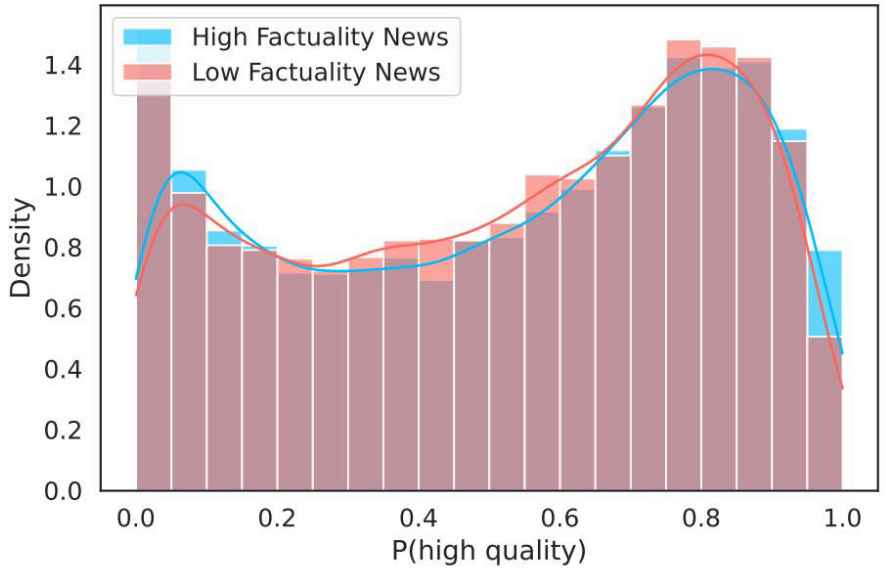
Socioeconomic Bias in Quality Classification

Dependent variable: $P(\text{high quality})$
Observations: 968 schools

Feature	**Coefficient**
Intercept	0.076
% Rural	-0.069^{***}
% Adults $\geq$ Bachelor Deg.	0.059^{**}
$\log_2$(Median Home Value)	0.010^{*}
$\log_2$(Number of students)	0.006^{*}
$\log_2$(Student:Teacher ratio)	-0.007
Is Public	0.015^{*}
Is Magnet	0.013
Is Charter	0.033
R^2	0.140
adj. R^2	0.133

Gururangan et al. 2022

What does this definition of “quality” end up capturing?



Gururangan et al. 2022

n-gram overlap between a training document and a test item

```
def ngrams(t, n=13):
    w = t.lower().split()
    return {" ".join(w[i:i+n]) for i in range(len(w)-n+1)}

overlap = ngrams(train_doc) & ngrams(test_item)
len(overlap), len(ngrams(test_item))     # (4, 5)
# 'bangkok is the capital and most populous city of
#  thailand, located on the' ... appears verbatim

# practice: 8- to 13-gram matches, sometimes plus
# embedding or paraphrase near-duplicate detection
```

- Four of the five 13-grams in the test item sit verbatim in a training page: the benchmark score for that item measures **memory**, not ability.
- This is the leakage taxonomy from section 1.11 at internet scale: Kapoor and Narayanan's train-test contamination, now impossible to rule out for closed models (CS224N slide 72).
- Decontaminate **the training set against the benchmarks** before training, publish the procedure, and treat surprising benchmark jumps as a contamination hypothesis first.
- Synthetic data can contaminate too: an LLM that saw the benchmark writes benchmark-shaped "new" examples.

PART F1 · 2:05–2:25

Instruction data: the smallest dataset that matters most

From 52k machine-written Alpaca pairs to 1,000 hand-picked LIMA examples: how instruction data is generated, formatted, filtered, and who writes the labels.

SOURCE DECKS

Lecture 8: Post-training (CS224N Winter 2026, Stanford) · **Lecture 4: Fine-tuning**, instruction tuning and SFT datasets sections, EE-628 Training LLMs, EPFL (Cevher and Gulcehre).
web.stanford.edu/class/cs224n · epfl.ch/labs/lions/teaching/ee-628-training-large-language-models/

CS224N © the authors, shown unmodified · EPFL deck CC BY-NC-SA

Language modeling ≠ assisting users

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION GPT-3

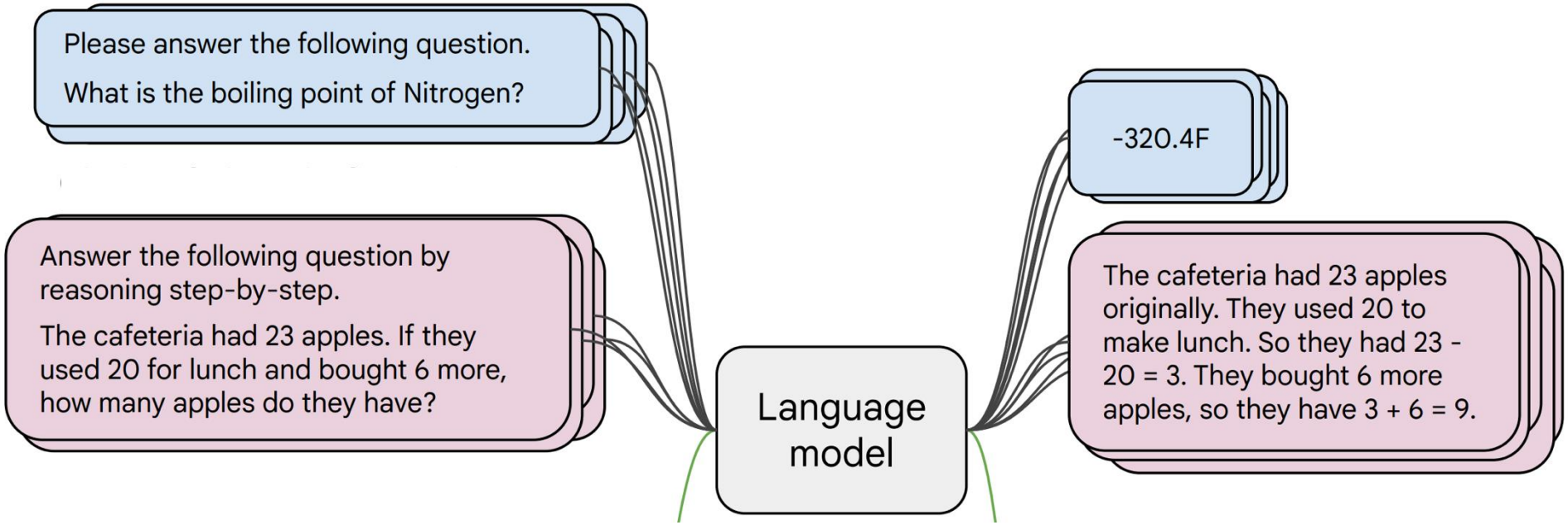
```
Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.
```

Language models are not *aligned* with user intent [Ouyang et al., 2022]

Finetuning to the rescue!

Instruction finetuning

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM

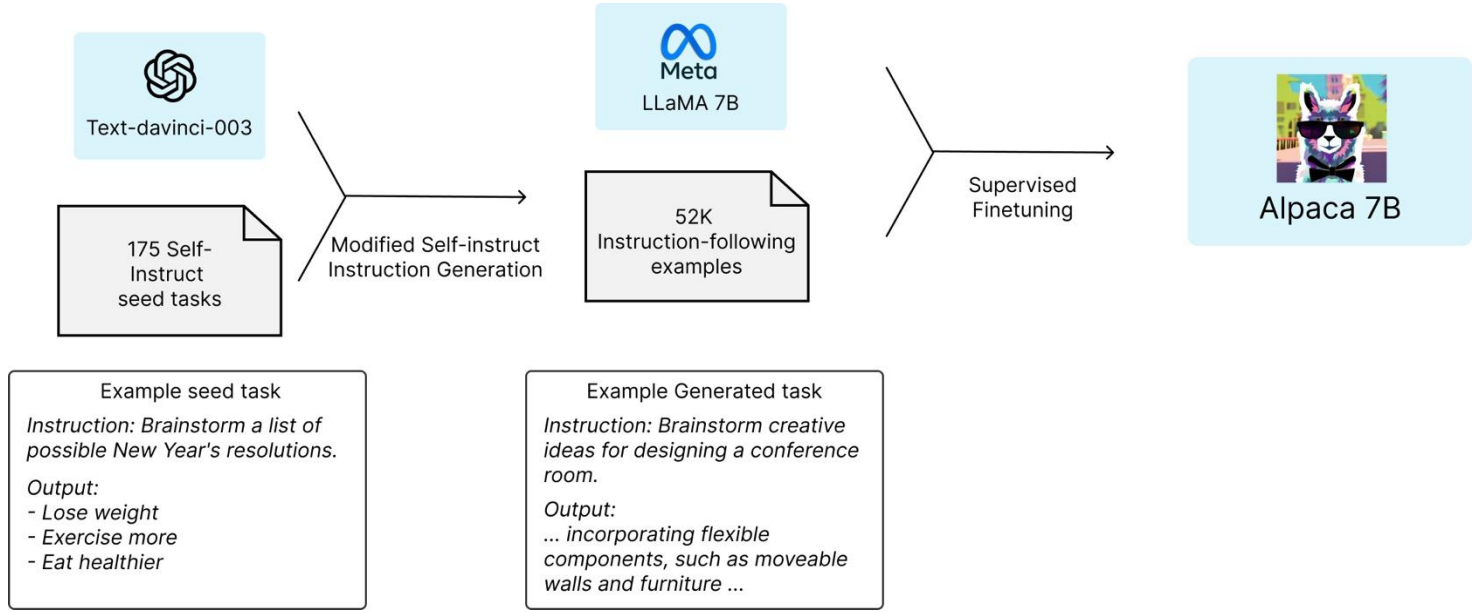


- Evaluate on **unseen tasks**

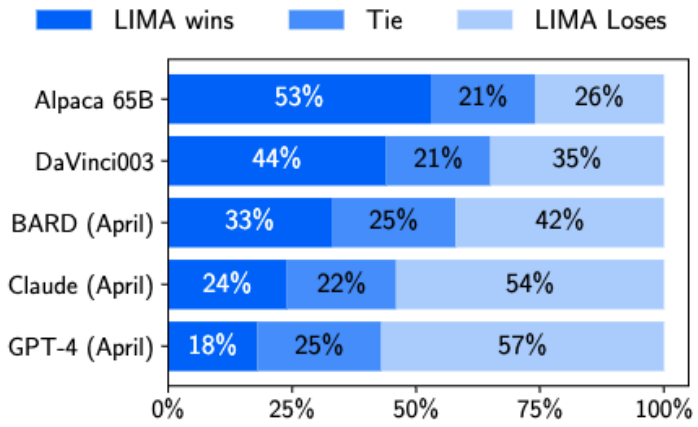
[FLAN-T5; Chung et al., 2022]

What have we learned from this?

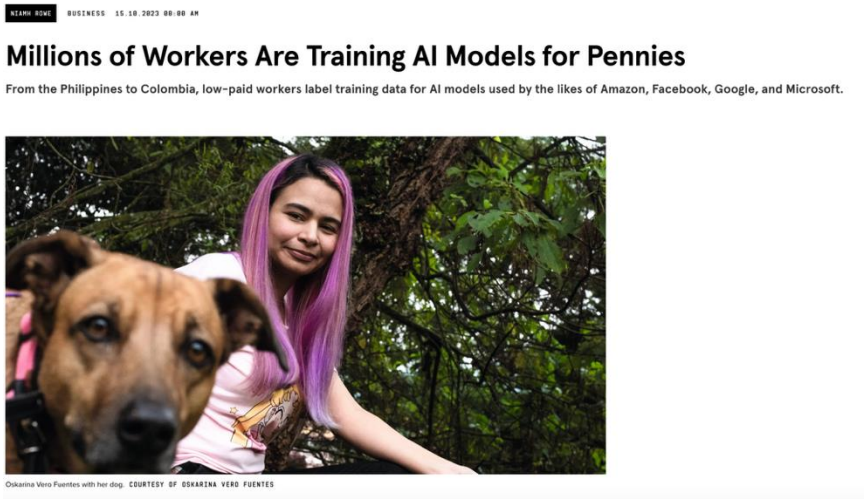
- Generate instructions, input, and output from a LM [Wang et al., 2022]
  - **Alpaca**: fine-tuned from the LLaMA 7B model on 52K instruction-following examples
- You don't need many samples to instruction tune (e.g., "*LIMA: Less Is More for Alignment*" Zhou et al., 2023)



Source	#Examples
Training	
Stack Exchange (STEM)	200
Stack Exchange (Other)	200
wikiHow	200
Pushshift r/WritingPrompts	150
Natural Instructions	50
Paper Authors (Group A)	200



Where does the labels come from?

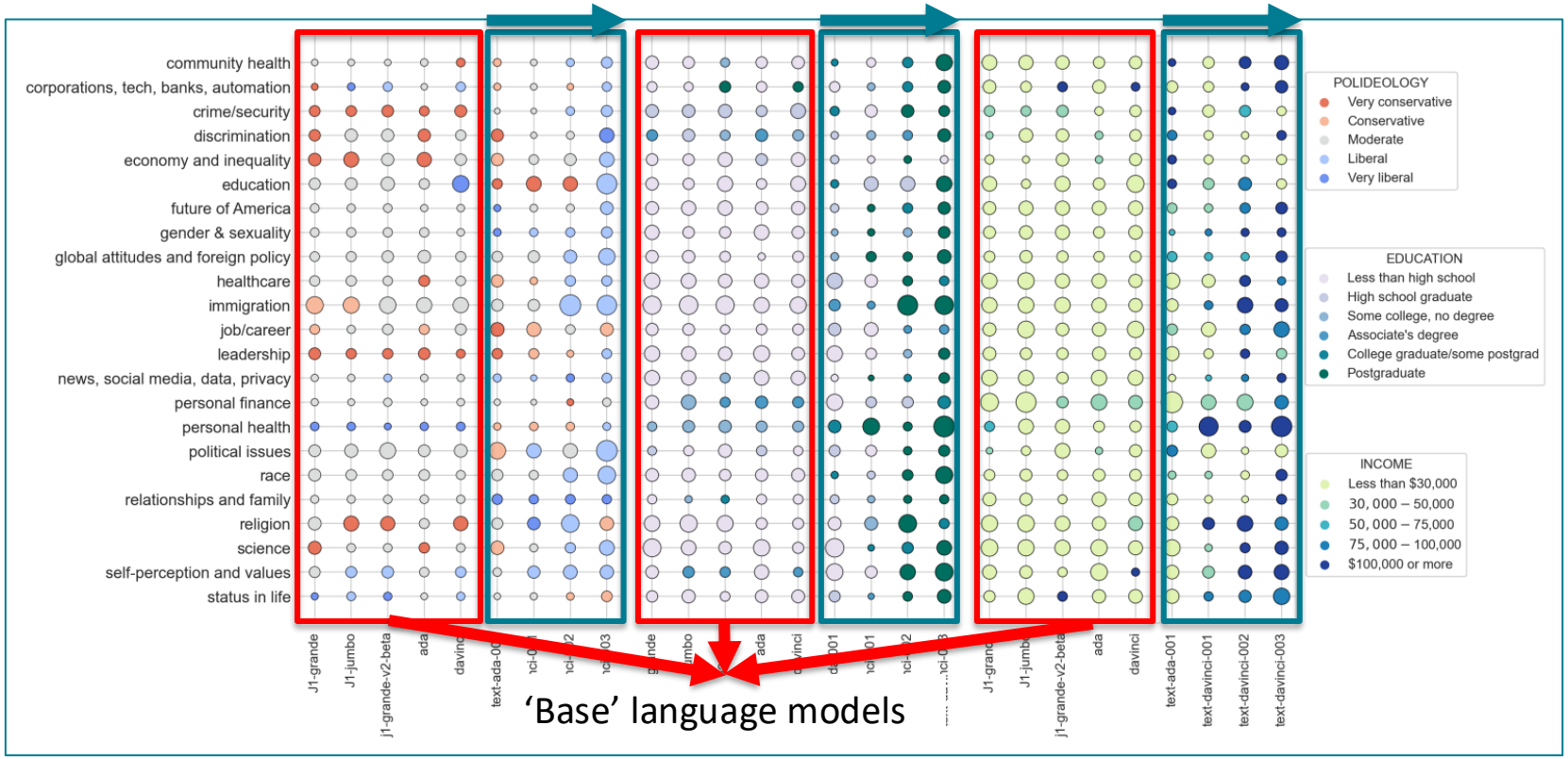


- RLHF labels are often obtained from overseas, low-wage workers

Where does the label come from?

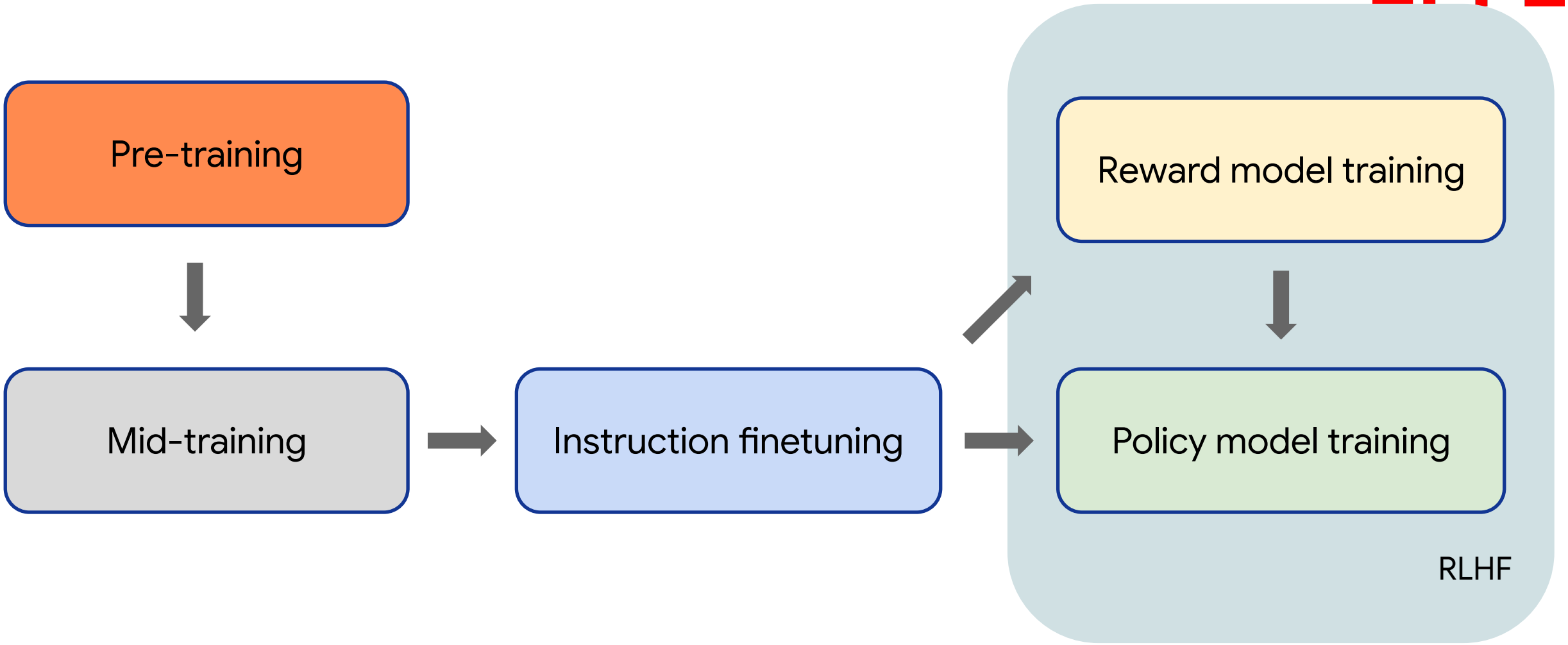
Table 12: Labeler demographic data

What gender do you identify as?	
Male	50.0%
Female	44.4%
Nonbinary / other	5.6%
What ethnicities do you identify as?	
White / Caucasian	31.6%
Southeast Asian	52.6%
Indigenous / Native American / Alaskan Native	0.0%
East Asian	5.3%
Middle Eastern	0.0%
Latinx	15.8%
Black / of African descent	10.5%
What is your nationality?	
Filipino	22%
Bangladeshi	22%
American	17%
Albanian	5%
Brazilian	5%
Canadian	5%
Colombian	5%
Indian	5%
Uruguayan	5%
Zimbabwean	5%
What is your age?	
18-24	26.3%
25-34	47.4%
35-44	10.5%
45-54	10.5%
55-64	5.3%
65+	0%
What is your highest attained level of education?	
Less than high school degree	0%
High school degree	10.5%
Undergraduate degree	52.6%
Master's degree	36.8%
Doctorate degree	0%



[Santurkar+ 2023, OpinionQA]

- We also need to be quite careful about how annotator biases might creep into LMs


Pre-training
Mid-training
Instruction finetuning
Reward model training
Policy model training
RLHF

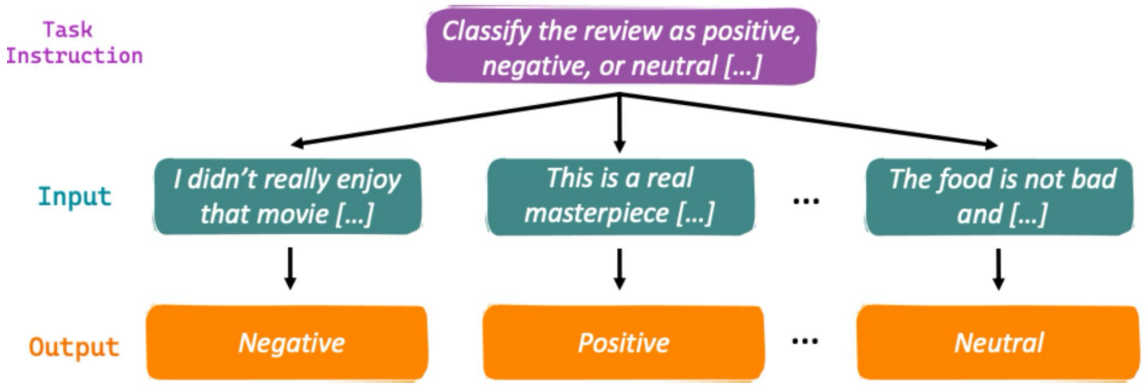
Instruction finetuning

1

*Frame **all** tasks in the form of*

natural language instruction to natural language response mapping

Instruction tuning dataset designs



a) <u>Scaling inputs</u>

For each task instruction, scaling up various input-output pairs.

- Conventional multi-task learning paradigm.

<u>b) Scaling input-free tasks</u>

Scaling up instruction-output pairs directly. (e.g., Self-Instruct and Alpaca).

- Input contexts are omitted / tightly combined with instructions.

Insights to the finetuning datasets

A typical format of an instruction dataset is as follows:

```
Below is an instruction that describes a task, paired with an input that provides
further context. Write a response that appropriately completes the request.

### Instruction:
{instruction}

### Input:
{input}

### Response:
```

These messages are typically formatted in JSON:

```
{
  "messages": [
    {"role": "system", "content": "You are a helpful and friendly AI assistant."},
    {"role": "user", "content": "What is the capital of France?"},
    {"role": "assistant", "content": "The capital of France is Paris."}
  ]
}
```

The roles typically are:

- **System:** Provides instructions on the assistant's behavior and response style.
- **User:** Represents the user's input or query.
- **Assistant:** Represents the chatbot's response given the instruction.

Self-instruct

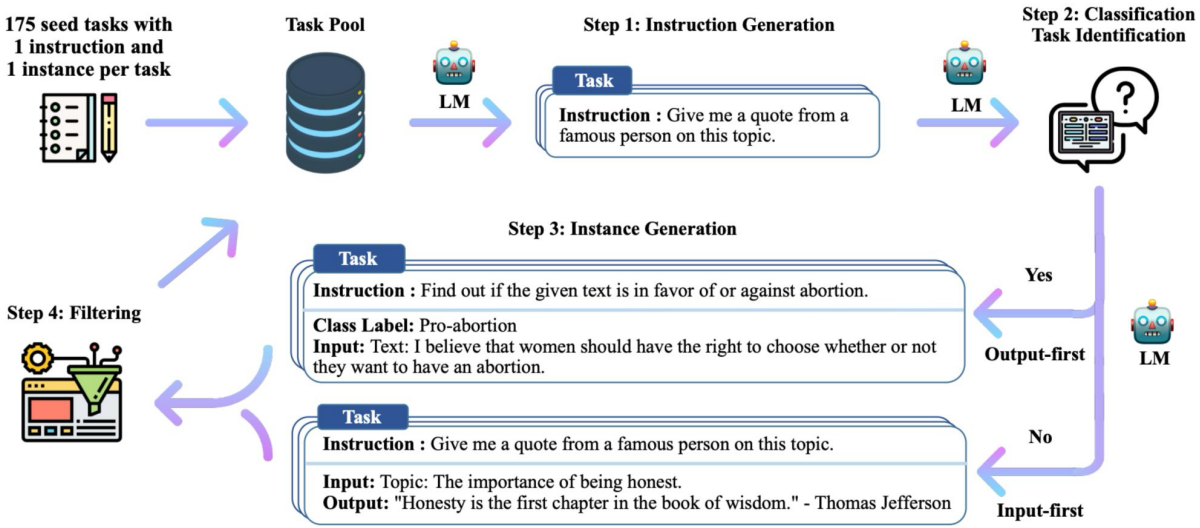
Wang et. al., 2023 proposed a dataset generated with bootstrapping method to automatically create instruction-response pairs for training instruction-tuned language models without extensive human annotation.

Step 1: Form the seed set

Create a seed set with a small set of manually created human-written instructions and responses.

Step 2: Instruction Generation (Self-Instruct)

Existing large language model (such as GPT-3) is prompted with examples from the seed set to generate new instructions.



Example prompt:

```
Here are some tasks and their responses:
Task: Explain what the greenhouse effect is.
Response: The greenhouse effect is...
Task: Give three examples of mammals.
Response: Examples include...
Now generate another new task:
```

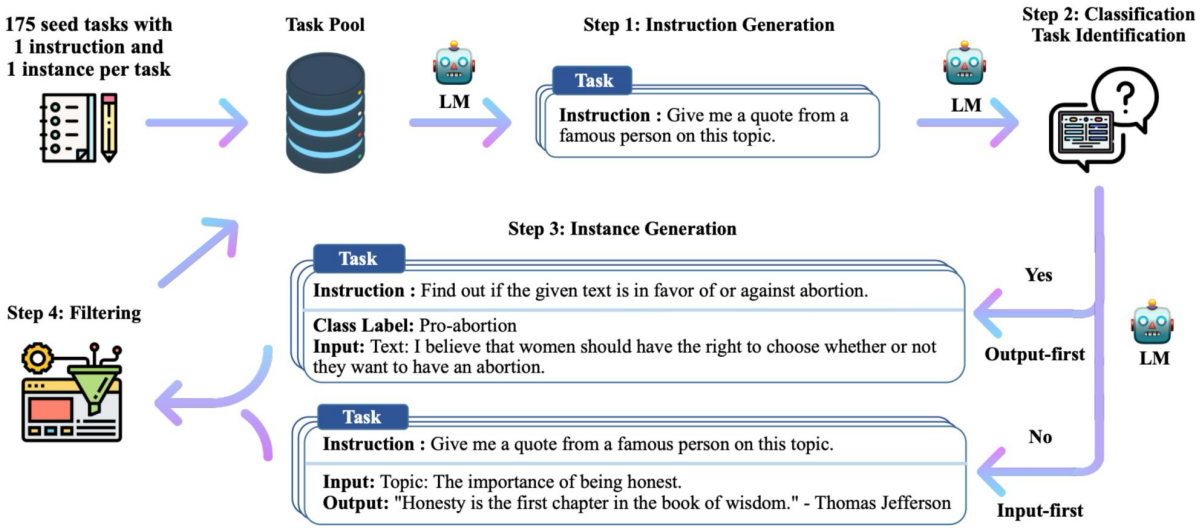
Self-instruct

Step 3: Filtering and Selection

- Automatically filter and prune instructions that are too similar, low quality, ambiguous, or repetitive, using heuristic or model-based criteria.
- The filtering ensures instruction diversity and quality.

Step 4: Response Generation

- For each new instruction retained after filtering, the same language model (or another capable model) is prompted to generate a suitable response.



Example prompt:

```
Task: Write a short summary of
the movie "Inception".
Response:
```

Alpaca

52k instructions and demos generated by OpenAI's `text-davinci-003` engine.
Built using the Self-Instruct recipe with the following modifications:

- The `text-davinci-003` engine to generate the instruction data instead of davinci.
- A new prompt was written that explicitly gave the requirement of instruction generation to `text-davinci-003`.
- Much more aggressive batch decoding was used, i.e., generating 20 instructions at once, which significantly reduced the cost of data generation.
- The data generation pipeline was simplified by discarding the difference between classification and non-classification instructions.
- Only a single instance was generated for each instruction, instead of 2 to 3 instances as in Self-Instruct.

Stanford Alpaca

Alpaca - An Example


```
{
    "instruction": "Create a classification task by clustering the given list of items.",
    "input": "Apples, oranges, bananas, strawberries, pineapples",
    "output": "Class 1: Apples, Oranges\nClass 2: Bananas, Strawberries\nClass 3: Pineapples",
    "text": "Below is an instruction that describes a task, paired with an input that provides further context.
}
```


- *instruction:* describes the task the model should perform. Each of the 52K instructions is unique.
- *input:* optional context or input for the task. For example, when the instruction is "Summarize the following article", the input is the article. Around 40% of the examples have an input.
- *output:* the answer to the instruction as generated by text-davinci-003.
- *text:* the instruction, input and output formatted with the prompt template used by the authors for fine-tuning their models.

FLAN

One of the largest publicly available instruc

Diverse Tasks:

- NLP tasks like sentiment analysis, te

Focuses on **Multi-task learning** and instruct

Mostly curated from existing resources: sucl reformatted into instruction-response pairs.

Contains CoT, few-shot and zero-shot task t

		Model Details				Data Collection & Training Details			
Release	Collection	Model	Base	Size	Public?	Prompt Types	Tasks in Flan	# Exs	Methods
2020 05	UnifiedQA	UnifiedQA	RoBerta	110-340M	P	ZS	46 / 46	750k	
2021 04	CrossFit	BART-CrossFit	BART	140M	NP	FS	115 / 159	71.M	
2021 04	Natural Inst v1.0	Gen. BART	BART	140M	NP	ZS / FS	61 / 61	620k	+ Detailed k-shot Prompts
2021 09	Flan 2021	Flan-LaMDA	LaMDA	137B	NP	ZS / FS	62 / 62	4.4M	+ Template Variety
2021 10	P3	T0, T0+, T0++	T5-LM	3-11B	P	ZS	62 / 62	12M	+ Template Variety + Input Inversion
2021 10	MetaICL	MetaICL	GPT-2	770M	P	FS	100 / 142	3.5M	+ Input Inversion + Noisy Channel Opt
2021 11	ExMix	ExT5	T5	220M-11B	NP	ZS	72 / 107	500k	+ With Pretraining
2022 04	Super-Natural Inst.	*Tk*-Instruct	T5-LM, mT5	11-13B	P	ZS / FS	1556 / 1613	5M	+ Detailed k-shot Prompts + Multilingual
2022 10	GLM	GLM-130B	GLM	130B	P	FS	65 / 77	12M	+ With Pretraining + Bilingual (en, zh-cn)
2022 11	xP3	BLOOMz, mT0	BLOOM, mT5	13-176B	P	ZS	53 / 71	81M	+ Massively Multilingual
2022 12	Unnatural Inst.†	T5-LM-Unnat. Inst.	T5-LM	11B	NP	ZS	~20 / 117	64k	+ Synthetic Data
2022 12	Self-Instruct†	GPT-3 Self Inst.	GPT-3	175B	NP	ZS	Unknown	82k	+ Synthetic Data + Knowledge Distillation
2022 12	OPT-IML Bench†	OPT-IML	OPT	30-175B	P	ZS + FS, CoT	~2067 / 2207	18M	+ Template Variety + Input Inversion + Multilingual
2022 10	Flan 2022 (ours)	Flan-T5, Flan-PaLM	T5-LM, PaLM	10M-540B	P NP	ZS + FS, CoT	1836	15M	+ Template Variety + Input Inversion + Multilingual

Mid-training OLMO 2 Recipe

After the initial pre-training on mostly web-data, we would further train the model on a mixture of web data that is heavily filtered for quality and then they use a collection of high-quality data that is mainly synthetically generated.

The model used to generate the datasets change based on the task. For example, they used the best open-weights model for a specific task of interest.

They call their mid-training dataset as DolmaMix and used an approach called micro-annealing on math datasets.

Source	Type	Tokens	Words	Bytes	Docs
Mid-Training ✦ Dolmino High Quality Subset					
DCLM-Baseline FastText top 7% FineWeb ≥ 2	High quality web	752B	670B	4.56T	606M
FLAN from Dolma 1.7 decontaminated	Instruction data	17.0B	14.4B	98.2B	57.3M
peS2o from Dolma 1.7	Academic papers	58.6B	51.1B	413B	38.8M
Wikipedia & Wikibooks from Dolma 1.7	Encyclopedic	3.7B	3.16B	16.2B	6.17M
Stack Exchange 09/30/2024 dump curated Q&A data	Q&A	1.26B	1.14B	7.72B	2.48M
High quality total		**832.6B**	**739.8B**	**5.09T**	**710.8M**
Mid-training ✦ Dolmino Math Mix					
TuluMath	Synthetic math	230M	222M	1.03B	220K
Dolmino SynthMath	Synthetic math	28.7M	35.1M	163M	725K
TinyGSM-MIND	Synthetic math	6.48B	5.68B	25.52B	17M
MathCoder2 Synthetic Ajibawa-2023 M-A-P Matrix	Synthetic Math	3.87B	3.71B	18.4B	2.83M
Metamath OWM-filtered	Math	84.2M	76.6M	741M	383K
CodeSearchNet OWM-filtered	Code	1.78M	1.41M	29.8M	7.27K
GSM8K Train split	Math	2.74M	3.00M	25.3M	17.6K
Math total		**10.7B**	**9.73B**	**45.9B**	**21.37M**

Less, but better: four ways to choose the few thousand

METHOD	IDEA	HEADLINE RESULT (AUTHORS' REPORTS)
LIMA (Zhou et al., NeurIPS 2023)	1,000 carefully curated prompts and responses, no RL	a 65B model competitive with far larger tuned sets; the "superficial alignment hypothesis"
AlpaGasus (Chen et al., ICLR 2024)	an LLM scores Alpaca's 52k pairs; keep the good ones	9k of 52k, trained 5.7x faster, matches or beats the full set
DEITA (Liu et al., ICLR 2024)	score complexity, quality, and diversity; pick the frontier	about 6k examples rival sets ten times larger
LESS (Xia et al., ICML 2024)	optimizer-aware selection by low-rank gradient similarity to a target task	about 5% of the data can beat training on all of it
IFD / Cherry LLM (Li et al., NAACL 2024)	instruction-following difficulty from the model's own losses	self-guided selection without an external judge

- Common thread: the selection statistic is computed on **candidate training data**, never on the evaluation set; every one of these is a leakage question in disguise.
- Diversity is the axis that is easiest to lose when a judge model picks "high quality" examples that all look alike.

Augment with a model, but keep the humans in the loop

```
# LLM augmentation for a Thai clinical classifier
prompt = f"""Rewrite this note in the same clinical
register, keep every finding and the diagnosis,
change wording only. Note: {note}"""
para = llm(prompt)
if keeps_label(para, note):       # judge or human
    augmented.append((para, label))

# provenance, always
record = {"text": para, "source": "synthetic",
          "generator": MODEL_ID, "seed": note_id}
```

- Self-Instruct and Alpaca showed that a model can write its own training data; Evol-Instruct showed it can make that data harder. The **filtering step** is what separates a dataset from noise.
- **Model collapse**: Shumailov et al., Nature 2024 (cite this, not the earlier preprint): models trained recursively on generated data lose the tails of the distribution, generation after generation.
- Rules of the road: mark synthetic rows, cap their share, never evaluate on them, and keep a fresh human-written slice for validation.
- For Thai, generated text needs a native reader: fluency errors that an English judge model misses are common.

PART F2 · 2:25–2:40

RAG data preparation: chunks are the new documents

Retrieval-augmented generation moves preprocessing to inference time: how you split, embed, and index documents decides what the model can ever cite.

SOURCE DECKS

Information Retrieval and RAG, RAG section (SLP3 slides, Jurafsky and Martin, Nov 2025) · **Lecture 10: RAG and agents** (CS224N Winter 2026, Stanford).

web.stanford.edu/~jurafsky/slp3/slides · web.stanford.edu/class/cs224n

© the respective authors · shown unmodified · selected pages

IR plays a central role in modern LLMs

How to answer factual questions like

- Where is the Louvre Museum located?
- How to get a script l in latex?
- Where does the energy in a nuclear explosion come from?

LLMs Hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate up to 88% of the time!

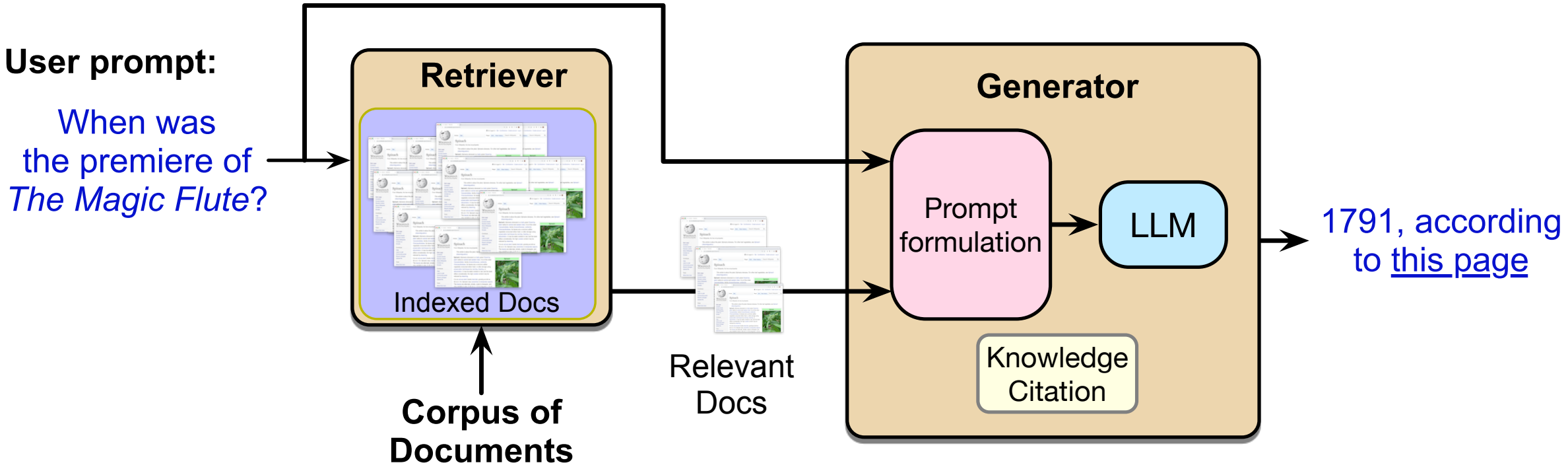
Dahl et al. (2024)

Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

Retrieval Augmented Generation (RAG)



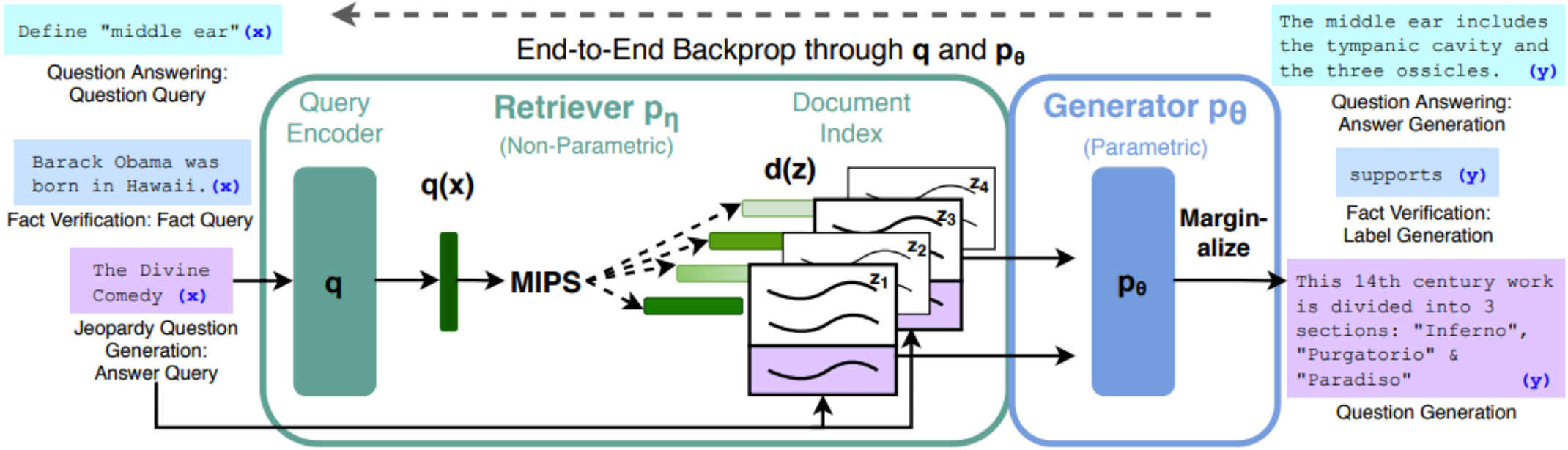
Basic RAG

Given a document collection D and a user query q

- Call a retriever to return top *k* passages
- Create a prompt that includes *q* and the passages
- Call an LLM with the prompt

RAG and it's open problems

Retrieval Augmented Generation (RAG) is very powerful!



See also works like REALM, DPR, ORQA etc..

[Lewis et al., 2021]

Problem: How many documents can we use?

The retriever is key – if we have to use only one document, then we have to get that right.

Why not get lots of documents and pass it to the LM?

Input Context

```
Write a high-quality answer for the given question using only the provided search
results (some of which might be irrelevant).

Document [1](Title: Asian Americans in science and technology) Prize in physics for
discovery of the subatomic particle J/ψ. Subrahmanyan Chandrasekhar shared...
Document [2](Title: List of Nobel laureates in Physics) The first Nobel Prize in
Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...
Document [3](Title: Scientist) and pursued through a unique method, was essentially
in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

Question: who got the first nobel prize in physics
Answer:
```

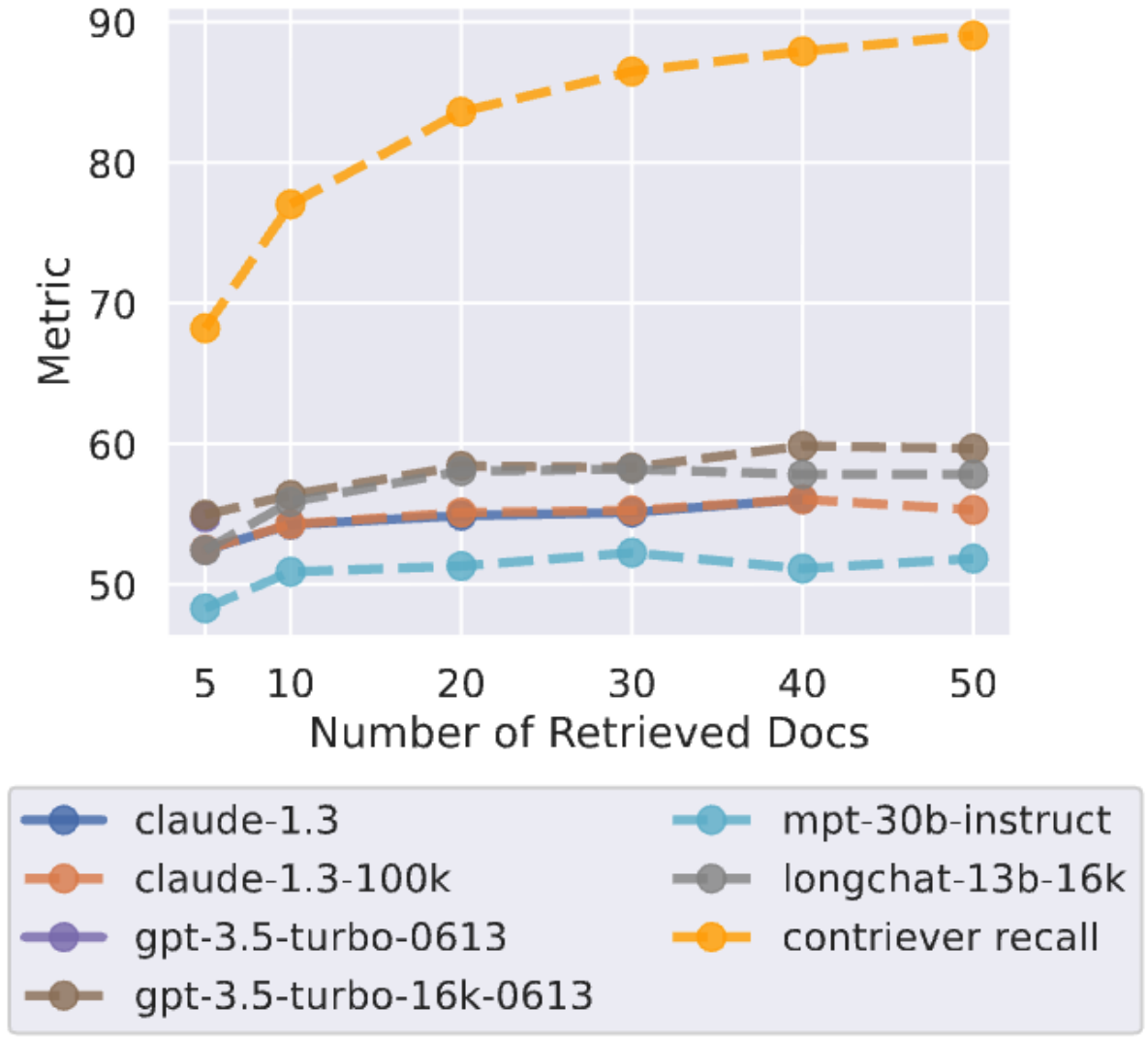
Desired Answer

```
Wilhelm Conrad Röntgen
```

In practice: LMs cant use many documents

Retriever performance (yellow) rises slowly to 90% recall

RAG performance (other lines) saturate very quickly – after 10-20 documents.



Fixed, recursive, sentence-window: pick one and measure

```
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter)
import tiktoken
enc = tiktoken.get_encoding("o200k_base")

splitter = RecursiveCharacterTextSplitter(
    chunk_size=400, chunk_overlap=60,        # in tokens
    length_function=lambda s: len(enc.encode(s)),
    separators=["\n\n", "\n", ". ", " ", ""])
chunks = splitter.split_text(document)

# keep provenance with every chunk
rows = [{"doc_id": d, "chunk_id": i,
         "text": c, "section": heading_of(c)}
        for i, c in enumerate(chunks)]
```

- **Recursive splitting** (paragraph, then line, then sentence, then space) is the common default; count length in **tokens of the embedding model**, not characters, and remember the Thai premium.
- **Overlap** keeps sentences that straddle a boundary retrievable at the cost of index size.
- Thai has no sentence punctuation: use `pythainlp.tokenize.sent_tokenize` as the sentence separator, otherwise the splitter falls through to spaces and breaks mid-clause.
- Store **document id, position, and section heading** with each chunk; citations and de-duplication depend on it.

Context-aware chunking: what helps, what is contested

- **Late chunking** (Gunther et al., Jina AI 2024): embed all tokens of the long document first, then pool per chunk, so each chunk keeps document context. No extra training.
- **Contextual retrieval** (Anthropic, Sep 2024): prepend a generated 50 to 100 token context to each chunk before embedding and BM25 indexing. Self-reported: from a 5.7% baseline retrieval-failure rate, contextual embeddings cut failures 35%, plus BM25 49%, plus reranking 67%.
- **Semantic chunking** (split where consecutive-sentence embedding distance jumps): **contested**. Qu, Tu and Bao (Findings of NAACL 2025) find the extra compute is not consistently justified over fixed-size chunks.

Evaluate chunking like any preprocessing choice: hold out real questions, measure retrieval recall per strategy, and report the tokenizer, chunk size, and overlap you used.

- Numbers above are the proposers' own; the course caveat from section 2.8 applies.
- Hybrid retrieval (BM25 plus dense) is where the tf-idf of class 1 quietly returns.

PART F3 · 2:40–3:00

Clinical text, learned era: context, features, de-identification

Mr. Huntington was treated for Huntington's disease at Huntington Hospital. Why context beats term lists, how learned de-identification works, and what changes when the model is a transformer.

SOURCE DECK

Lecture 8: Natural Language Processing, Part 2, 6.S897 / HST.956 Machine Learning for Healthcare, Spring 2019 · Peter Szolovits, MIT OpenCourseWare.
https://ocw.mit.edu/courses/6-s897-machine-learning-for-healthcare-spring-2019/

License: CC BY-NC-SA 4.0 · attribution page retained · third-party figures marked in the deck stay under their own terms

Outline

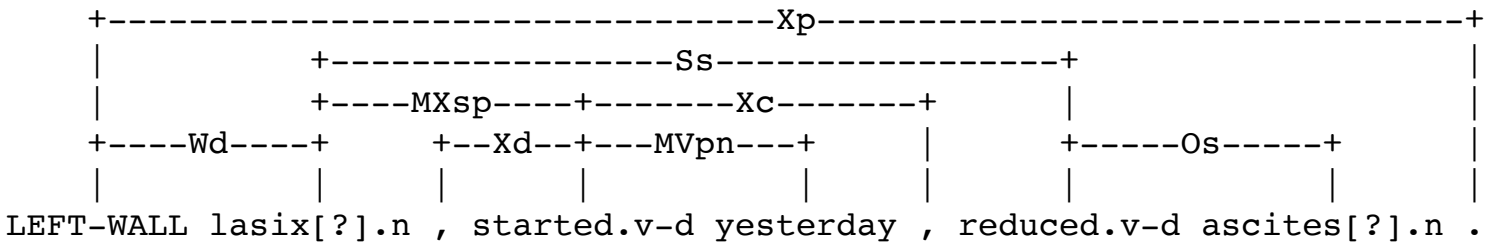
- Term spotting + handling negation, uncertainty
- ML to expand terms
- pre-NN ML to identify entities and relations
- language models
- Neural methods

The Importance of Context

- “Mr. Huntington was treated for Huntington’s Disease at Huntington Hospital, located on Huntington Avenue.”
  - Huntington
  - Huntington’s Disease
  - Mr. Huntington’s Disease
- “Atenalol was administered to Mr. Huntington.”
  - vs. “Atenalol was considered for control of heart rate.”
  - vs. “Atenalol was ineffective and therefore discontinued.”

Building Models

- Features of text from which models can be built
  - words, parts of speech, capitalization, punctuation
  - document section, conventional document structures
  - identified patterns and thesaurus terms
  - lexical context
    - ➡ all of the above, for n-tuples of words surrounding target
  - syntactic context
    - ➡ all of the above, for words syntactically related to target
    - E.g., “The lasix, started yesterday, reduced ascites ...”



(Output from Link Grammar Parser, w/o special medical dictionary)

Uzuner, Ö., Sibanda, T. C., Luo, Y., & Szolovits, P. (2008). A de-identifier for medical discharge summaries. Artificial Intelligence in Medicine, 42(1), 13–35. http://doi.org/10.1016/j.artmed.2007.10.001

Example of Features Available for Model

Mr. Blind is a 79-year-old white white male with a history of diabetes mellitus, inferior myocardial infarction, who underwent open repair of his increased diverticulum

263 266 "Mr."
TUI: T060,T083,T047,T048,T116,T192,T081,T028,T078,T077; SP-POS: noun; SEM: _modifier,_disease,_procparam;
CUI: C0024487,C0024943,C0025235,C0025362,C0026266,C0066563,C0311284,C0475209,C1384671,
C1413973,C1417835,C1996908,C2347167,C2349188; lptok: 6;
MeSH: C07.465.466,C10.292.300.800,C10.597.606.643,C14.280.484.461,C23.888.592.604.646,D12.776.826.750.530,
D12.776.930.682.530,E05.196.867.519,F01.700.687,F03.550.600,Z01.058.290.190.520;
267 468 "Blind is a 79-year-old white white...hsandpot Center." sent: nil;
267 272 "Blind"
TUI: T062,T047,T170; SP-POS: verb,adj,noun; SEM: _disease; CUI: C0150108,C0456909,C1561605,C1561606;
lptok: 1; MeSH: C10.597.751.941.162,C11.966.075,C23.888.592.763.941.162;
273 277 "is a" TUI: T185,T169,T078; SEM: _modifier; CUI: C1278569,C1292718,C1705423;
273 275 "is" SP-POS: aux,noun,adj; lptok: 2;
276 277 "a" SP-POS: det,noun,adj; lptok: 3;
278 289 "79-year-old" lptok: 4;
290 295 "white" TUI: T098,T080; SP-POS: noun,adj; SEM: _modifier; CUI: C0007457,C0043157,C0220938; lptok: 5;
296 301 "white" TUI: T098,T080; SP-POS: noun,adj; SEM: _modifier; CUI: C0007457,C0043157,C0220938; lptok: 6;
302 306 "male"
TUI: T032,T098,T080; SP-POS: adj,noun; SEM: _modifier,_bodyparam;
CUI: C0024554,C0086582,C1706180,C1706428,C1706429; lptok: 7;
307 311 "with" SP-POS: prep,conj; lptok: 8;
312 313 "a" SP-POS: det,noun,adj; lptok: 9;
314 342 "history of diabetes mellitus" TUI: T033; SEM: _finding; CUI: C0455488;

314 321 "history" TUI: T090,T170,T032,T033,T080,T077; SP-POS: noun; SEM: _modifier,_finding,_bodyparam; CUI: C0019664,C0019665,C0262512,C0262926,C0332119,C1705255,C2004062; lptok: 10; MeSH: K01.400,Y27;
322 324 "of" SP-POS: prep; lptok: 11;
325 333 "diabetes" TUI: T047; SP-POS: noun; SEM: _disease; CUI: C0011847,C0011849,C0011860; lptok: 12; MeSH: C18.452.394.750,C18.452.394.750.149,C19.246,C19.246.300;
334 342 "mellitus" lptok: 13;
342 343 "," lptok: 14;
344 374 "inferior myocardial infarction" TUI: T047; SEM: _disease; CUI: C0340305;
344 352 "inferior" TUI: T082,T054; SP-POS: noun,adj; SEM: _modifier; CUI: C0542339,C0678975; lptok: 15;
353 374 "myocardial infarction" TUI: T047; SEM: _disease; CUI: C0027051; MeSH: C14.280.647.500,C14.907.585.500;
353 363 "myocardial" TUI: T024,T082; SP-POS: adj; SEM: _modifier; CUI: C0027061,C1522564; lptok: 16; MeSH: A02.633.580,A07.541.704,A10.690.552.750;
364 374 "infarction" TUI: T046; SP-POS: noun; SEM: _disease; CUI: C0021308; lptok: 17; MeSH: C23.550.513.355,C23.550.717.489;
374 375 "," lptok: 18;
376 379 "who" SP-POS: pron; lptok: 19;
380 389 "underwent" SP-POS: verb; lptok: 20;
390 401 "open repair" TUI: T061; SEM: _procedure; CUI: C0441613;
390 394 "open" TUI: T082; SP-POS: adj,verb,adv; SEM: _modifier; CUI: C0175566,C1882151; lptok: 21;
395 401 "repair" TUI: T040,T169,T061,T052,T201; SP-POS: noun,verb; SEM: _finding,_procedure,_modifier,_bodyparam; CUI: C0043240,C0205340,C0374711,C1705181,C2359963; lptok: 22; MeSH: G16.100.856.891;
402 404 "of" SP-POS: prep; lptok: 23;
405 408 "his" SP-POS: noun,pron; lptok: 24;
409 418 "increased" TUI: T081,T169; SP-POS: verb,adj; SEM: _modifier; CUI: C0205217,C0442805,C0442808; lptok: 25;
419 431 "diverticulum" TUI: T190,T170; SP-POS: noun; SEM: _disease; CUI: C0012817,C1546602; lptok: 26; MeSH: C23.300.415;

11,146 annotations for this document of 1,518 tokens

Learning Models

- Given a target classification, build a machine learning model predicting that class
  - support vector machines (SVM)
  - classification trees
  - naive Bayes or Bayesian networks
  - artificial neural networks
  - ...
- class(word) = function ($feature_1$, $feature_2$, $feature_3$, ...)
  - sometimes, astronomically large (binary) feature set; SVM can deal with it
    - f_1 ... $f_{100,000}$: whether the word is “a”, “aback”, “abacus”, ..., “zymotic”
    - $f_{100,001}$ ...: whether word’s POS is “noun”, “verb”, “adj”, ...
    - $f_{100,100}$ ...: whether the word maps to CUI “C0000001”, “C0000002”, ...
    - $f_{3,000,000}$ ...: same as above, but for 1st, 2nd, 3rd word to right/left
    - $f_{6,000,000}$ ...: {lp-link, word} for 1st, 2nd, 3rd link in parse to right/left
    - ...

Using this model for de-identification

Table 6 Evaluation on authentic discharge summaries

Method	Class	Precision (%)	Recall (%)	*F*-measure (%)
Stat De-id	PHI	98.46	95.24	**96.82**
IFinder	PHI	26.17	61.98	36.80*
H + D	PHI	82.67	87.30	84.92*
CRFD	PHI	91.16	84.75	87.83*
Stat De-id	Non-PHI	99.84	99.95	**99.90**
IFinder	Non-PHI	98.68	94.19	96.38*
H + D	Non-PHI	99.58	99.39	99.48*
CRFD	Non-PHI	99.62	99.86	99.74*

The *F*-measure differences from Stat De-id in PHI and in non-PHI are significant at $\alpha = 0.05$.

Table 7 Evaluation of SNoW and Stat De-id on authentic discharge summaries

Method	Class	Precision (%)	Recall (%)	*F*-measure (%)
Stat De-id	PHI	98.40	93.75	**96.02**
SNoW	PHI	96.36	91.03	93.62*
Stat De-id	Non-PHI	99.90	99.98	**99.94**
SNoW	Non-PHI	99.86	99.95	99.90*

The *F*-measure differences from Stat De-id in PHI and in non-PHI are significant at $\alpha = 0.05$.

Courtesy of Elsevier, Inc., https://www.sciencedirect.com. Used with permission.

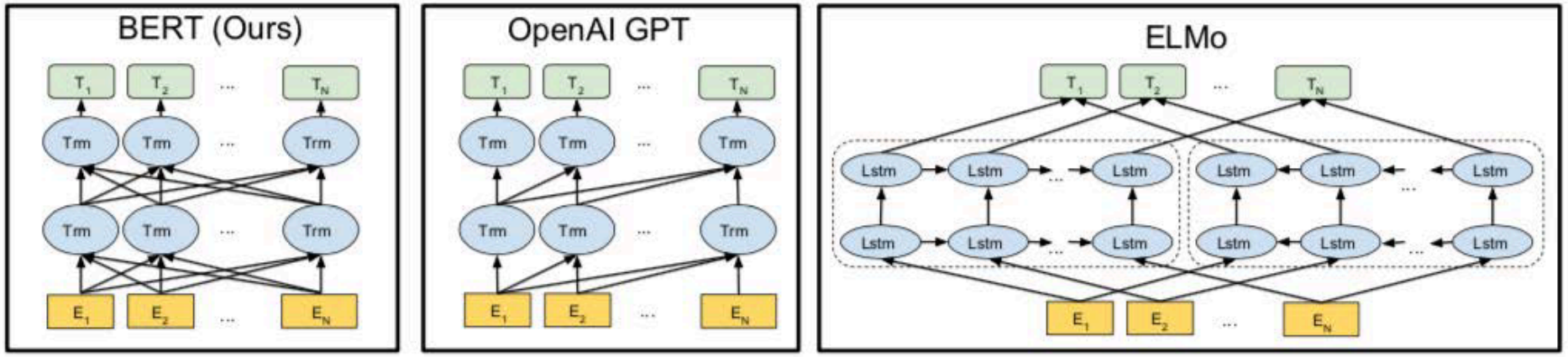
Uzuner, Ö., Sibanda, T. C., Luo, Y., & Szolovits, P. (2008). A de-identifier for medical discharge summaries. Artificial Intelligence in Medicine, 42(1), 13–35. http://doi.org/10.1016/j.artmed.2007.10.001

“Revolutionary Advances” in Embeddings

- The year 2018 has been an inflection point for machine learning models handling text (or more accurately, Natural Language Processing or NLP for short). Our conceptual understanding of how best to represent words and sentences in a way that best captures underlying meanings and relationships is rapidly evolving.
  —Jay Alammar (http://jalammar.github.io/illustrated-bert/ — *good tutorial*)

- Bidirectional LSTM applied to learn context-specific embeddings (ELMo)
- Transformer architecture — focus on attention mechanism
- Bidirectional Encoder Representations from Transformers (BERT)
- Generative Pre-Training (GPT-2) — transformer with multi-task training, huge corpus, huge model

BERT
Bidirectional Encoder Representations from Transformers



© Devlin et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see https://ocw.mit.edu/help/faq-fair-use/

Figure 1: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTM to generate features for downstream tasks. Among three, only BERT representations are jointly conditioned on both left and right context in all layers.

- Word-piece tokens
- Predict masked tokens (~15%)
- Predict next sentence
- Trained on 800M word Books, 2,500M word Wikipedia corpus

- Large performance improvement on many tasks

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018, October 10). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv*.

MIT OpenCourseWare
https://ocw.mit.edu

6.S897 / HST.956 Machine Learning for Healthcare
Spring 2019

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

From six regexes to a sequence labeller

```
# de-identification as token classification (NER)
from transformers import pipeline
deid = pipeline("token-classification",
                model="<a de-id model trained on
i2b2/n2c2>",
                aggregation_strategy="simple")
spans = deid(note)          # [{'entity_group': 'NAME',
'start': 4, ...}]
for s in sorted(spans, key=lambda x: -x["start"]):
    note = note[:s["start"]] + f"[{s['entity_group']}]" +
note[s["end"]:]

# then the regexes from class 1 as a safety net,
# then a human audit on a random sample
```

- The i2b2 2006 and 2014 and n2c2 challenges made de-identification a benchmarked **sequence-labelling** task; learned systems catch names in prose that no regex can.
- Learned systems also **miss** unpredictably, so the practical stack is model plus rules plus audit, with the audit sample size chosen from the recall you must guarantee.
- The corpus you de-identify is itself training data for a memorizing model (CS224N slide 56): membership inference and extraction are the LLM-era privacy risks, and deduplication is part of the defence.
- MIMIC-III/IV notes (Johnson et al.) are the reference de-identified corpora; access agreements are part of preprocessing too.

What changes for Thai notes, and what does not

- **Subword tokenizers absorb code switching**: Thai narrative and English abbreviations pass through one vocabulary, no segmenter required. The classical segmentation problem largely dissolves at the model boundary.
- **The premium bites hardest here**: a long Thai discharge summary can cost 2 to 4 times the tokens of its English equivalent; prompt and context budgets must be set per language (part D2).
- **De-identification models trained on English i2b2 notes do not transfer**: Thai names, Thai ID formats, Buddhist-era dates, and hospital names need Thai-labelled data or the regex safety net from class 1.
- **Legal frame**: Thailand's Personal Data Protection Act (PDPA, B.E. 2562) treats health data as sensitive personal data; de-identification and consent are preprocessing requirements, not afterthoughts, and sending raw notes to an external API is a data transfer decision.
- **Local and open-weight models** keep data on site; measure their Thai fertility and Thai quality before assuming they match English performance.
- The audit habit from image class 2 carries over: sample, read, count errors, and record the tokenizer, model, and de-identification recall in the datasheet.

Regulatory details change; verify the current PDPA guidance and your hospital's data-governance rules before deployment.

If you remember nothing else

- **1.** The tokenizer is part of the dataset: it fixes the vocabulary, the context budget, and the bill, and it charges Thai two to four times what it charges English.
- **2.** Web-scale preprocessing is subtraction: deduplicate at three levels, filter with rules you can read and classifiers you have audited, and keep the numbers.
- **3.** Every curation statistic and every selection score is fitted on training candidates only; benchmarks and validation data never influence them.
- **4.** Quality filters, judge models, and synthetic data all have a direction of bias; check who and which language is being removed before celebrating the gain.
- **5.** For clinical and Thai text, preprocessing is also compliance: de-identification recall, PDPA, and provenance belong in the methods section next to chunk size and tokenizer version.

Your corpus, audited

- **Task.** On 10,000 documents from your project (or a public Thai corpus): run MinHash deduplication at two thresholds, apply three heuristic quality rules ported to your language, and measure tokenizer fertility for two tokenizers. Report what each step removed, with five examples of removed documents you disagree with.
- **Contamination check.** Take your evaluation set and search your training corpus for 13-gram overlaps. Report the hit rate honestly, even if it is zero.
- **Reading.** Lee et al., Deduplicating training data makes language models better, ACL 2022 · Penedo et al., The FineWeb datasets, NeurIPS 2024 · Petrov et al., NeurIPS 2023 · Shumailov et al., Nature 2024 · Kapoor and Narayanan, Patterns 2023.
- **Question to sleep on.** If 92% of the web is "low quality" for a classifier trained to recognize textbooks, whose language and whose topics were in the 92%?

Source decks used in class 2

1 **CS224N Natural Language Processing with Deep Learning, Winter 2026, Stanford University.** Lecture 14, Tokenization and Multilinguality (guest lecture by Julie Kallini); Lecture 7, Pretraining; Lecture 8, Post-training; Lecture 10, RAG and agents; Lecture 11, Evaluation.
https://web.stanford.edu/class/cs224n/ · © the authors; shown unmodified with attribution; no explicit reuse license.

2 **EE-628 Training Large Language Models, Spring 2025, EPFL (LIONS)**, Volkan Cevher and Caglar Gulcehre. Lecture 3, Data; Lecture 4, Fine-tuning.
https://www.epfl.ch/labs/lions/teaching/ee-628-training-large-language-models/ · Creative Commons BY-NC-SA, as stated on each deck's license slide (retained).

3 **CS 329X Human-Centered LLMs, Stanford University**, Lecture 5, Data, Data, and Data (William Held).
https://web.stanford.edu/class/cs329x/ · © the author; shown unmodified with attribution.

4 **Speech and Language Processing, 3rd edition draft, lecture slides**, Dan Jurafsky and James H. Martin: Large Language Models introduction (Aug 2026 version, scaling and pretraining-data pages), Words and Tokens (BPE section), Large Language Models (Aug 2025), Information Retrieval and RAG (Nov 2025).
https://web.stanford.edu/~jurafsky/slp3/slides/ · © the authors, posted freely for teaching; shown unmodified.

5 **MIT OpenCourseWare, 6.S897 / HST.956 Machine Learning for Healthcare, Spring 2019**, Lecture 8, NLP Part 2 (Peter Szolovits). CC BY-NC-SA 4.0; OCW attribution page retained; marked third-party figures excluded from the license.

Demos, papers, and further reading

6 **Authored slides** (marked in the footer) were written for this course. Live demos used tiktoken (cl100k_base, o200k_base), the XLM-R SentencePiece tokenizer via Hugging Face tokenizers, the tokenizers library BPE trainer, datasketch MinHash/LSH, PyThaiNLP, and plain Python; translations of the demo sentence were written for this deck. Training-token figures on the D0 chart are as reported in the respective model papers and cards (BERT, GPT-3, Gopher, PaLM, Chinchilla, LLaMA, Llama 2, Llama 3, Qwen2.5).

7 **Papers cited on authored slides**: Petrov et al. NeurIPS 2023; Ahia et al. EMNLP 2023; Xue et al. ByT5 TACL 2022; Clark et al. CANINE TACL 2022; Lee et al. ACL 2022; Kandpal et al. ICML 2022; Penedo et al. FineWeb NeurIPS 2024; Rae et al. Gopher 2021; Zhou et al. LIMA NeurIPS 2023; Chen et al. AlpaGasus ICLR 2024; Liu et al. DEITA ICLR 2024; Xia et al. LESS ICML 2024; Li et al. IFD NAACL 2024; Shumailov et al. Nature 2024; Gunther et al. Late Chunking 2024; Anthropic Contextual Retrieval 2024; Qu, Tu and Bao, Findings of NAACL 2025; Kapoor and Narayanan Patterns 2023; Johnson et al. MIMIC Scientific Data 2016.

8 **Not slides, but recommended**: Stanford CS336 Language Modeling from Scratch, executable data lectures (stanford-cs336.github.io) · Hugging Face LLM course, chapter on tokenizers (Apache-2.0) · Dolma and FineWeb dataset papers for published filtering rules.

9 **Compilation note.** Curated by T. Intharah for the Research Master's module Data Preparation for ML and AI, Khon Kaen University, September 2026. For classroom use; each source retains its own license.